

Threshold Blind Signatures from CDH

Michael Reichle and Zoé Reinke

Department of Computer Science
ETH Zurich, Switzerland
`michael.reichle@inf.ethz.ch`
`zoe.reinke@inf.ethz.ch`

Abstract. Blind signatures are a versatile cryptographic primitive with many applications, especially in privacy-preserving technologies. Threshold blind signature schemes (TBS) enhance blind signatures with a signing procedure distributed among up to n signers to reduce the risk attached to the compromise of the secret key.

So far, TBS constructions over groups rely on strong assumptions, e.g., the algebraic group model (AGM) or interactive assumptions. In this work, we propose two TBS based on the computational Diffie-Hellman assumption (CDH) in the ROM:

- The first TBS over pairing-free groups without the AGM.
- The first round-optimal TBS over pairing-based groups that does not rely on interactive assumptions.

The starting point for both constructions is the pairing-based signature scheme derived from Boneh-Boyen IBE (Eurocrypt'04), and their recent blind signature variants by Katsumata et al. (Asiacrypt'23) and Klooß et al. (Asiacrypt'24) based on the decisional Diffie-Hellman assumption (DDH). We first remove the reliance on DDH in their schemes, then provide careful adaptations to the threshold setting.

Table of Contents

Threshold Blind Signatures from CDH	1
<i>Michael Reichle and Zoé Reinke</i>	
1 Introduction	3
1.1 Our Contribution	4
1.2 Technical Overview	4
1.3 Concurrent Work	9
2 Preliminaries	9
2.1 Assumptions and Information-theoretical Tools	9
2.2 Blind Signatures	11
2.3 Proof Systems	13
2.4 Pseudorandom Function	15
2.5 Commitment Schemes	15
2.6 Threshold Blind Signatures	16
3 BS: Pairing-free Blind Signature from CDH	17
3.1 Notation and Preparations	18
3.2 Construction	19
3.3 Security Analysis	21
4 Formal Security Proofs for BS	22
4.1 Correctness of BS	22
4.2 One-more Unforgeability of BS	25
4.3 Blindness of BS	32
5 TBS: Pairing-free Threshold Blind Signature from CDH	38
5.1 Construction	38
5.2 Security Analysis	39
6 Formal Security Proofs for TBS	41
6.1 Correctness of TBS	41
6.2 One-more Unforgeability of TBS	42
6.3 Blindness of TBS	49
7 pTBS: Pairing-based Threshold Blind Signature from CDH	50
7.1 Construction	50
7.2 Security Analysis	50
8 Formal Security Proofs for pTBS	52
8.1 Correctness of pTBS	52
8.2 OMUF-3 Security of pTBS	53
8.3 Blindness of pTBS	56

1 Introduction

Blind signatures are cryptographic building blocks involving two parties called the signer and the user. The signer is in possession of a key pair (sk, vk) , while the user holds a message m for which he wants to obtain a signature verifying under the signer's public key vk . The signer and the user run an interactive signing protocol, after which the signer should not have learnt the message m and the user should not be able to forge signatures. These two properties are called blindness and one-more unforgeability (OMUF). More formally, blindness guarantees that the signer is not able to link any message-signature pair to the issuing signing session. OMUF ensures that the user cannot compute more valid signatures for distinct messages than the number of completed signing sessions. Blind Signatures were developed in the context of electronic cash by Chaum [14], and have since found wide applications in privacy-preserving technologies.

Threshold blind signatures (TBS) are interactive protocols that keep the properties of blindness and OMUF, but distribute the signing process among multiple signers. In TBS schemes, the user interacts with $t + 1$ -out-of- n signers to obtain a valid signature. Importantly, OMUF ensures that even if the user colludes with up to t malicious signers, it remains hard to forge signatures. Threshold blind signature schemes can be used for the same purposes as blind signature schemes, but have the advantage that they diminish the risk attached to compromised secret keys.

Blind Signatures in Pairing-free Groups. Blind signature schemes over pairing-free groups are widely standard with many known tradeoffs (e.g., [35, 4, 1, 26, 24, 20, 25, 12, 13]). The first blind signature scheme in pairing-free groups was proposed by Pointcheval and Stern [35] in the random oracle model (ROM). Later, a construction achieving partial blindness was given by Abe and Okamoto [4]. Partial blindness allows the signer and user to agree on a common public message τ which is signed in conjunction with the hidden message m . While these constructions and later works following their template [24, 25] are elegant, their security was proven in the ROM under the restriction that at most $\text{polylog}(\lambda)$ -many signing sessions are concurrently opened, where λ denotes the security parameter. Indeed, afterwards efficient attacks were discovered against [35, 4] in the concurrent setting [40, 6].

Subsequently, several blind signatures with unlimited concurrency were proposed [1, 38, 26, 16, 20]. However, their security is only proven in both the ROM and the algebraic group model (AGM). It remained an open problem to construct blind signatures in pairing-free groups with unlimited concurrency, but without resorting to the AGM, until the work by Chairattana-Apirom et al. [13], hereafter denoted CTZ. CTZ presents two schemes:

- An efficient blind signature with constant signature and communication size under an interactive CT-CDH assumption (and in a slightly weaker model).
- A blind signature proven secure under the CDH assumption. However, the communication and signature size linearly depends on λ .

Subsequently, Klooß et al. [29] proposed a pairing-free blind signature scheme with constant signature size and linear communication size proven secure under a decisional assumption, namely DDH. This construction was later optimized by Klooß and Reichle to achieve both constant signature and communication size [28]. Also, Brandt et al. [11] provide a scheme with an (almost) tight security proof with constant signature and communication under DDH. However, the tightness comes at the cost of larger concrete sizes. In summary, while significant progress in this area was made in recent years, to this date no *efficient, concurrently secure* pairing-free blind signature scheme proven secure under a search assumption in the ROM (without the AGM or interactive assumptions) has been proposed.¹

Threshold Blind Signatures in Pairing-free Groups. In comparison to blind signatures, threshold blind signatures only recently gained more attention of the cryptographic community. The first blind threshold signature scheme for pairing-free groups, Snowblind [16] based on [38], relies on the AGM. Recently, Lehmann et al. [32] revisited the security definitions of TBS and proposed a hierarchy of security notions for OMUF. Also, [32] gives a variant of Snowblind [16] that satisfies the strongest security notion in their hierarchy, namely OMUF-3. There is no known TBS construction in pairing-free groups that does not rely on the AGM to date.

Blind Signatures in Pairing-friendly Groups. Over pairing-friendly groups, there are many constructions relying on algebraic models or interactive assumptions [8, 2, 19, 18, 22, 21]. Also, several

¹ A concurrent work [30] constructs efficient blind signatures from the discrete logarithm assumption in the ROM. We compare our work to theirs in Section 1.3.

constructions in the ROM from standard assumptions are known [7, 3, 12, 23]. The most efficient construction is given by [27] proven secure in the ROM under DDH, with signatures consisting of 2 group elements.

Threshold Blind Signatures in Pairing-friendly Groups. The first TBS schemes are based on pairings, introduced by Vo et al. [39] and Kuchta and Manulis [31]. Later, [32] gives variants that satisfy the stronger security notion OMUF-3. However, all constructions rely on an interactive variant of CDH.

Our Goal. In this paper, we develop *efficient* threshold blind signatures that are provably secure in only the ROM under the CDH assumption.

1.1 Our Contribution

We propose two threshold blind signatures, namely TBS and pTBS, proven secure under the CDH assumption in the ROM. Both constructions satisfy OMUF-3, the strongest notion in [32]’s hierarchy and are concretely efficient:²

- Our scheme TBS is over pairing-free groups and has signatures that consist of 10 group and scalar elements, while communication is linear in the security parameter λ for both schemes. For $\lambda = 128$, this corresponds to 320 Byte signatures. In comparison, the only known blind signature proven under CDH in the ROM, namely CTZ-3 [13], has signatures of size 10.5 KB.³
- Our scheme pTBS is over pairing-friendly groups and has signatures that consist of 2 group elements, i.e., pTBS preserves the signature size of the state of the art [27] in the threshold setting.

A comparison between our threshold blind signature schemes and other group-based constructions with OMUF-3 is given in Table 1. We also provide an overview of non-threshold blind signatures over pairing-free groups in Table 2.

Scheme	Assumption	Model	Moves	Total Communication	Signature
SB+ [16, 32]	DLog	AGM,ROM	7	$\Theta(t^2)(\lambda + \mathbb{G} + \mathbb{Z}_p)$	$1\mathbb{G} + 2\mathbb{Z}_p$
TBS (cf. Section 5)	CDH	ROM	6	$\Theta(\lambda + t)(\lambda + \mathbb{G} + \mathbb{Z}_p)$	$2\mathbb{G} + 8\mathbb{Z}_p$
tBlindBLS-3 [39, 32]	BOMDH	ROM	2	$(t + 2) \cdot \mathbb{G}$	$1\mathbb{G}$
pTBS (cf. Section 7)	CDH	ROM	2	$\Theta(\lambda + t)(\lambda + \mathbb{G} + \mathbb{Z}_p)$	$2\mathbb{G}$

Table 1: We compare threshold blind signatures over groups that satisfy OMUF-3. Here, BOMDH refers to the interactive bilinear one-more Diffie-Hellman assumption. Schemes above (resp. below) the line rely on pairing-free (reps. pairing-friendly) groups.

Both our schemes are based on the signature derived from Boneh-Boyen IBE [9], hereafter BB⁴, which is secure under CDH. In more detail, our scheme pTBS is based on the blind signature [27], itself based on BB, of which we provide a careful threshold adaptation. Our scheme TBS is based on [29], itself an adaptation of [27] without pairings. For the latter, we note that [29] crucially relies on the DDH assumption in their security argument. We first propose a blind signature BS that we prove secure under CDH, then adapt it to the threshold setting.

1.2 Technical Overview

First, let us discuss our pairing-free scheme. Our starting point is the blind signature scheme presented by [29]. Recall that their scheme is secure in the ROM under the DDH assumption over a pairing-free group $\mathbb{G}$ of prime-order p . We employ additive notation for $\mathbb{G}$. We will briefly present the main

² We adapt OMUF-3 to partial blindness and consider forgeries for distinct messages, i.e., we do not consider strong unforgeability. Also, we note that our scheme pTBS is proven under an analogue of CDH over pairings.

³ A concurrent work constructs blind signatures from DLog (cf. Footnote 1).

⁴ We note that there is another signature scheme by Boneh-Boyen [10]. These are distinct schemes.

Scheme	Assumption	Moves	Communication	Signature
Cl-Schnorr [20]	OMDL, mROS	3	$2\mathbb{G} + 3\mathbb{Z}_p$	$1\mathbb{G} + 1\mathbb{Z}_p$
Abe [1, 26]	DLog	3	$\lambda + 3\mathbb{G} + 6\mathbb{Z}_p$	$2\mathbb{G} + 6\mathbb{Z}_p$
TZ [38]	DLog	3	$2\mathbb{G} + 4\mathbb{Z}_p$	$4\mathbb{Z}_p$
Snowblind [16]	DLog	3	$2\mathbb{G} + 4\mathbb{Z}_p$	$1\mathbb{G} + 2\mathbb{Z}_p$
CTZ-1 [13]	CT-OMCDH	4	$5\mathbb{G} + 5\mathbb{Z}_p$	$1\mathbb{G} + 4\mathbb{Z}_p$
CTZ-2 [13]	CT-OMCDH	5	$5\mathbb{G} + 5\mathbb{Z}_p$	$1\mathbb{G} + 4\mathbb{Z}_p$
CTZ-3 [13]	CDH	4	$\Theta(\lambda)(\lambda + \mathbb{G} + \mathbb{Z}_p)$	$\Theta(\lambda)(\lambda + \mathbb{G} + \mathbb{Z}_p)$
KRW [29]	DDH	4	$\Theta(\lambda)(\lambda + \mathbb{G} + \mathbb{Z}_p)$	$2\mathbb{G} + 5\mathbb{Z}_p$
BHCR [11]	DDH	4	$37\mathbb{G} + 40\mathbb{Z}_p$	$10\mathbb{G} + 29\mathbb{Z}_p$
KR-1 [28]	DDH	4	$10\mathbb{G} + 9\mathbb{Z}_p$	$1\mathbb{G} + 5\mathbb{Z}_p$
KR-2 [28]	DDH	5	$10\mathbb{G} + 9\mathbb{Z}_p$	$1\mathbb{G} + 6\mathbb{Z}_p$
KLR [30]	DLog	4	$\Theta(\lambda)(\lambda + \mathbb{G} + \mathbb{Z}_p)$	$6\mathbb{Z}_p$
BS (cf. Section 3)	CDH	4	$\Theta(\lambda)(\lambda + \mathbb{G} + \mathbb{Z}_p)$	$2\mathbb{G} + 8\mathbb{Z}_p$

Table 2: Comparison of concurrently-secure and pairing-free blind signature schemes. The schemes above the line are proven to be secure in the ROM and the AGM, while those below the line only rely on the ROM.

characteristics of their construction that are relevant for our work. Then, we describe our techniques to remove the reliance on the DDH assumption in [29] and to build a threshold version of our scheme.

Starting Point. The blind signature scheme from [29] is based on a digital signature scheme derived from Boneh-Boyen’s IBE [9]. That is, a BB signature σ consists of two group elements $S = (S_1, S_2)$, where S_1 and S_2 are of the following form:

$$S_1 = uV + s(\overline{m}U + H), \quad S_2 = sG. \quad (1)$$

Here, G is a generator for $\mathbb{G}$, the element $u \in \mathbb{Z}_p$ forms the secret key, and $U = uG \in \mathbb{G}$, $H \in \mathbb{G}$ and $V \in \mathbb{G}$ are part of the public key. Also, $\overline{m} = H(m)$ is the hash of the message m to be signed with randomness $s \xleftarrow{\$} \mathbb{Z}_p$. Verification of this signature relies on pairings. However, as observed in [29], this reliance can be removed by using a Σ -protocol to prove that $\mathbf{S} = (S_1, S_2)$ is of the above form.

In order to turn this scheme into a blind signature scheme, the signing process is split among a user and a signer. For the purpose of hiding the (hashed) message from the signer, the user sends the signer a Pedersen commitment [34] $C = \overline{m}U + tG$ (with $t \xleftarrow{\$} \mathbb{Z}_p$) to $\overline{m}$ along with a proof π_{Ped} certifying that the user knows an opening for commitment C . The signer then homomorphically generates a blinded BB (pre-)signature $\mathbf{T} = (uV + s(C + H), sG)$ on the commitment C , from which the user derives a valid signature $\mathbf{S}$ by eliminating the commitment’s randomness t implicitly contained in $\mathbf{T}$. For blindness, the user rerandomizes $\mathbf{S}$. Apart from the pre-signature $\mathbf{T}$, the signer also issues a proof that $\mathbf{T}$ is well-formed via an appropriate Σ -protocol. The user also adapts the Σ -protocol transcript to show that $\mathbf{S}$ is distributed as in Eq. (1) by leveraging linear properties of the Σ -protocol, randomizes the transcripts, and lastly compiles them into a non-interactive proof π via Fiat-Shamir.⁵ The signature consists of the randomized BB signature $\mathbf{S}$ and the randomized proof π .

To facilitate the proof of one-more unforgeability, the proof π is implemented as an OR-proof for the statement that given a tuple $\mathbf{D} = (D_1, D_2, D_3)$ it holds that

1. either the BB signature $\mathbf{S}$ is well-formed, i.e., satisfies Eq. (1), or
2. the tuple $\mathbf{D}$ forms a valid DDH tuple.

Roughly, the first statement enforces that the adversary is forced to provide well-formed signatures, whereas the second statement serves as a simulation trapdoor in the OMUF proof. Further, the tuple $\mathbf{D}$ allows to achieve partial blindness by deriving $\mathbf{D}$ from a random oracle evaluated on the common message τ . As noted before, the blind signature by [29] sketched above is secure in the ROM under the DDH assumption.

Towards a CDH-based Blind Signature. We aim to prove the scheme by [29] secure under the CDH assumption. To this end, we first replace the second statement of the OR-proof (i.e., $\mathbf{D}$ is a valid DDH

⁵ These steps are closely interwoven in their execution.

tuple) with a statement claiming knowledge of the discrete logarithm of a value $W = H_{\text{DLog}}(\tau)$ which again is derived by hashing the common message τ . This modification eliminates the DDH tuple from the scheme, similar to [13].

However, this change alone is not enough to prove security under CDH. In order to understand the issue, let us give a high-level overview of the OMUF proof by [29]. Recall that in the OMUF game, the adversary obtains access to a signing oracle and is asked to provide $\ell + 1$ valid signatures for distinct messages, while only finishing up to ℓ signing sessions. Note that the pre-signature $\mathbf{T}$ (from which the user derives a BB signature $\mathbf{S}$) is revealed to the user *before* the signing session is finalized. However, the proof strategy by [29] relies on the puncturing strategy from BB: the adversary is forced to provide a forgery for a message m^* for which the verification key vk is punctured. That is, the game can issue signatures for any message $m \neq m^*$ and the signature on m^* which is provided by the adversary allows to solve CDH. By the pigeon-hole principle, the adversary will never *finalize* a signing session where m^* is committed in the Pedersen commitment C with sufficient probability. It is still possible though that an *incomplete* session requires simulating a BB pre-signature $\mathbf{T}$ for m^* which is not possible if the verification key vk is punctured. Under the DDH assumption, the BB pre-signatures $\mathbf{T}$ look random as long as the corresponding session is never finished. As this is true for sessions with m^* , it is sufficient to simply send a random tuple $\mathbf{T}$ in this case.

Hiding $\mathbf{T}$ via Pedersen Commitments. As we aim to avoid the DDH assumption, we cannot argue in this manner. Instead, we randomize the pre-signatures $\mathbf{T}$ sent by the signer via linearly homomorphic commitments for $\mathbb{Z}_p$. First, we observe that the second element T_2 of a BB pre-signature $\mathbf{T} = (T_1, T_2)$ is already uniform. Therefore, it suffices to hide T_1 until the signing session is finished. To do so, we let the signer send $T_{\$,1} = T_1 + dG$ to the user *along with* a Pedersen commitment $C_S = \text{Com}(d, r)$, instead of sending T_1 in plain. In the last message of the signing session, the signer opens the commitment C_S . Observe that now, the BB pre-signature $\mathbf{T}_\$ = (T_{\$,1}, T_2)$ simply consists of two uniform group elements until C_S is opened. Further, as Pedersen commitments are statistically hiding, we can argue that unfinished signing sessions leak no information.

An additional detail is that in order to blind the proof π , the user must know the BB signature $\mathbf{S}$ as it is part of the statement proven by π . As the user needs to randomize π already in the second round, but learns $\mathbf{S}$ only in the very last move, this causes an issue. To resolve this, we let the user also blind the proof π with the randomized term $S_{\$,1}$. All statements are linear, therefore it is possible to recover an appropriate Fiat-Shamir proof π in the end (at the cost of a second Pedersen commitment).

Before we continue, we note that employing commitments to avoid leaking critical information from unfinished signing sessions is not new to our work. Chairattana-Apirom et al. [13] employ homomorphic commitments with a special equivocation property and Brandt et al. [11] employ homomorphic dual-mode commitments for this purpose. Neither choice is an option for us. The equivocation property in [13] relies on the underlying signature having a specific form which is not the case for BB signatures. On the other hand, the dual-mode commitments in [11] rely on DDH.

We observe that in our case, any linear commitment that is binding and hiding under CDH is sufficient. Concretely, we use Pedersen for efficiency and the signature consists of only an additional 3 $\mathbb{Z}_p$ elements (after some optimizations) compared to [29].

Complications due to Partial Blindness. Similar to [29], our scheme satisfies partial blindness which leads to further complications in the OMUF proof. To understand the issue, recall that partial blindness allows the adversary to finish arbitrarily-many sessions for common messages τ that are distinct to the forgeries' common message τ^* . In particular, that means the simulation must provide BB pre-signatures for arbitrary messages within signing sessions with common messages $\tau \neq \tau^*$ (that will potentially be finished later on). Again, under DDH it is possible to simulate these sessions by outputting random tuples $\mathbf{T}$.

However, in our case, this is not possible as we want to avoid DDH. We observe that, for $m = m^*$ with $\tau \neq \tau^*$, the game might have to open the commitment C_S to the pre-signatures $\mathbf{T}$ in the last move of the protocol, de-randomizing the random looking pre-signature $\mathbf{T}_\$$. The game therefore has to send correctly distributed BB pre-signatures $\mathbf{T}$ for m^* if $\tau \neq \tau^*$. As vk is punctured for message m^* however, the game is not able to compute $\mathbf{T}_\$$ honestly via the secret key and the proof fails. To resolve this issue, we decouple signing sessions with distinct common messages τ by leveraging the symmetry in BB signatures. Roughly, observe that the signer can either issue valid signatures via v or u , the DLog of V or U , respectively. We cannot choose a τ -dependent U as its discrete logarithm u forms the signing key (which must work for every common message τ). However, we can choose V based on the

common message τ which allows us to simulate BB signatures *even* for the punctured message m^* . We refer to the main body for more details.

Ensuring that $\mathbf{S}$ is Well-formed. Recall that we replaced the DDH tuple $\mathbf{D}$ with a random group element W . In [29], the tuple $\mathbf{D}$ serves as a “toggle” for the proof π . That is, if $\mathbf{D}$ is a DDH tuple, then the game can simulate the proof π via the discrete logarithms in $\mathbf{D}$. Otherwise, if $\mathbf{D}$ is not a DDH tuple, then the adversary must provide valid BB signatures $\mathbf{S}$. Under the DDH assumption, [29] show that the adversary must provide valid BB signatures in its forgeries, even when the game simulates the BB pre-signatures and uses the $\mathbf{D}$ -branch of the OR-proof π , allowing to solve CDH.

We cannot employ this argument, as we want to avoid reliance on DDH. Instead, we ensure that the adversary provides valid BB forgeries via a forking-based argument, following the proof strategy by [13]. That is, if the BB forgeries are invalid, then we show that the adversary must break the DLog assumption (either by breaking binding of the homomorphic commitment or by computing the discrete logarithm w of W). In summary, the protocol looks as follows:

1. The user sends a Pedersen commitment C to $\bar{m} = H(m)$ and a proof π that certifies knowledge of an opening of C .
2. The signer computes a BB pre-signature $\mathbf{T} = (T_1, T_2)$ from the commitment C and masks T_1 via $T_{\$,1} = T_1 + dG$ over $\mathbb{G}$ with a random mask dG . The signer commits to d in a homomorphic commitment C_S , and sends $(\mathbf{T}_{\$}, C_S)$ to the user, where $\mathbf{T}_{\$} = (T_{\$,1}, T_2)$. Also, the signer initiates a Σ -protocol proof that either $\mathbf{T}$ is well-formed or it knows the discrete logarithm of $W = H_{DLog}(\tau)$.
3. The user computes a masked BB signature $\mathbf{S}_{\$}$ from $(T_{\$,1}, T_2)$, where the first term is still additively masked by dG . Next, the user randomizes the commitment C_S , the blinding factor d committed in C_S and the obtained BB signature in $\mathbf{S}_{\$}$. Then, the user adapts and blinds the (incomplete) Σ -protocol transcript for the signature $\mathbf{S}_{\$}$, following the techniques in [29], and compiles the transcript via Fiat-Shamir. It blinds the obtained challenge c and sends the blinded c to the signer.
4. The signer computes the Σ -protocol response z and outputs z and an opening for C_S .
5. The user finally derives an actual BB signature $\mathbf{S}$ by removing the masking term dG from $\mathbf{S}_{\$}$. He then adapts and blinds the final part of the Σ -protocol transcript and outputs the randomized Fiat-Shamir proof π , BB signature $\mathbf{S}$, commitment C_S and opening (d, r) for C_S .

With these adaptations, we can show security of our blind signature under CDH. Also, blindness follows similar to [29] as the Pedersen commitment and the masking term d are rerandomized by the user.

Towards a Threshold Signing Protocol. Finally, let us describe how to construct a threshold blind signature TBS based on our blind signature. To distribute signing among $t + 1$ -out-of- n parties, a natural first step is to distribute the signing key for the BB signature $\mathbf{T}$. That is, the key generation algorithm computes n Shamir shares u_i of secret key u , whereby any subset of $t + 1$ shares allows to linearly reconstruct u . To issue a BB signature in a distributed manner, each signer issues a partial signature following Eq. (1), that is, it sets

$$(T_{1,i}, T_{2,i}) = (\lambda_i u_i V + s_i(\bar{m}U + H), T_2 = s_i G), \quad (2)$$

where λ_i is the i -th Lagrange coefficient for the chosen set $\mathcal{S}$ of signers and $s_i \xleftarrow{\$} \mathbb{Z}_p$. As above, we can leverage the linearity of BB to (partially) sign a commitment C to obtain a partial pre-signature $\mathbf{T}_i$. Additionally, each signer $i \in \mathcal{S}$ issues a Σ -protocol proving that its partial signature $\mathbf{T}_i$ on C is well-formed or it knows the discrete logarithm of $W = H_{DLog}(\tau)$. To ensure unforgeability under CDH, the tuple $\mathbf{T}_i$ is again blinded to $\mathbf{T}_{\$,i} = \mathbf{T}_i + (d_i G, 0)$ and the randomness d_i is committed in a Pedersen commitment $C_{S,i}$. As the Σ -protocol and the BB signature are linear, the user simply adds up the partial signatures and Σ -protocol transcripts to obtain a blind signature, following the blinding steps outlined above.

Proving Security. As an issued TBS signature is distributed exactly as in our non-threshold variant BS, we show blindness of TBS leveraging blindness of BS. Showing unforgeability is more involved. Here, in order for our overall proof strategy to work, we must always be able to simulate one of the two branches of the OR-proof and, further, we must be able to switch freely between both modes of simulation, while retaining soundness of the OR-proof. To achieve this, we employ a commit-and-open protocol for the challenge of the DLog-branch of the Σ -protocol: each signer i commits to the challenge $c_{1,i}$ it employs for simulating the DLog-branch in $\mathbf{cm}_i$, then opens $\mathbf{cm}_i$ to $c_{1,i}$ after receiving $(\mathbf{cm}_j)_j$ from the other signers j . This gives the simulator enough leverage to program the aggregated challenge

$c_1 = \sum_j c_{1,j}$ in the signing protocol, allowing to simulate either branch of the OR-proof later on. Further, we show that revealing partial signatures retains unforgeability, i.e., even given access to a signing oracle for partial signatures, it remains hard to forge BB signatures if *no* partial signature for the forgery’s message was issued.

However, another complication arises since partial signatures may leak additional information in *incomplete* signing sessions. Recall that in OMUF-3 (cf. [32]), such incomplete sessions do not count towards the number of forgeries the adversary $\mathcal{A}$ must supply; in some sense such sessions may give “free” information to $\mathcal{A}$. Here, the leaked information is non-trivial: if signer i opens its commitment $C_{S,i}$ to blinding term d_i in the last round, it is possible to recompute $\mathbf{T}_i = \mathbf{T}_{\mathbb{S},i} - (d_i G, 0)$, from which $\mathcal{A}$ can recompute a partial BB signature $\mathbf{S}_i$ (cf. Eq. (2)). This is possible, even if another honest signer i' in the session did not yet complete the session. As a consequence, the simulator must supply correctly distributed partial BB pre-signatures even for sessions with m^* . This conflicts with the puncturing of vk on m^* .

To remedy this, we can employ a masking technique, as in [32, tBlindBLS-3, Figure 3]: the signer’s last round message is masked with a zero share Δ_i derived from evaluation(s) of a pseudorandom function (PRF) with pairwise-shared seeds. This ensures that the final round does not leak anything beyond the aggregated response. Here, it is important that the PRF is evaluated on fresh input. We therefore evaluate it on the tuple $(c_{1,j})_j$: since an honest signer’s challenge $c_{1,i}$ has high min-entropy, the tuple must be fresh in each session. In our scheme TBS, we mask the signer’s *full* message in the last round, including the Σ -protocol response z_i (similar to, e.g., [17, 32]) and the opening for $C_{S,i}$ (similar to [33]).

With these modifications, we can follow the proof strategy of our (non-threshold) blind signature, carefully adapting the proof to the distributed setting. Note that instead of applying the masking-based technique directly, we could employ the generic transformation proposed by [32, Figure 5] to obtain OMUF-3 security. However, this would increase the number of rounds by 1 and communication in the last round by a factor $t + 1$.

Let us remark that our techniques to thresholdize BS do not rely on any specific structure of BS. Applying the above modifications (i.e., commit-and-open for the challenge and masking the full response) to other AGM-free blind signatures (e.g., [28, 11]), combined with our proof technique (i.e., using the commit-and-open protocol to control the honest Σ -protocol branch), likely yields AGM-free threshold blind signatures with other efficiency/security tradeoffs.

Our Pairing-based Variant. Our pairing-based scheme pTBS is derived from the pairing-based blind signature by [27] proven under DDH—which is also based on BB and the basis for [29]. Roughly, the structure of the signatures is (almost) as in Eq. (1) and blind issuance also relies on signing a user’s commitment C to message m via BB. As above, the user supplies a proof π_{Ped} to ascertain well-formedness of C . Here, it is simple to obtain a scheme based on CDH: we replace the DDH-based proof system in [27] with the one from [29]. To issue signatures in a threshold manner, each signer signs the user’s commitment C with its partial key as in Eq. (2). Note that the scheme is round-optimal, as the pairing allows for verification (therefore no Σ -protocol must be issued). However, proving unforgeability in the threshold setting is more subtle.

As before, partial signatures may leak additional information without having completed a full signing session. Again, we can employ masking to remedy this: each signer masks its partial signature $\mathbf{T}_i$ with a zero share Δ_i . Here, the PRF-input based on which Δ_i is derived cannot have high min-entropy as the input must be (1) consistent across all signers and (2) established without communication (to retain round-optimality). Therefore, as in [32, tBlindBLS-3], we evaluate the PRF on the user’s message (C, π_{Ped}) to be signed.

Note that there is a subtlety since the PRF-input might not be fresh in different signing sessions: On repeated input (C, π) , the same mask Δ_i is reused as it is derived deterministically from (C, π) . However, the output $\mathbf{T}_i$ might differ (as signing is randomized). Therefore, the adversary may learn, e.g., differences of partial signatures $\mathbf{T}_i$ on message m^* by cancelling out the masks Δ_i . It seems difficult to both puncture the verification key and simulatenously, handle the simulation of partial signatures in a consistent manner. Our idea is to instead *derandomize* signing: each signer i computes its randomness for the partial signature $\mathbf{T}_i$ based on a PRF-evaluation on (C, π) , i.e., the same input that is used to generate the zero-shares. Therefore, on repeated queries, the adversary learns no additional information. On fresh queries, the zero share Δ_i again conceals partial signatures, which allows us to prove security of pTBS. Note that as [32, tBlindBLS-3] is based on BLS, this subtle issue does not appear in their proof (as BLS is already deterministic).

1.3 Concurrent Work

The concurrent work [30] constructs blind signatures in the ROM from DLog. We give a brief comparison with our construction BS (cf. Section 3):

- *Efficiency.* In terms of efficiency, both constructions are comparable. The communication is dominated by some proof sent by the user, and signatures are $6\mathbb{Z}_p$ elements (compared to $8\mathbb{Z}_p$ and $2\mathbb{G}$ elements in our construction).
- *Assumptions.* Both constructions are based on a standard search assumption over groups. However, DLog is a weaker assumption than CDH.
- *Techniques.* We obtain our construction by adapting [29] and carefully hiding information from incomplete sessions via homomorphic commitments to avoid DDH. Conceptually, we show that [29] is provable under CDH with minor adaptations. In contrast, [30] builds on [28] and replaces the underlying Σ -protocol in [28] with a Σ -protocol that enables a proof based on DLog instead of DDH.

The focus of our work is threshold blind signatures from BB. While we believe that our approach also allows to thresholdize works like [28, 30], this is out of scope.

2 Preliminaries

Notation. The security parameter is denoted by λ . Throughout, $\mathbb{G}$ denotes a group of prime order p and $G \in \mathbb{G}$ denotes a generator of $\mathbb{G}$. We use additive notation and denote group elements like G with uppercase letters, while scalars $x \in \mathbb{Z}_p$ are denoted by lowercase letters. Vectors of group elements and vectors of scalars are denoted by bold letters (e.g., $\mathbf{G}$ or $\mathbf{x}$). We denote by $y \leftarrow \mathcal{A}(x)$ a probabilistic algorithm $\mathcal{A}$ running with input x and producing output y . To express that a value v is assigned to a variable x , we write $x := v$. Also, $x \xleftarrow{\$} \mathcal{D}$ means that x is sampled uniformly at random from set $\mathcal{D}$, if $\mathcal{D}$ is a set, and that x is sampled from distribution $\mathcal{D}$, if $\mathcal{D}$ is a distribution. For an interactive protocol run by parties A and B on inputs in_A and in_B respectively, we use the notation $A(\text{in}_A) \longleftrightarrow B(\text{in}_B)$. To express that an interactive protocol is run by a set of parties $\mathcal{S}$ each running an instantiation of algorithm A with respective inputs in_{A_i} and a party running a different algorithm B with input in_B , we write $\{A(\text{in}_{A_i})\}_{i \in \mathcal{S}} \longleftrightarrow B(\text{in}_B)$. In specifications of algorithms, we write **req** C to state that, if the condition C is not satisfied, the algorithm returns $\perp$. To signify, in a specification of a game, that the game returns 0 if a condition C is met, we write **abort** **if** C .

2.1 Assumptions and Information-theoretical Tools

Denote by GrGen a PPT algorithm that on input 1^λ returns a group description $\mathcal{G} = (\mathbb{G}, p, G)$, where p is the prime order of $\mathbb{G}$ and $G \in \mathbb{G}$ is a generator. Additionally, let PGen(1^λ) denote a PPT algorithm that on input 1^λ , outputs $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2)$, where $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are cyclic groups of prime order p , G_1 and G_2 generate $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a bilinear mapping.

Assumptions. The DLog assumption expresses that given a group $\mathbb{G}$, a generator G and a uniform group element $X \in \mathbb{G}$, it is hard to compute x such that $X = xG$. The CDH assumption states that given group $\mathbb{G}$, generator G and group elements $A = aG$ and $B = bG$, it is hard to compute $C = abG$. We denote by co-DLog and co-CDH their natural analogues over pairings. We may write DLog and CDH for short if clear by context. These are formally defined below.

Definition 1 (DLog Assumption). *The discrete logarithm (DLog) assumption holds for GrGen if for any PPT adversary $\mathcal{A}$, it holds that*

$$\text{AdvDL}_{\mathcal{A}}^{\text{GrGen}}(\lambda) := \Pr[x \leftarrow \mathbb{Z}_p, x' \leftarrow \mathcal{A}(G, xG) : x = x'] = \text{negl}(\lambda),$$

where $(\mathbb{G}, p, G) \leftarrow \text{GrGen}(1^\lambda)$.

Definition 2 (CDH Assumption). *The computational Diffie-Hellman (CDH) assumption holds for GrGen if for any PPT adversary $\mathcal{A}$, it holds that*

$$\text{AdvCDH}_{\mathcal{A}}^{\text{GrGen}}(\lambda) := \Pr[a, b \leftarrow \mathbb{Z}_p, C \leftarrow \mathcal{A}(G, aG, bG) : C = (a \cdot b)G] = \text{negl}(\lambda),$$

where $(\mathbb{G}, p, G) \leftarrow \text{GrGen}(1^\lambda)$.

Definition 3 (co-DLog Assumption). The co-discrete logarithm (co-DLog) assumption holds for PGen if for any PPT adversary $\mathcal{A}$, it holds that

$$\text{AdvDL}_{\mathcal{A}}^{\text{PGen}}(\lambda) := \Pr[x \leftarrow \mathbb{Z}_p, x' = \mathcal{A}(G_1, G_2, xG_1, xG_2) : x = x'] = \text{negl}(\lambda),$$

where $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2) \leftarrow \text{PGen}(1^\lambda)$.

Definition 4 (co-CDH Assumption). The pairing-based computational Diffie-Hellman (co-CDH) assumption holds for PGen if for any PPT adversary $\mathcal{A}$, it holds that

$$\text{AdvCDH}_{\mathcal{A}}^{\text{PGen}}(\lambda) := \Pr[a, b \leftarrow \mathbb{Z}_p, C = \mathcal{A}(G_1, G_2, aG_1, aG_2, bG_1, bG_2) : C = abG_1] = \text{negl}(\lambda),$$

where $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2) \leftarrow \text{PGen}(1^\lambda)$.

Forking Lemma. We will make use of the Forking Lemma introduced by [5]. Roughly, it ensures that if an event occurs with good probability, then with good probability (albeit for a square-root loss) the event will also occur if the adversary is run again with changed random oracle outputs as inputs.

Lemma 1 (Forking Lemma). Fix an integer $q \geq 1$ and a set H of size $h \geq 2$. Let $\mathcal{A}$ be a randomized algorithm that on input $x, h_1, \dots, h_q$ returns a pair, the first element of which is an integer in the range $0, \dots, q$ and the second element of which we refer to as a sideoutput. Let IG be a randomized algorithm that we call the input generator. The accepting probability of $\mathcal{A}$, denoted acc , is defined as the probability that $J \geq 1$ in the experiment

$$x \xleftarrow{\$} \text{IG}; h_1, \dots, h_q \xleftarrow{\$} H; (J, \sigma) \xleftarrow{\$} \mathcal{A}(x, h_1, \dots, h_q).$$

The forking algorithm $\mathcal{F}_{\mathcal{A}}$ associated to $\mathcal{A}$ is the randomized algorithm that takes input x and proceeds as follows.

Algorithm $\mathcal{F}_{\mathcal{A}}(x)$
– Pick coins ρ for $\mathcal{A}$ at random – $h_1, \dots, h_q \xleftarrow{\\$} H$ – $(I, \sigma) \leftarrow \mathcal{A}(x, h_1, \dots, h_q; \rho)$ – If $I = 0$ then return $(0, \epsilon, \epsilon)$ – $h'_1, \dots, h'_q \xleftarrow{\\$} H$ – $(I', \sigma') \leftarrow \mathcal{A}(x, h_1, \dots, h_{I-1}, h'_I, \dots, h'_q; \rho)$ – If $(I = I' \text{ and } h_I = h'_I)$ then return $(1, \sigma, \sigma')$. Else return $(0, \epsilon, \epsilon)$.

Let $\text{frk} = \Pr[b = 1 : x \xleftarrow{\$} \text{IG}; (b, \sigma, \sigma') \xleftarrow{\$} \mathcal{F}_{\mathcal{A}}(x)]$.

Then

$$\text{frk} \geq \text{acc} \cdot \left(\frac{\# \text{acc}}{q} - \frac{1}{h} \right).$$

Alternatively,

$$\text{acc} \leq \frac{q}{h} + \sqrt{q \cdot \text{frk}}.$$

Shamir Secret Sharing. Our threshold blind signature schemes employ Shamir secret sharing [37], which is based on polynomial interpolation, to compute secret key shares sk_i for all signing parties. We recall Shamir's secret sharing below, following the definitions in [16].

Definition 5 (Polynomial Interpolation). Let $\mathbb{F}$ be a field of size at least $t + 1$, and let $\mathcal{S} \subseteq \mathbb{F}$ be such that $|\mathcal{S}| \geq t + 1$. Then, any set of at least $t + 1$ evaluations $(i, P(i))_{i \in \mathcal{S}}$ for a polynomial $P(z) = a_0 + a_1z + a_2z^2 + \dots + a_tz^t$ of degree t over $\mathbb{F}$ can be interpolated to evaluate the polynomial on any other point $z_0 \in \mathbb{F}$ as $P(z_0) = \sum_{i \in \mathcal{S}} P(i) \cdot L_i(z_0)$, where $L_i(z)$ is the Lagrange polynomial of form

$$L_i(z) = \prod_{j \in \mathcal{S}; j \neq i} \frac{z - j}{i - j}.$$

Definition 6 (Shamir's Secret Sharing). Shamir's secret sharing consists of a tuple of PPT algorithms (IssueShares , Recover) defined as follows:

- $\text{IssueShares}(x, n, t+1) \rightarrow \{(1, x_1), \dots, (n, x_n)\}$: Define a polynomial $P(z) = x + a_1z + a_2z^2 + \dots + a_tz^t$ by sampling t random coefficients $a_1, \dots, a_t \xleftarrow{\$} \mathbb{Z}_p$. Output the set of participant shares $\{(i, x_i)\}_{i \in [n]}$, where each $x_i, i \in [n]$, is the evaluation of $P(i) : x_i \leftarrow x + \sum_{j \in [t]} a_j i^j$.
- $\text{Recover}(t+1, \{(i, x_i)\}_{i \in \mathcal{S}}) \rightarrow x$: The recover algorithm is deterministic and takes as input at least $t+1$ shares and returns the original secret. Recover x as $x \leftarrow \sum_{i \in \mathcal{S}} \lambda_i^{\mathcal{S}} x_i$, where the Lagrange coefficient for party i in the set $\mathcal{S}$ is defined by $\lambda_i^{\mathcal{S}} = L_i(0) = \prod_{j \in \mathcal{S}, j \neq i} \frac{j}{j-i}$.

2.2 Blind Signatures

We define (partially) blind signatures and their desired security properties. The definitions are taken and adapted from [29].

Definition 7 (Partially Blind Signature Scheme). A partially blind signature scheme with message space $\mathcal{M}$ and common message space $\mathcal{T}$ is a tuple of PPT algorithms $\text{BS} = (\text{Setup}, \text{KeyGen}, \text{Signer}, \text{User}, \text{Verify})$ with the following syntax:

- $\text{Setup}(1^\lambda)$: takes as input security parameter λ and returns public parameters par . These are given as input to all other algorithms implicitly.
- $\text{KeyGen}()$: outputs a pair of keys (vk, sk) . We assume that sk includes vk implicitly.
- $\text{Signer}(\text{sk}, \tau) \longleftrightarrow \text{User}(\text{vk}, m, \tau)$: Signer takes as input a secret key sk and common message $\tau \in \mathcal{T}$. User takes as input a key vk , a message $m \in \mathcal{M}$ and common message $\tau \in \mathcal{T}$. After the execution, User returns a signature σ and we write $\sigma \leftarrow \langle \text{Signer}(\text{sk}, \tau), \text{User}(\text{vk}, m, \tau) \rangle$.
- $\text{Verify}(\text{vk}, m, \tau, \sigma)$: is deterministic and takes as input public key vk , message $m \in \mathcal{M}$, a common message $\tau \in \mathcal{T}$, and a signature σ , and outputs $b \in \{0, 1\}$.

Note that (plain) blind signatures are a special case of partially blind signatures where $|\mathcal{T}| = 1$. Blind signatures should satisfy three properties, namely correctness, one-more unforgeability and blindness. Intuitively, these three properties give the following guarantees:

Correctness entails that whenever a user and a signer correctly run the signing protocol with a correctly generated key pair (vk, sk) on a message $m \in \mathcal{M}$ and a common message $\tau \in \mathcal{T}$, the generated signature should successfully verify for vk , τ and m .

One-more unforgeability guarantees that the user is not able to generate valid signatures on its own without interacting with the signer, even after having successfully completed several signing sessions with the signer.

Lastly, blindness ensures that the signer is not able to map a run of the interactive signing protocol to a message-signature pair (m, σ) output by the user. Note that it is easy to recover formal definitions from the corresponding notions of threshold blind signatures defined in Section 2.6.

These properties are formally defined below.

Definition 8 (Correctness). A partially blind signature BS is correct with correctness error γ_{err} if for all $(\text{vk}, \text{sk}) \in \text{KeyGen}(1^\lambda)$ and all $m \in \mathcal{M}, \tau \in \mathcal{T}$, it holds that

$$\Pr[\sigma \leftarrow \langle \text{Signer}(\text{sk}, \tau), \text{User}(\text{vk}, m, \tau) \rangle : \text{Verify}(\text{vk}, m, \tau, \sigma) = 1] \geq 1 - \gamma_{\text{err}}(\lambda).$$

Definition 9 (One-More Unforgeability). Let $\text{BS} = (\text{Setup}, \text{KeyGen}, \text{Signer}, \text{User}, \text{Verify})$ be a blind signature scheme. Consider an algorithm $\mathcal{A}$ and the game described in Fig. 1a. We denote by $\text{AdvOMUF}_{\mathcal{A}}^{\text{BS}}(\lambda)$ the probability that this game outputs 1. We say that BS is one-more unforgeable (OMUF), if for every PPT algorithm $\mathcal{A}$, it holds that

$$\text{AdvOMUF}_{\mathcal{A}}^{\text{BS}}(\lambda) = \text{negl}(\lambda).$$

Definition 10 (Partial Blindness). Let $\text{BS} = (\text{Setup}, \text{KeyGen}, \text{Signer}, \text{User}, \text{Verify})$ be a blind signature scheme. Consider an algorithm $\mathcal{A}$ and the game described in Fig. 1b. We denote by $\text{AdvPBlind}_{\mathcal{A}}^{\text{BS}}(\lambda)$ the difference between the probability that the game with $b = 0$ outputs 1 and the probability that the game with $b = 1$ outputs 1. We say that BS satisfies partial blindness if

$$\text{AdvPBlind}_{\mathcal{A}}^{\text{BS}}(\lambda) = \text{negl}(\lambda).$$

$\text{Exp}_{\text{BS}, \mathcal{A}}^{\text{OMUF}}(1^\lambda)$

```

1:  $\text{par} \leftarrow \text{Setup}(1^\lambda)$ 
2:  $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}()$ 
3:  $\text{SID} := \emptyset, \text{queried}[\cdot] := 0$ 
4:  $\text{common}[\cdot] := \perp, \text{state}[\cdot] := \perp, \text{round}[\cdot] := \perp$ 
5:  $(\tau^*, (m_k^*, \sigma_k^*)_{k \in [\ell+1]}) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Signer}(\cdot)}}(\text{vk})$ 
6: req  $\text{queried}[\tau^*] \leq \ell \wedge |\{m_1, \dots, m_{\ell+1}\}| = \ell + 1$ 
7: return  $\forall i \in [\ell + 1] : \text{Verify}(\text{vk}, m_i, \sigma_i) = 1$ 

```

$\text{Next}(\text{sid}, \tau)$

```

1: if  $\text{sid} \in \text{SID}$  then return  $\perp$ 
2:  $\text{SID} \leftarrow \text{SID} \cup \{\text{sid}\}$ 
3:  $\text{common}[\text{sid}] := \tau, \text{round}[\text{sid}] := 0$ 
4: return 1

```

$\mathcal{O}^{\text{Signer}}(j, \text{sid}, \text{pm}_{j-1}^U)$

```

1: req  $\text{round}[\text{sid}] = j - 1$ 
2:  $\text{round}[\text{sid}] := j$ 
3:  $\tau^{\text{sid}} := \text{common}[\text{sid}]$ 
4: if  $j = 1$  then
5:    $(\text{st}_1^S, \text{pm}_1^S) \leftarrow \text{ISign}_1(\text{sk}, \tau^{\text{sid}}, \text{pm}_0^U)$ 
6: else
7:    $(\text{st}_j^S, \text{pm}_j^S) \leftarrow \text{ISign}_j(i, \text{st}_{j-1}^S, \text{pm}_{j-1}^U)$ 
8: if  $j = r$  then  $\text{queried}[\tau^{\text{sid}}] := \text{queried}[\tau^{\text{sid}}] + 1$ 
9: return  $\text{pm}_j^S$ 

```

(a) The one-more unforgeability experiment for blind signatures.

$\text{Exp}_{\text{BS}, \mathcal{A}}^{\text{blind}}(1^\lambda)$

```

1:  $\text{par} \leftarrow \text{Setup}(1^\lambda)$ 
2:  $b \xleftarrow{\$} \{0, 1\}$ 
3:  $S_1, \dots, S_r := \emptyset$ 
4:  $b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{User}_0(\cdot)}, \dots, \mathcal{O}^{\text{User}_r(\cdot)}}()$ 
5: return  $b = b'$ 

```

$\mathcal{O}^{\text{User}_0}(\text{sid}, \text{vk}^{\text{sid}}, \tau^{\text{sid}}, m_0^{\text{sid}}, m_1^{\text{sid}})$

```

1: return  $\perp$  if  $\text{sid} \in S_1$ 
2:  $S_1 := S_1 \cup \{\text{sid}\}$ 
3:  $(\text{st}_{0,0}^U, \text{pm}_{0,0}^U) \leftarrow \text{User}_0(\text{vk}^{\text{sid}}, m_b^{\text{sid}}, \tau^{\text{sid}})$ 
4:  $(\text{st}_{1,0}^U, \text{pm}_{1,0}^U) \leftarrow \text{User}_0(\text{vk}^{\text{sid}}, m_{1-b}^{\text{sid}}, \tau^{\text{sid}})$ 
5: return  $(\text{pm}_{0,0}^U, \text{pm}_{1,0}^U)$ 

```

$\mathcal{O}^{\text{User}_j}(\text{sid}, \text{pm}_{0,j}^S, \text{pm}_{1,j}^S)$

```

1: return  $\perp$  if  $\text{sid} \in S_j \wedge \text{sid} \notin S_{j-1}$ 
2:  $S_j := S_j \cup \{\text{sid}\}$ 
3:  $(\text{st}_{0,j}^U, \text{pm}_{0,j}^U) \leftarrow \text{User}_j(\text{st}_{0,j-1}^U, \text{pm}_{0,j}^S)$ 
4:  $(\text{st}_{1,j}^U, \text{pm}_{1,j}^U) \leftarrow \text{User}_j(\text{st}_{1,j-1}^U, \text{pm}_{1,j}^S)$ 
5: return  $(\text{pm}_{0,j}^U, \text{pm}_{1,j}^U)$ 

```

$\mathcal{O}^{\text{User}_r}(\text{sid}, \text{pm}_{0,r}^S, \text{pm}_{1,r}^S)$

```

1: return  $\perp$  if  $\text{sid} \in S_r \wedge \text{sid} \notin S_{r-1}$ 
2:  $S_r := S_r \cup \{\text{sid}\}$ 
3:  $\sigma_b^{\text{sid}} \leftarrow \text{User}_r(\text{st}_{0,r-1}^U, \text{pm}_{0,r}^S)$ 
4:  $\sigma_{1-b}^{\text{sid}} \leftarrow \text{User}_r(\text{st}_{1,r-1}^U, \text{pm}_{1,r}^S)$ 
5: if  $\sigma_0^{\text{sid}} = \perp \vee \sigma_1^{\text{sid}} = \perp$  then return  $(\perp, \perp)$ 
6: return  $(\sigma_0^{\text{sid}}, \sigma_1^{\text{sid}})$ 

```

(b) The partial blindness experiment for blind signatures. Above, we have $j \in \{1, \dots, r-1\}$.

Fig. 1: Security games for blind signatures.

2.3 Proof Systems

Relations and Σ -Protocols. We define NP-relations and Σ -Protocols, following the definitions from [29].

Definition 11 (NP-Relation and Language). Let $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ be a binary relation. We say that R is an NP-relation, if there is a polynomial p such that R can efficiently be decided and for every $(x, w) \in R$, we have $|w| \leq p(|x|)$. We denote by $\mathcal{L}_R = \{x \in \{0, 1\}^* \mid \exists w \text{ s.t. } (x, w) \in R\}$ the language induced by R .

Definition 12 (Σ -Protocol). Let R be an NP-relation. A Σ -protocol for relation R with challenge space $\mathcal{C}$ is a tuple of PPT algorithms $\Sigma = (\text{Init}, \text{Resp}, \text{Verify})$ such that

- $\text{Init}(x, w)$: given a statement $x \in \mathcal{L}_R$ and a witness w , outputs a first flow message (i.e., commitment) A and a state st , where we assume st includes (x, w) ,
- $\text{Resp}(st, c)$: given a state st and a challenge $c \in \mathcal{C}$, deterministically outputs a third flow message (i.e., response) z ,
- $\text{Verify}(x, A, c, z)$: given a statement $x \in \mathcal{L}_R$, a commitment A , a challenge $c \in \mathcal{C}$, and a response z , deterministically outputs a bit $b \in \{0, 1\}$.

We call the tuple (A, c, z) the transcript and say that it is valid for x if $\text{Verify}(x, A, c, z)$ outputs 1. When the context is clear, we simply say it is valid and omit x .

Σ -Protocols for NP-relations should satisfy three properties, namely correctness, special honest-verifier zero-knowledge (HVZK) and special soundness, defined below.

Definition 13 (Correctness). Let R be an NP-relation and $\Sigma = (\text{Init}, \text{Resp}, \text{Verify})$ be a Σ -protocol for R . We say Σ is correct, if for all $(x, w) \in R$, $(A, st) \leftarrow \text{Init}(x, w)$, $c \in \mathcal{C}$, and $z \leftarrow \text{Resp}(st, c)$, it holds that $\text{Verify}(x, A, c, z) = 1$.

Definition 14 (Special HVZK). Let R be an NP-relation and $\Sigma = (\text{Init}, \text{Resp}, \text{Verify})$ be a Σ -protocol for R . We say that Σ is special honest-verifier zero-knowledge (HVZK), if there exists a PPT zero-knowledge simulator Sim such that for any (potentially unbounded) adversary $\mathcal{A}$, it holds that for any $(x, w) \in R$ and $c \in \mathcal{C}$ that $D_{\text{real}} = D_{\text{sim}}$ for

$$\begin{aligned} D_{\text{real}} &:= \{(A, c, z) \mid A \leftarrow \text{Init}(x, w), z \leftarrow \text{Resp}(st, c)\}, \\ D_{\text{sim}} &:= \{(A, c, z) \mid (A, z) \leftarrow \text{Sim}(x, c)\}. \end{aligned}$$

In this work, we write HVZK for short.

Definition 15 (Special Soundness). Let R be an NP-relation and $\Sigma = (\text{Init}, \text{Resp}, \text{Verify})$ be a Σ -protocol for R . We say that Σ is (2-)special sound, if there exists a deterministic PT extractor Ext such that given two valid transcripts $\{(A, c_b, z_b)\}_{b \in \{0, 1\}}$ for statement x with $c_0 \neq c_1$, along with x , outputs a witness w such that $(x, w) \in R$.

We additionally define the property of (perfect) randomizability [28]. Looking ahead, we follow the rerandomization procedure in Lemma 2 as in [28] in our blind signature construction. This specific randomization procedure allows to issue Fiat-Shamir proofs in a blind manner, whereas Definition 16 and Corollary 1 facilitate the blindness proof.

Definition 16 ((Perfect) Randomizable Transcripts). Let Σ be a Σ -protocol for relation R with challenge space $\mathcal{C}$, and suppose Σ is HVZK. Let Rand be an efficient randomization algorithm such that $\text{Rand}(x, (A, c, z))$, given a valid transcript (A, c, z) for x , outputs a new valid transcript for x . We say Σ has randomizable transcripts (resp. strongly randomizable transcripts) if a Rand exists such that for all $x \in \mathcal{L}_R$ (resp. all x) and all accepting $\pi^* = (A^*, c^*, z^*)$, the distributions

$$\begin{aligned} D_{\text{SHVZK}} &:= \{(x, (A, c, z)) \mid c \xleftarrow{\$} \mathcal{C}; (A, z) \leftarrow \text{Sim}(x, c)\} \\ D_{\text{Rand}} &:= \{(x, (A, c, z)) \mid (A, c, z) \leftarrow \text{Rand}(x, \pi^*)\} \end{aligned}$$

are identical.

We also recall the following corollary from [28].

Corollary 1. *Suppose Σ is a Σ -protocol for relation R which is HVZK and has randomizable transcripts. Then for all $(\mathbb{x}, \mathbb{w}) \in R$ and all accepting $\pi^* = (A^*, c^*, z^*)$, the following distributions are identical:*

$$\begin{aligned} D_{\text{Real}} &:= \{(\mathbb{x}, (A, c, z)) \mid A, \text{st} \leftarrow \text{Init}(\mathbb{x}, \mathbb{w}); c \xleftarrow{\$} \mathcal{C}; z \leftarrow \text{Resp}(\text{st}, c)\} \\ D_{\text{SHVZK}} &:= \{(\mathbb{x}, (A, c, z)) \mid c \xleftarrow{\$} \mathcal{C}; (A, z) \leftarrow \text{Sim}(\mathbb{x}, c)\} \\ D_{\text{Rand}} &:= \{(\mathbb{x}, (A, c, z)) \mid (A, c, z) \leftarrow \text{Rand}(\mathbb{x}, \pi^*)\}. \end{aligned}$$

Non-interactive proof system. Our blind signature scheme will also use straightline-extractable non-interactive zero-knowledge proofs as a building block. The definitions below follow [29].

Definition 17 (Non-Interactive Proof System). *A non-interactive proof system (NIPS) for NP-relation R using a random oracle H is a pair $\text{NIPS} = (\text{Prove}, \text{Ver})$ of PPT algorithms with access to a random oracle, where*

- $\text{Prove}^H(\mathbb{x}, \mathbb{w})$: generates a proof π given $(\mathbb{x}, \mathbb{w}) \in R$.
- $\text{Ver}^H(\mathbb{x}, \pi)$: verifies a proof π for statement $\mathbb{x}$ and outputs 0 or 1.

We are interested in NIPS that satisfy three properties, namely correctness, witness-indistinguishability and knowledge soundness. Roughly, correctness guarantees that any proof π computed for a statement $\mathbb{x}$ and a witness $\mathbb{w}$ should successfully verify for $\mathbb{x}$ with probability close to 1. Informally, witness-indistinguishability expresses that it should be hard for an adversary to tell which witness $\mathbb{w}$ was used to compute a certain proof for a statement $\mathbb{x}$. Lastly, knowledge soundness means that there exists an algorithm called the extractor that successfully computes a witness for a given statement $\mathbb{x}$, when provided with $\mathbb{x}$, a proof for $\mathbb{x}$ and all adversarial random oracle queries made during the computation of the proof as inputs. A NIPS is relaxed knowledge sound if the extractor computes the witness only for a relaxed relation $\bar{R} \supseteq R$, where $\bar{R}$ is called the knowledge relation. Formal definitions are given below.

Definition 18 (Correctness of NIPS). *Let $\text{NIPS} = (\text{Prove}, \text{Ver})$ be a non-interactive proof system for a relation R . It has correctness error γ_{err} if for all $(\mathbb{x}, \mathbb{w}) \in R$, it holds that*

$$\Pr [\pi \leftarrow \text{Prove}^H(\mathbb{x}, \mathbb{w}) : \text{Ver}^H(\mathbb{x}, \pi) = 1] \geq 1 - \gamma_{\text{err}}(\lambda),$$

where the probability is over the choice of H and the randomness of Prove, Ver . We call NIPS correct if $\gamma_{\text{err}}(\lambda) = \text{negl}(\lambda)$. We say it is perfectly correct if $\gamma_{\text{err}} = 0$.

Definition 19 (Witness Indistinguishability). *Let $\text{NIPS} = (\text{Prove}^H, \text{Ver}^H)$ be a non-interactive proof system for a relation R in the random oracle model. Let $\mathcal{A}$ be an algorithm which makes at most $Q = Q(\lambda)$ queries to H and let*

$$\text{AdvWl}_{\mathcal{A}}^{\text{NIPS}}(Q, \lambda) = \Pr [b \leftarrow \mathcal{A}^{H, \mathcal{O}_0}(1^\lambda) : b = 1] - \Pr [b \leftarrow \mathcal{A}^{H, \mathcal{O}_1}(1^\lambda) : b = 1],$$

where $\mathcal{O}_i(\mathbb{x}, \mathbb{w}_0, \mathbb{w}_1)$ returns $\text{Prove}^H(\mathbb{x}, \mathbb{w}_i)$ for $i \in \{0, 1\}$. We call NIPS statistically (resp. computationally) witness indistinguishable (WI), if for any unbounded (resp. PPT) adversary $\mathcal{A}$, the advantage $\text{AdvWl}_{\mathcal{A}}^{\text{NIPS}}(\lambda)$ is negligible.

Definition 20 (Relaxed Knowledge Soundness). *Let $\text{NIPS} = (\text{Prove}, \text{Ver})$ be a non-interactive proof system for a relation R and let $\bar{R} \supseteq R$ be an NP-relation. Let Ext be a PPT algorithm. Let $\mathcal{A}$ be an oracle algorithm and let*

$$\begin{aligned} \text{Real}_{\mathcal{A}}(\lambda) &:= \Pr [b \leftarrow \mathcal{A}^{H, \mathcal{O}_{\text{Ver}}}(1^\lambda) : b = 1], \\ \text{Ideal}_{\mathcal{A}}(\lambda) &:= \Pr [b \leftarrow \mathcal{A}^{H, \mathcal{O}_{\text{Ext}}}(1^\lambda) : b = 1]. \end{aligned}$$

Here, $\mathcal{A}$ has (black-box) access to the random oracle H and to an oracle $\mathcal{O}_{\text{Prove}}$ or $\mathcal{O}_{\text{Ext}}$, which are as follows:

- $\mathcal{O}_{\text{Ver}}(\mathbb{x}, \pi)$: Return $\text{Ver}(\mathbb{x}, \pi)$.

- $\mathcal{O}_{\text{Ext}}(\mathbb{x}, \pi)$: If $\text{Ver}(\mathbb{x}, \pi) = 1$ and $(\mathbb{x}, \mathbb{w}) \notin \tilde{\mathcal{R}}$ for $\mathbb{w} \leftarrow \text{Ext}(\mathcal{Q}, \mathbb{x}, \pi)$, return 0. Else, return 1. Here, $\mathcal{Q}$ denotes the set of $\mathcal{A}$'s $\mathcal{H}$ queries.

The advantage of $\mathcal{A}$ against knowledge soundness is $\text{AdvKS}_{\mathcal{A}}^{\text{NIPS}, \tilde{\mathcal{R}}}(\lambda) := |\text{Real}_{\mathcal{A}}(\lambda) - \text{Ideal}_{\mathcal{A}}(\lambda)|$. We say that Ext is a knowledge extractor for NIPS and knowledge relation $\tilde{\mathcal{R}}$, if for every PPT algorithm $\mathcal{A}$, the advantage $\text{AdvKS}_{\mathcal{A}}^{\text{NIPS}, \tilde{\mathcal{R}}}(\lambda)$ is negligible in λ . We say that NIPS is knowledge sound, if there is a knowledge extractor for NIPS.

2.4 Pseudorandom Function

A pseudorandom function is a function $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ such that $F(k, \cdot)$ is indistinguishable from a random function $f : \mathcal{X} \rightarrow \mathcal{Y}$ for random $k \leftarrow \mathcal{K}$.

Definition 21 (PRF Security). Consider function $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ and denote by $\mathbb{F}[\mathcal{X}, \mathcal{Y}]$ the set of all functions from $\mathcal{X}$ to $\mathcal{Y}$. We say that F is a secure pseudorandom function if for all PPT $\mathcal{A}$, we have

$$\text{AdvPRF}_{\mathcal{A}}^F(\lambda) = \left| \Pr_{k \leftarrow \mathcal{K}} [\mathcal{A}^{F(k, \cdot)}(1^\lambda) = 1] - \Pr_{f \leftarrow \mathbb{F}[\mathcal{X}, \mathcal{Y}]} [\mathcal{A}^{f(\cdot)}(1^\lambda) = 1] \right| = \text{negl}(\lambda).$$

2.5 Commitment Schemes

We define commitment schemes. For simplicity, we follow the convention that to verify the commitment c , we check that $c = \text{Commit}(\text{CK}, m, r)$ holds. Therefore, we sometimes refer to r as opening. Note that this convention ensures correctness by definition.

Definition 22 (Commitment Scheme). A (non-interactive) commitment scheme with message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is a tuple of PPT algorithms $\text{Com} = (\text{Setup}, \text{Commit})$ with the following syntax:

- $\text{Setup}(1^\lambda)$: takes as input security parameter λ and returns a public commitment key CK .
- $\text{Commit}(\text{CK}, m, r)$: takes as input public commitment key CK , a message $m \in \mathcal{M}$, and (explicit) randomness $r \in \mathcal{R}$, and returns commitment c .

When the public commitment key CK is obvious from the context, we sometimes omit the parameter CK from Commit as input.

We consider commitment schemes that are perfectly hiding and computationally binding. Intuitively, the hiding property ensures that the commitment c does not leak any information about the committed message m , while the binding property expresses that it is hard to open c to two distinct messages. We also require Com to be linearly homomorphic, and write $a \cdot \text{Commit}(m, r) = \text{Commit}(a \cdot m, a \cdot r)$, $\text{Commit}(m_1, r_1) + \text{Commit}(m_2, r_2) = \text{Commit}(m_1 + m_2, r_1 + r_2)$. Formal definitions are given below.

Definition 23 (Perfectly Hiding). Let $\text{Com} = (\text{Setup}, \text{Commit})$ be a commitment scheme. We say Com is perfectly hiding if for any (unbounded) algorithm $\mathcal{A}$ the following holds:

$$\Pr \left[b = b' \mid \begin{array}{l} b \xleftarrow{\$} \{0, 1\}, \text{CK} \leftarrow \text{Setup}(1^\lambda), (m_0, m_1, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(\text{CK}), \\ r \xleftarrow{\$} \mathcal{R}, c \leftarrow \text{Commit}(\text{CK}, m_b, r), b' \leftarrow \mathcal{A}(c, \text{st}_{\mathcal{A}}) \end{array} \right] = \frac{1}{2}.$$

Definition 24 (Computationally Binding). Let $\text{Com} = (\text{Setup}, \text{Commit})$ be a commitment scheme. We say Com is computationally binding if for any PPT algorithm $\mathcal{A}$ the following holds:

$$\Pr \left[c_0 = c_1 \wedge m_0 \neq m_1 \mid \begin{array}{l} \text{CK} \leftarrow \text{Setup}(1^\lambda), (m_0, m_1, r_0, r_1) \leftarrow \mathcal{A}(\text{CK}) \\ c_i = \text{Commit}(\text{CK}, m_i, r_i) \text{ for } i \in \{0, 1\} \end{array} \right] = \text{negl}(\lambda).$$

We denote by $\text{AdvBnd}_{\mathcal{A}}^{\text{Com}}$ the advantage of $\mathcal{A}$ on binding of Com given by the above probability.

Definition 25 (Linear Homomorphism). Let $\text{Com} = (\text{Setup}, \text{Commit})$ be a commitment scheme. We call Com linearly homomorphic, if (in additive notation) the following conditions hold for all $m, m_1, m_2 \in \mathcal{M}$ and $r, r_1, r_2 \in \mathcal{R}$:

- $a \cdot \text{Commit}(m, r) = \text{Commit}(a \cdot m, a \cdot r)$,
- $\text{Commit}(m_1, r_1) + \text{Commit}(m_2, r_2) = \text{Commit}((m_1 + m_2), (r_1 + r_2))$.

2.6 Threshold Blind Signatures

We introduce threshold blind signatures, following the definitions from [32].

Definition 26 (Threshold Blind Signature). A threshold (partially) blind signature scheme with message space $\mathcal{M}$ and common message space $\mathcal{T}$ is a tuple of PPT algorithms $\text{TBS} = (\text{Setup}, \text{KeyGen}, \text{ISign}, \text{USign}, \text{Verify})$ such that:

- $\text{Setup}(1^\lambda)$: takes as input security parameter λ and returns public parameters par . These are given as input to all other algorithms implicitly.
- $\text{KeyGen}(n, t+1)$: takes as input the number of signers n as well as the threshold $t+1$ and outputs the public key vk representing the set of all signers, the set $\{\text{sk}_i\}_{i \in [n]}$ of secret keys for each individual signer as well as additional auxiliary information aux , which may be used by the user and the signers in the course of the signing protocol, but which is not necessary for signature verification.
- $\{\text{ISign}(k, \text{sk}_k, \text{aux}, \text{ctx}, \tau)\}_{k \in \mathcal{S}} \longleftrightarrow \text{USign}(\text{vk}, \text{aux}, m, \mathcal{S}, \tau)$: is an r round interactive protocol between a set $\mathcal{S}$ of signers running the signing algorithm ISign and one party running the user algorithm USign . The j -th round signer algorithm ISign_j takes as input signer index k , secret key share sk_k , auxiliary information aux , context data ctx and common message $\tau \in \mathcal{T}$. The j -th round user algorithm USign_j takes as input a key vk , a message $m \in \mathcal{M}$, auxiliary information aux , a set of signers $\mathcal{S}$ and common message $\tau \in \mathcal{T}$. After the execution, USign returns a signature σ and we write $\sigma \leftarrow \langle \{\text{ISign}(k, \text{sk}_k, \text{aux}, \text{ctx}, \tau)\}_{k \in \mathcal{S}}, \text{USign}(\text{vk}, \text{aux}, m, \mathcal{S}, \tau) \rangle$. We omit initial inputs in later rounds and assume these are passed on in the state.

More formally, we use the following notation to describe the interactive signing protocol, where ISign_j and USign_j describe the j -th round of the algorithms ISign and USign , respectively:

$$\begin{aligned}
(\text{st}_0^U, \text{pm}_0^U) &\leftarrow \text{TBS.USign}_0(\text{vk}, \text{aux}, m, \mathcal{S}, \tau), \\
(\text{st}_{1,i}^I, \text{pm}_{1,i}^I) &\leftarrow \text{TBS.ISign}_1(i, \text{sk}_i, \text{aux}, \text{ctx}, \tau, \text{pm}_0^U), \\
(\text{st}_1^U, \text{pm}_1^U) &\leftarrow \text{TBS.USign}_1(\text{st}_0^U, \{\text{pm}_{1,i}^I\}_{i \in \mathcal{S}}) \\
(\text{st}_{j,i}^I, \text{pm}_{j,i}^I) &\leftarrow \text{TBS.ISign}_j(i, \text{st}_{j-1,i}^I, \text{pm}_{j-1}^U), \\
(\text{st}_j^U, \text{pm}_j^U) &\leftarrow \text{TBS.USign}_j(\text{st}_{j-1}^U, \{\text{pm}_{j,i}^I\}_{i \in \mathcal{S}}) \\
(\text{ssid}, \mathcal{S}', \text{pm}_{r,i}^I) &\leftarrow \text{TBS.ISign}_r(i, \text{st}_{r-1,i}^I, \text{pm}_{r-1}^U), \\
\sigma / \perp &\leftarrow \text{TBS.USign}_r(\text{st}_{r-1}^U, \{\text{pm}_{r,i}^I\}_{i \in \mathcal{S}}).
\end{aligned}$$

- $\text{TBS.Verify}(\text{vk}, m, \tau, \sigma)$: takes as input public key vk , message $m \in \mathcal{M}$, common message $\tau \in \mathcal{T}$, and signature σ , and deterministically outputs bit b .

Threshold blind signatures should satisfy correctness, blindness and one-more unforgeability. Analogously to blind signatures, correctness states that a signature derived from an honest protocol execution verifies and blindness ensures that the signers cannot link the derived signature to the issuing signing session. For one-more unforgeability, we follow the strongest notion OMUF-3 in [32]’s hierarchy except for two modifications: (1) we do not consider *strong* unforgeability, i.e., in our notion the adversary wins if it provides forgeries for ℓ pairwise-distinct messages, and (2) we generalize the definitions to partial blindness. Roughly, OMUF-3 states that it is hard for an adversary to forge ℓ signatures on distinct messages with common message τ after *completing* at most $\ell - 1$ signing sessions with τ . Here, a session only counts as complete if all honest signing parties have been invoked for the final round with shared post-fix session identifier ssid , shared context ctx and shared set of honest co-signers $\mathcal{S}_{\text{hon}}$.

As alluded to above, we make two slight adjustments to the definition of OMUF-3 from [32]. First, we introduce a common message τ and thus generalize the definitions to the setting of threshold partially blind signatures. Secondly, we slightly harden the winning conditions of the OMUF-3 experiment by letting the adversary win whenever he outputs the required number of ℓ valid message-signature pairs and all ℓ messages are pairwise distinct (instead of only demanding that all ℓ message-signature pairs are pairwise distinct).

Definition 27 (Correctness of a Threshold Blind Signature). A threshold (partially) blind signature scheme TBS is correct with correctness error γ_{err} if for all allowable $1 \leq t < n$, for all $\mathcal{S} \subseteq [n]$ such that $t < |\mathcal{S}| \leq n$, all $m \in \mathcal{M}$, $\tau \in \mathcal{T}$ it holds that for $\text{in}_i = (i, \text{sk}_i, \text{aux}, \text{ctx}, \tau)$ and $\text{in}_U = (\text{vk}, \text{aux}, m, \mathcal{S}, \tau)$ that

$$\Pr[\sigma \leftarrow \langle \{\text{ISign}(\text{in}_i)\}_{i \in \mathcal{S}}, \text{USign}(\text{in}_U) \rangle : \text{TBS.Verify}(\text{vk}, \sigma, m) = 1] \geq 1 - \gamma_{\text{err}}.$$

$\text{Exp}_{\text{TBS}, \mathcal{A}}^{\text{OMUF}}(1^\lambda)$ <pre> 1: $\text{par} \leftarrow \text{Setup}(1^\lambda)$ 2: $S_1, \dots, S_r := \emptyset$ 3: $\text{ES}_3[\cdot] := \emptyset$ 4: $\text{El}_3[\cdot, \cdot] := \emptyset$ 5: $\mathcal{C} \leftarrow \mathcal{A}(n, t)$ 6: if $\mathcal{C} > t$ then return 0 7: $(\text{vk}, (\text{sk}_i)_{i \in [n]}, \text{aux}) \leftarrow \text{KeyGen}(n, t)$ 8: $(\tau^*, \ell, (m_k^*, \sigma_k^*)_{k \in [\ell]}) \leftarrow \mathcal{A}^{\text{Signer}(\cdot)}(\text{vk}, (\text{sk}_i)_{i \in \mathcal{C}}, \text{aux})$ 9: if $\ell \leq \text{ES}_3[\tau^*]$ then return 0 10: return $\forall k \in [\ell] : \left(\begin{array}{l} \text{Verify}(\text{vk}, \sigma_k^*, m_k^*) = 1 \\ \wedge \forall j \in [\ell] \setminus \{k\} : m_k^* \neq m_j^* \end{array} \right)$ </pre> $\mathcal{O}^{\text{Signer}}(j, \text{sid}, i, \text{ctx}_{j-1}^{\text{sid}}, \tau_{j-1}^{\text{sid}}, \text{pm}_{j-1}^{\text{U}})$ <pre> 1: return $\perp$ if $(i, \text{sid}) \in S_j$ 2: return $\perp$ if $j > 1 \wedge (i, \text{sid}) \notin S_1, \dots, S_{j-1}$ 3: $S_j := S_j \cup \{(i, \text{sid})\}$ 4: if $j = 1$ then 5: $(\text{st}_1^{(S,i)}, \text{pm}_1^{(S,i)}) \leftarrow \text{ISign}_1(i, \text{sk}_i, \text{aux}, \text{ctx}_0^{\text{sid}}, \tau_0^{\text{sid}}, \text{pm}_0^{\text{U}})$ 6: else 7: $(\text{st}_j^{(S,i)}, \text{pm}_j^{(S,i)}) \leftarrow \text{ISign}_j(i, \text{st}_{j-1}^{(S,i)}, \text{pm}_{j-1}^{\text{U}})$ 8: if $j = r$ then 9: $\text{parse}(\text{ssid}_i^{\text{sid}}, \mathcal{S}_i^{\text{sid}}) := \text{st}_{r-1}^{(S,i)}$ 10: $\mathcal{S}_{\text{hon}} := \mathcal{S}_i^{\text{sid}} \setminus \mathcal{C}$ 11: $\text{El}_3[\text{ssid}_i^{\text{sid}}, \mathcal{S}_{\text{hon}}] := \text{El}_3[\text{ssid}_i^{\text{sid}}, \mathcal{S}_{\text{hon}}] \cup \{i\}$ 12: if $\mathcal{S}_{\text{hon}} \subseteq \text{El}_3[\text{ssid}_i^{\text{sid}}, \mathcal{S}_{\text{hon}}]$ then 13: $\text{ES}_3[\tau] := \text{ES}_3[\tau] \cup \{\text{ssid}_i^{\text{sid}}\}$ 14: return $(\text{pm}_{j-1}^{(S,i)}, \text{ssid}_i^{\text{sid}}, \mathcal{S}_i)$ 15: return $\text{pm}_j^{(S,i)}$ </pre>	$\text{Exp}_{\text{TBS}, \mathcal{A}}^{\text{blind}}(1^\lambda)$ <pre> 1: $\text{par} \leftarrow \text{Setup}(1^\lambda)$ 2: $b \xleftarrow{\\$} \{0, 1\}$ 3: $S_1, \dots, S_r := \emptyset$ 4: $b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{User}_0(\cdot)}, \dots, \mathcal{O}^{\text{User}_r(\cdot)}}()$ 5: return $b = b'$ </pre> $\mathcal{O}^{\text{User}_0}(\text{sid}, \text{vk}^{\text{sid}}, \text{aux}^{\text{sid}}, \tau_0^{\text{sid}}, m_0^{\text{sid}}, m_1^{\text{sid}}, \mathcal{S}_0^{\text{sid}}, \mathcal{S}_1^{\text{sid}})$ <pre> 1: return $\perp$ if $\text{sid} \in S_1$ 2: $S_1 := S_1 \cup \{\text{sid}\}$ 3: $(\text{st}_{0,0}^{\text{U}}, \text{pm}_{0,0}^{\text{U}}) \leftarrow \text{USign}_0(\text{vk}^{\text{sid}}, \text{aux}^{\text{sid}}, m_b^{\text{sid}}, \mathcal{S}_0^{\text{sid}}, \tau_0^{\text{sid}})$ 4: $(\text{st}_{1,0}^{\text{U}}, \text{pm}_{1,0}^{\text{U}}) \leftarrow \text{USign}_0(\text{vk}^{\text{sid}}, \text{aux}^{\text{sid}}, m_{1-b}^{\text{sid}}, \mathcal{S}_1^{\text{sid}}, \tau_0^{\text{sid}})$ 5: return $(\text{pm}_{0,0}^{\text{U}}, \text{pm}_{1,0}^{\text{U}})$ </pre> $\mathcal{O}^{\text{User}_j}(\text{sid}, (\text{pm}_{0,j}^{(S,i)})_{i \in \mathcal{S}_0^{\text{sid}}}, (\text{pm}_{1,j}^{(S,i)})_{i \in \mathcal{S}_1^{\text{sid}}})$ <pre> 1: return $\perp$ if $\text{sid} \in S_j \wedge \text{sid} \notin S_{j-1}$ 2: $S_j := S_j \cup \{\text{sid}\}$ 3: $(\text{st}_{0,j}^{\text{U}}, \text{pm}_{0,j}^{\text{U}}) \leftarrow \text{USign}_j(\text{st}_{0,j-1}^{\text{U}}, (\text{pm}_{0,j}^{(S,i)})_{i \in \mathcal{S}_0^{\text{sid}}})$ 4: $(\text{st}_{1,j}^{\text{U}}, \text{pm}_{1,j}^{\text{U}}) \leftarrow \text{USign}_j(\text{st}_{1,j-1}^{\text{U}}, (\text{pm}_{1,j}^{(S,i)})_{i \in \mathcal{S}_1^{\text{sid}}})$ 5: return $(\text{pm}_{0,j}^{\text{U}}, \text{pm}_{1,j}^{\text{U}})$ </pre> $\mathcal{O}^{\text{User}_r}(\text{sid}, (\text{pm}_{0,r}^{(S,i)})_{i \in \mathcal{S}_0^{\text{sid}}}, (\text{pm}_{1,r}^{(S,i)})_{i \in \mathcal{S}_1^{\text{sid}}})$ <pre> 1: return $\perp$ if $\text{sid} \in S_r \wedge \text{sid} \notin S_{r-1}$ 2: $S_r := S_r \cup \{\text{sid}\}$ 3: $\sigma_b^{\text{sid}} \leftarrow \text{USign}_r(\text{st}_{0,r-1}^{\text{U}}, (\text{pm}_{0,r}^{(S,i)})_{i \in \mathcal{S}_0^{\text{sid}}})$ 4: $\sigma_{1-b}^{\text{sid}} \leftarrow \text{USign}_r(\text{st}_{1,r-1}^{\text{U}}, (\text{pm}_{1,r}^{(S,i)})_{i \in \mathcal{S}_1^{\text{sid}}})$ 5: if $\sigma_0^{\text{sid}} = \perp \vee \sigma_1^{\text{sid}} = \perp$ then return $(\perp, \perp)$ 6: return $(\sigma_0^{\text{sid}}, \sigma_1^{\text{sid}})$ </pre>
--	--

(a) One-more unforgeability definition based on OMUF-3 in [32], adapted to partial blindness.

(b) Partial blindness experiment for TBS adpated from [32], where $j \in \{1, \dots, r-1\}$.

Fig. 2: Security games for threshold blind signatures.

Definition 28 (OMUF-3). Let $\text{TBS} = (\text{Setup}, \text{KeyGen}, \text{ISign}, \text{USign}, \text{Verify})$ be a threshold blind signature scheme. We consider the game given in Fig. 2a and denote by $\text{Adv}_{\mathcal{A}, n, t}^{\text{TOMUF}^{\text{TBS}, 3}}(\lambda)$ the probability that the game outputs 1. We say that TBS is 3-one-more unforgeable if, for all PPT adversaries $\mathcal{A}$, and $n, t \in \mathbb{N}$ such that $t < n$

$$\text{Adv}_{\mathcal{A}}^{\text{TOMUF}^{\text{TBS}}(\lambda)} = \text{negl}(\lambda).$$

Definition 29 (Partial Blindness of a Threshold Blind Signature Scheme). Let $\text{TBS} = (\text{Setup}, \text{KeyGen}, \text{ISign}, \text{USign}, \text{Verify})$ be a threshold blind signature scheme. We consider the game given in Fig. 2b and denote by $\text{Adv}_{\mathcal{A}, \text{TBS}}^{\text{blind-t}}(\lambda)$ the difference between the probability that the game with $b = 0$ outputs 1 and the probability that the game with $b = 1$ outputs 1. We say that TBS satisfies partial blindness if for all PPT adversaries $\mathcal{A}$, and $n, t \in \mathbb{N}$ such that $t < n$, it holds that

$$\text{Adv}_{\mathcal{A}}^{\text{TPBlind}^{\text{TBS}}(\lambda)} = \text{negl}(\lambda).$$

3 BS: Pairing-free Blind Signature from CDH

Now, we present our blind signature BS proven under CDH. This scheme will serve as a basis for our threshold blind signature TBS in Section 5. We refer to Section 1 for a high-level overview of our scheme. To begin, let us establish some convenient notation similar to [29].

3.1 Notation and Preparations

We introduce two maps $\phi_{G,V}^{\text{BB}}: \mathbb{G} \times \mathbb{Z}_p^2 \rightarrow \mathbb{G}^3$ and $\phi_G^{\text{DLog}}: \mathbb{Z}_p \rightarrow \mathbb{G}$ that are defined as follows:

$$\phi_{G,V}^{\text{BB}}(X, (s, u)) = \begin{pmatrix} u \cdot V + s \cdot X \\ s \cdot G \\ u \cdot G \end{pmatrix}, \quad (3)$$

where $X = \overline{m}U + H$ and $V \in \mathbb{G}$, and

$$\phi_G^{\text{DLog}}(w) = wG. \quad (4)$$

We note that map $\phi_{G,V}^{\text{BB}}$ establishes that the image is a valid BB signature (cf. Eq. (1)), while map ϕ_G^{DLog} expresses a “knowledge of DLog” statement. We also note that maps ϕ_G^{DLog} and $\phi_{G,V}^{\text{BB}}(X, \cdot)$ are linear, when value X is fixed. When values G and V are clear from the context, we will write $\phi_0(X, (s, u))$ and $\phi_1(w)$ for short. For map ϕ_0 , we define relation R_{bb} and induced language $\mathcal{L}_{\text{bb}}$:

$$R_{\text{bb}} := \{(\mathbf{x}_0, \mathbf{w}_0) \mid \mathbf{S} = \phi_0(X, (s, u))\},$$

where $\mathbf{x}_0 = (G, V, X, \mathbf{S}) \in \mathbb{G}^6$ and $\mathbf{w}_0 = (s, u) \in \mathbb{Z}_p^2$. Analogously, for map ϕ_1 , we define relation R_{dlog} and induced language $\mathcal{L}_{\text{dlog}}$:

$$R_{\text{dlog}} := \{(\mathbf{x}_1, \mathbf{w}_1) \mid W = \phi_1(w)\},$$

where $\mathbf{x}_1 = (G, W) \in \mathbb{G}^2$ and $\mathbf{w}_1 = w \in \mathbb{Z}_p$. For the relations R_{bb} and R_{dlog} , we define the Σ -protocols $\Sigma_0 = (\text{Init}_0, \text{Resp}_0, \text{Verify}_0)$ and $\Sigma_1 = (\text{Init}_1, \text{Resp}_1, \text{Verify}_1)$, respectively, with challenge space $\mathbb{Z}_p$. We denote the HVZK simulators of Σ_0 and Σ_1 by Sim_0 and Sim_1 , respectively. Instantiations of Σ_0 and Σ_1 are given below.

Instantiation of the Σ -protocols. We can instantiate both Σ_0 and Σ_1 with a standard Schnorr-type Σ -protocol with challenge space $\mathbb{Z}_p$ for the relation of statements $\mathbf{x} = \mathbf{T}$ and witnesses $\mathbf{w} = \mathbf{w}$ with $\phi(\mathbf{w}) = \mathbf{T}$, where $\phi: \mathbb{Z}_p^\omega \rightarrow \mathbb{G}^\kappa$ is a linear map. Concretely, such a Σ -protocol looks as follows:

Σ -protocol $\Sigma = (\text{Init}, \text{Resp}, \text{Verify})$ for R_ϕ with $\mathbf{x} = (\phi, \mathbf{T})$ and $\mathbf{w} = \mathbf{w}$
– $\text{Init}(\mathbf{x}, \mathbf{w})$: 1. Sample $\mathbf{r} \xleftarrow{\\$} \mathbb{Z}_p^\omega$ and set $\mathbf{A} := \phi(\mathbf{r})$. 2. Output commitment $\mathbf{A}$ and state $\text{st} := (\mathbf{w}, \mathbf{r})$. – $\text{Resp}(\text{st}, c)$: Output $\mathbf{z} := c \cdot \mathbf{w} + \mathbf{r}$, – $\text{Verify}(\mathbf{x}, \mathbf{A}, c, \mathbf{z})$: Output 1 if $\mathbf{A} = \phi(\mathbf{z}) - c \cdot \mathbf{T}$ and 0 otherwise.

It is well-known that the generic Σ -protocol satisfies useful properties (cf. [28]).

Lemma 2 (Properties of the generic Σ -protocol Σ). *The above generic Σ -protocol Σ is correct, special sound, HVZK and efficient. Additionally, it is strongly randomizable. More concretely*

- $\text{Sim}(\mathbf{T}, c)$ samples $\mathbf{z} \xleftarrow{\$} \mathbb{Z}_p^\omega$, sets $\mathbf{A} = c \cdot \mathbf{T} - \phi(\mathbf{z})$ and outputs $(\mathbf{A}, \mathbf{z})$.
- $\text{Rand}(\mathbf{T}, (\mathbf{A}^*, c^*, \mathbf{z}^*))$ samples $c' \xleftarrow{\$} \mathbb{Z}_p$ and $\mathbf{z}' \xleftarrow{\$} \mathbb{Z}_p^\omega$ and outputs

$$(\mathbf{A}, c, \mathbf{z}) = (\mathbf{A}^* - c' \mathbf{T} + \phi(\mathbf{z}'), c^* + c', \mathbf{z}^* + \mathbf{z}').$$

NIPS for Pedersen Openings. We define relations R_{Ped} and $\tilde{R}_{\text{Ped}}$ as follows:

$$R_{\text{Ped}} := \{(\mathbf{x}, \mathbf{w}) \mid C = \overline{m}U + tG\}, \text{ where } \mathbf{x} = (C, U, G), \mathbf{w} = (\overline{m}, t), \quad (5)$$

$$\tilde{R}_{\text{Ped}} := \{(\mathbf{x}, \mathbf{w}) \mid \mathbf{w}G = U \vee (\mathbf{x}, \mathbf{w}) \in R_{\text{Ped}}\}, \text{ where } \mathbf{x} = (C, U, G). \quad (6)$$

We denote by $\text{NIPS}_{\text{Ped}} = (\text{NIPS}_{\text{Ped}}.\text{Prove}^{\text{HPed}}, \text{NIPS}_{\text{Ped}}.\text{Ver}^{\text{HPed}})$ a NIPS for relation R_{Ped} that is statistically witness-indistinguishable and online-extractable for relation $\tilde{R}_{\text{Ped}}$. NIPS_{Ped} has access to random oracle $\text{H}_{\text{Ped}}: \{0, 1\}^* \rightarrow \mathcal{Y}_{\text{Ped}}$ with image space $\mathcal{Y}_{\text{Ped}}$. A suitable instantiation is given in [29, Section C].

Furthermore, $\text{Com} = (\text{Com}.\text{Setup}, \text{Com}.\text{Commit})$ denotes a linearly homomorphic commitment scheme. Looking ahead, we require that Com is perfectly hiding and computationally binding. If clear by context, we abbreviate $\text{Com}.\text{Commit}$ as Com . A suitable instantiation for Com are Pedersen commitments.

Random Oracles. Our scheme is in the random oracle model. For readability, we define several random oracles and give a brief overview here.

- $H_M : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ is used to hash the messages m .
- $H_\Sigma : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ is used to compile Σ_0 and Σ_1 into an OR-proof via Fiat-Shamir.
- $H_{\text{Ped}} : \{0, 1\}^* \rightarrow \mathcal{Y}_{\text{Ped}}$ is the random oracle for NIPS_{Ped}.
- $H_{\text{DLog}} : \{0, 1\}^* \rightarrow \mathbb{G}$ is used to derive the element W for the DLog proof Σ_1 .
- $H_V : \{0, 1\}^* \rightarrow \mathbb{G}$ is used to derive a part of the BB verification key.

3.2 Construction

Setup(1^λ)	Verify(vk, m, τ, σ)
1 : $(\mathbb{G}, p, G) \leftarrow \text{GrGen}(1^\lambda)$	1 : parse $(S_1, S_2, \pi, (d_1, r_1), (d_2, r_2)) = \sigma$
2 : $\text{par}_{\text{Com}} \leftarrow \text{Com.Setup}(1^\lambda)$	2 : parse $(\mathbf{A}_0, A_1, c, c_0, \mathbf{z}_0, z_1) = \pi$
3 : $\text{par} := ((\mathbb{G}, p, G), \text{par}_{\text{Com}})$	3 : $\mathbf{S} := (S_1, S_2, U), \bar{m} := H_M(m), X := \bar{m}U + H$
4 : return par	4 : $\mathbf{S}_\S := \mathbf{S} + (d_1 G, 0, 0)^T$
KeyGen()	5 : $\mathbf{A}_{\S,0} := \mathbf{A}_0 - (d_2 G, 0, 0)^T$
1 : $u \xleftarrow{\S} \mathbb{Z}_p, U := uG$	6 : $C_S := \text{Com.Commit}(d_1, r_1)$
2 : $H \xleftarrow{\S} \mathbb{G}$	7 : $C_{A_0} := \text{Com.Commit}(d_2, r_2)$
3 : return vk := $(G, U, H), \text{sk} := u$	8 : $\mathbb{x}_0^C := (G, V, X, \mathbf{S}_\S)$
	9 : $W := H_{\text{DLog}}(\tau), \mathbb{x}_1 := (G, W)$
	10 : $c := H(\mathbb{x}_0^C, \mathbf{A}_{\S,0}, \mathbb{x}_1, A_1, m, C_S, C_{A_0}, \text{par}_{\text{Com}})$
	11 : $c_1 := c - c_0$
	12 : if $\text{Verify}_0(\mathbb{x}_0, \mathbf{A}_0, c_0, \mathbf{z}_0) = 0$ return 0
	13 : if $\text{Verify}_1(\mathbb{x}_1, A_1, c_1, z_1) = 0$ return 0
	14 : return 1

Fig. 3: The algorithms Setup, KeyGen and Verify belonging to scheme BS. Differences to the DDH-reliant scheme from [29] are highlighted in grey.

Our signature scheme BS is given in Fig. 3 and Fig. 4. It allows to issue signatures of the form (S_1, S_2, π) , where (S_1, S_2) is a BB signature and π is a Fiat-Shamir proof of the disjunctive relation $R_{\text{bb}} \cup R_{\text{dlog}}$. The latter is an OR-proof guaranteeing that either the signature (S_1, S_2) is well-formed or that the discrete logarithm of $H_{\text{DLog}}(\tau) = W$ is known. The OR-proof is instantiated by additive secret sharing of the challenge of Σ_0 and Σ_1 for R_{bb} and R_{dlog} , respectively, following the well-known techniques by [15].

In order to ensure that we can prove OMUF for our scheme BS, we let the signer issue a masked signature tuple $\mathbf{T}_\S^* = (S_1 + d_1 G, S_2)$ as well as a commitment C_S to value d_1 in the second move of our scheme. We let the signer reveal d_1 as well as the corresponding commitment randomness r only in the last round. This ensures that unfinished sessions do not reveal the BB signature (S_1, S_2) and the commitment ensures the tuple $(C_S, \mathbf{T}_\S^*)$ determines (S_1, S_2) *computationally*. Looking ahead, the latter property is required for (computational) soundness of the proof π .

As a consequence, the user does not know the actual signature tuple (S_1, S_2) which is required to derive the joint challenge of the Σ -protocols via H_Σ . Neither can the user blind the first flow of the Σ -protocol $\mathbf{A}_0^*$ at this point, as this requires knowledge of the signature tuple. Therefore, the user computes these values homomorphically, i.e., it sets up masked signatures $(S_{\S,1}, S_{\S,2})$ and masked commitment $\mathbf{A}_{\S,0}$ that still contain the masking term $d_1 G$. These values and the masking term $d_1 G$ are then blinded, similar to [29]. Then, the user derives the joint Σ -protocol challenge on the masked values $\mathbf{A}_{\S,0}$ and $(S_{\S,1}, S_{\S,2})$, as well as the commitment C_S and C_{A_0} which determine the randomized masking terms d_1 and d_2 .

ISign₁ (sk, τ, C, π _{Ped}) 1 : $\mathbb{X}_{\text{Ped}} := (C, U, G)$ 2 : req $\text{NIPSPed.Ver}^{\text{HPed}}(\mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}}) = 1$ 3 : $s^* \xleftarrow{\$} \mathbb{Z}_p$; $w_0 := (s^*, \text{sk})$ 4 : $d_1^*, r_1^* \xleftarrow{\$} \mathbb{Z}_p$ 5 : $X_C := C + H$; $W := \text{H}_{\text{DLog}}(\tau)$ 6 : $\mathbf{T}^* := \phi_0(X_C, w_0)$; $\mathbf{T}_{\S}^* := \mathbf{T}^* + (d_1^*G, 0, 0)^{\top}$ 7 : $C_S^* := \text{Com}(d_1^*, r_1^*)$ 8 : $\mathbb{X}_0^t := (G, V, X_C, \mathbf{T}^*)$; $\mathbb{X}_1 := (G, W)$ 9 : $c_1^* \xleftarrow{\$} \mathbb{Z}_p$; $(A_1^*, z_1^*) \leftarrow \text{Sim}_1(\mathbb{X}_1, c_1^*)$ 10 : $(\mathbf{A}_0^*, \text{st}_0) \leftarrow \text{Init}_0(\mathbb{X}_0^t, w_0)$ 11 : return $(\mathbf{T}_{\S}^*, \mathbf{A}_0^*, A_1^*, C_S^*)$	USign₀ (vk, m, τ) 1 : $t \xleftarrow{\$} \mathbb{Z}_p$; $\bar{m} := \text{H}_{\text{M}}(m)$, 2 : $C := \bar{m}U + tG$ 3 : $\mathbb{X}_{\text{Ped}} := (C, U, G)$; $\mathbb{W}_{\text{Ped}} := (\bar{m}, t)$ 4 : $\pi_{\text{Ped}} \leftarrow \text{NIPSPed.Prove}^{\text{HPed}}(\mathbb{X}_{\text{Ped}}, \mathbb{W}_{\text{Ped}})$ 5 : return (C, π_{Ped})
ISign₂ (c*) 1 : $c_0^* := c^* - c_1^*$ 2 : $\mathbf{z}_0^* \leftarrow \text{Resp}_0(\text{st}_0, c_0^*)$ 3 : return $(\mathbf{z}_0^*, z_1^*, c_0^*, d_1^*, r_1^*)$	USign₁ ($\mathbf{T}_{\S}^*, \mathbf{A}_0^*, A_1^*, C_S^*$) 1 : $s' \xleftarrow{\$} \mathbb{Z}_p$; 2 : $c'_0, c'_1 \xleftarrow{\$} \mathbb{Z}_p$; 3 : $\mathbf{z}'_0 \xleftarrow{\$} \mathbb{Z}_p^2$; $z'_1 \xleftarrow{\$} \mathbb{Z}_p$ 4 : $d'_1, r'_1, d'_2, r'_2 \xleftarrow{\$} \mathbb{Z}_p$ 5 : $X := \bar{m}U + H = X_C - tG$ 6 : $\mathbf{S}_{\S} := \mathbf{T}_{\S}^* - (t \cdot \mathbf{T}_{\S,2}^*, 0, 0)^{\top} + \phi_0(X, (s', 0)) + (d'_1G, 0, 0)^{\top}$ $= \mathbf{S} + ((d_1^* + d'_1)G, 0, 0)^{\top}$ 7 : $\mathbf{A}_{\S,0} := \mathbf{A}_0^* - (t \cdot \mathbf{A}_{0,2}^*, 0, 0)^{\top} + \phi_0(X, \mathbf{z}'_0) - c'_0\mathbf{S}_{\S} - (d'_2G, 0, 0)^{\top}$ $= \mathbf{A}_0 - ((c'_0(d_1^* + d'_1) + d'_2)G, 0, 0)^{\top}$ 8 : $C_S := C_S^* + \text{Com}(d'_1, r'_1) = \text{Com}(d_1^* + d'_1, r_1^* + r'_1)$ 9 : $C_{A_0} := c'_0C_S + \text{Com}(d'_2, r'_2)$ $= \text{Com}(c'_0(d_1^* + d'_1) + d'_2, c'_0(r_1^* + r'_1) + r'_2)$ 10 : $A_1 := A_1^* + \phi_1(z'_1) - c'_1W = A_1^* + z'_1G - c'_1W$ 11 : $\mathbb{X}_0^C := (G, V, X, \mathbf{S}_{\S})$; $\mathbb{X}_1 := (G, W)$ 12 : $c := \text{H}_{\Sigma}(\mathbb{X}_0^C, \mathbf{A}_{\S,0}, \mathbb{X}_1, A_1, m, C_S, C_{A_0}, \text{parCom})$ 13 : $c^* := c - c'_0 - c'_1$ 14 : return c^*
USign₂ ($\mathbf{z}_0^*, z_1^*, c_0^*, d_1^*, r_1^*$) 1 : req $\mathbf{A}_0^* = \phi_0(X_C, \mathbf{z}_0^*) - c_0^*(\mathbf{T}_{\S}^* - (d_1^*G, 0, 0)^{\top})$ $= \phi_0(X_C, \mathbf{z}_0^*) - c_0^*\mathbf{T}^*$ 2 : req $A_1^* = \phi_1(z_1^*) - c_1^*W$ 3 : req $C_S^* = \text{Com}(d_1^*, r_1^*)$ 4 : $c_0 := c_0^* + c'_0$; $c_1 := c_1^* + c'_1$ $\parallel c = c_0 + c_1$ 5 : $\mathbf{z}_0 := \mathbf{z}_0^* + \mathbf{z}'_0 + c_0^* \cdot (s', 0)$ 6 : $z_1 := z_1^* + z'_1$ 7 : $d_1 := d_1^* + d'_1$; $r_1 := r_1^* + r'_1$ 8 : $d_2 := c'_0d_1 + d'_2$; $r_2 := c'_0r_1 + r'_2$ 9 : $\mathbf{S} := \mathbf{S}_{\S} - (d_1G, 0, 0)^{\top}$ 10 : $\mathbf{A}_0 := \mathbf{A}_{\S,0} + (d_2G, 0, 0)^{\top}$ 11 : $\pi := (\mathbf{A}_0, A_1, c, c_0, \mathbf{z}_0, z_1)$ 12 : $\sigma_{\text{bb}} := (S_1, S_2, \pi, (d_1, r_1), (d_2, r_2))$ 13 : return σ_{bb}	

Fig. 4: The interactive signing protocol executed by the signer and the user in scheme BS. Algorithms **ISign₁** and **ISign₂** are executed by the signer, algorithms **USign₀**, **USign₁** and **USign₂** are executed by the user. Differences to the DDH-reliant scheme from [29] are highlighted in grey. The Σ -protocols and statements are blinded as in [29], except that some operations are performed with masked $\mathbf{T}_{\S}^*$. Note that we write **Com** short for **Com.Commit**. We leave the state implicit for readability.

As verification of signatures σ generated by BS involves recomputing the joint challenge c , it is necessary to include the openings (d_1, r_1) and (d_2, r_2) for the commitments C_S and C_{A_0} in the final signature σ .

Optimizations. The performance of our scheme BS can be improved via 2 natural optimizations. The numbers reported in Section 1 assume the following changes are made.

1. The values $\mathbf{A}_0, A_1$ are omitted from the proof π as these values can be reconstructed from the remaining transcript of π . We note that these are not omitted in the formal construction as it simplifies notation in the security analysis.

2. We can further reduce the signature size by one $\mathbb{Z}_p$ element. Instead of committing to d_1 and d_2 in separate commitments, we can commit to d_1 and d_2 in a single multi-Pedersen commitment of the form $C_S = d_1 G_1 + d_2 G_2 + rG$. Therefore, only a single opening value r is required. As we only require that d_1 and d_2 are computationally fixed by C_S , it is straightforward to adapt the proof accordingly.

3.3 Security Analysis

We give an overview of our security analysis. In particular, we show correctness, one-more unforgeability and blindness. Due to space limitations, we only provide proof sketches. Formal proofs for all statements are given in Section 4.

Correctness. It is easy but tedious to verify that the scheme is correct.

Theorem 1 (Correctness). *BS is correct with error γ_{err} , where γ_{err} is the correctness error of NIPS_{Ped} .*

A formal proof of Theorem 1 is given in Section 4.1.

One-more Unforgeability. We show that BS is one-more unforgeable. As discussed in Section 1.2, we follow the proof strategy by [29] but manage to remove the reliance on DDH in their argument. We refer to Section 1.2 for an overview of the puncturing-based proof strategy of [29]. Let us briefly discuss how our modifications allow to prove security under CDH in more detail.

Simulation strategy for BB signatures. We must be able to simulate BB (pre-)signatures $\mathbf{T}^*$ issued by the signer. As our goal is to puncture the BB verification key for a specific message $\bar{m}^*$, this turns out to be non-trivial. Here, $\bar{m}^*$ is the hash of a message for which the adversary provides a forgery but never finishes a signing session with the forgeries' common message τ^* . Note that $\bar{m}^*$ is guessed by the reduction in advance which is required for puncturing.⁶

Given the punctured setup, it remains easy to sign any message *not* equal to $\bar{m}^*$. However, the adversary might still query $\bar{m}^*$ in *some* sessions which we refer to as “challenge” sessions hereafter. These challenge sessions are all sessions where the Pedersen commitment C commits to $\bar{m}^*$ which can be tested by extracting from π_{Ped} . Therefore, we can identify whether any given signing session corresponds to a challenge session or not in the reduction. Let us explain how to simulate $\mathbf{T}^*$ in challenge sessions.

In [29], a DDH-based argument allows to randomize all such pre-signatures $\mathbf{T}^*$ which suffices for simulation. In our case, we cannot afford this argument as we wish to avoid decisional assumptions. However, as the signer now masks $\mathbf{T}^*$ with a random value d_1 and the commitment Com is hiding, we can argue that we can randomize all *unfinished* challenge sessions. In particular, we know that no session is finished if $\bar{m}^*$ is extracted from π_{Ped} and the common message τ is the forgery's common message τ^* (which can be guessed in advance). Therefore, by sending a random tuple $(\mathbf{T}_S^*, C_S^*)$, we can simulate such unfinished sessions as described in Section 1.2.

However, we must *also* simulate sessions where $\bar{m}^*$ is extracted *and* $\tau \neq \tau^*$. These sessions might be finished and if so, reveal a BB pre-signature $\mathbf{T}^*$ for $\bar{m}^*$ in plain (as the user can remove the masking term). Therefore, we must issue well-distributed pre-signatures $\mathbf{T}^*$. However, the verification key is punctured for $\bar{m}^*$ and simulation fails.

To resolve this issue, the crucial observation is that there are *two* ways to issue BB signatures due to **symmetry** in their structure:

$$S_1 = uV + s(\bar{m}U + H), \quad S_2 = sG.$$

Roughly, it is hard to compute signatures (without the signing key u) because $u \cdot V = uv \cdot G$ forms a CDH tuple. But we can compute (S_1, S_2) by knowing *either* the discrete logarithm u of U *or* the discrete logarithm v of V . If we choose V differently for each common message τ (e.g., by deriving $V_\tau = \text{H}_V(\tau)$ as in BS), then we can embed elements V_τ with known logarithm into H_V . This allows to sign *any* message, even if the remaining verification key is punctured. This crucially

⁶ This guess is correct with sufficient probability following the argument by [29] based on the pigeon-hole principle: There are only $k - 1$ finished sessions on common message τ^* and but k forgeries. We refer to Game 9 in Section 4.2 for more details.

relies on the structure of (S_1, S_2) and the fact that puncturing is compatible with this observation. To argue security, we embed a CDH challenge in the forgery's V_{τ^*} and puncture the verification key accordingly. As discussed above, for the forgeries' τ^* we can leverage the commitment's hiding property to simulate challenge sessions instead. In total, we can now simulate *all* sessions and the forgery for $\bar{m}^*$ yields a BB signature $\mathbf{S}$ for the punctured verification key, allowing to solve CDH.

Enforcing well-formedness. As signatures contain an OR-proof that either the BB signature $\mathbf{S}$ is valid (i.e., well-formed according to the above equation) or the discrete logarithm of W is known, we have to ensure that the adversary indeed provides us with valid BB signatures. To do so, in contrast to [29], we must rely on a forking-based argument to avoid the DDH assumption, following the proof technique by [13].

Theorem 2 (One-More Unforgeability). *For any PPT adversary $\mathcal{A}$ causing at most Q random oracle queries and finishing at most $k - 1$ signing sessions, there exist reductions $\mathcal{A}_{\text{KS}}, \mathcal{A}_{\text{DL}}, \mathcal{A}_{\text{Bnd}}$ and $\mathcal{A}_{\text{CDH}}$ having a runtime similar to $\mathcal{A}$ such that*

$$\begin{aligned} \text{AdvOMUF}_{\mathcal{A}}^{\text{BS}}(\lambda) &\leq \frac{Q^2}{p} + \text{AdvKS}_{\mathcal{A}_{\text{KS}}}^{\text{NIPS}_{\text{Ped}}, \tilde{R}_{\text{Ped}}}(\lambda) + \text{AdvDL}_{\mathcal{A}_{\text{DL}}}^{\text{GrGen}}(\lambda) + \\ &Q^2 \cdot \left(k \cdot \left(\sqrt{Q_{\text{H}_\Sigma}(\text{AdvDL}_{\mathcal{A}_{\text{DL}}}^{\text{GrGen}}(\lambda) + \text{AdvBnd}_{\mathcal{A}_{\text{Bnd}}}^{\text{Com}}(\lambda))} + \frac{Q_{\text{H}_\Sigma}}{p} \right) + \text{AdvCDH}_{\mathcal{A}_{\text{CDH}}}^{\text{GrGen}}(\lambda) \right). \end{aligned}$$

A formal proof of Theorem 2 is given in Section 4.2.

Blindness. Blindness follows as the commitment C_S is randomomized and the user blinds the signature and proofs as in [29]. We follow the techniques from [28, Theorem 4.7] for a more compact proof. An important point is that we must rely on a forking-based argument, similar to [13], to ensure that the issued BB signatures are well-formed. This is crucial: if the adversary $\mathcal{A}$ finds a way to issue and identify misformed BB signatures $\mathbf{S}$, then $\mathcal{A}$ could issue a well-formed signature in one signing session and a misformed signature in the other session, breaking blindness. However, the signer issues an interactive Σ -protocol proof of well-formedness for $\mathbf{T}^*$ which allows to deduce that the derived signature $\mathbf{S}$ is well-formed, else the signer must know the discrete logarithm of $W = \text{H}_{\text{DLog}}(\tau)$. Alternatively, we could let the signer prove well-formedness of $\mathbf{T}^*$ with a separate NIPS as in [13].

Theorem 3 (Blindness). *For any PPT adversary $\mathcal{A}$ causing at most Q random oracle queries and starting at most k signing sessions, there exist reductions $\mathcal{A}_{\text{WI}}$ and $\mathcal{A}_{\text{DL}}, \mathcal{A}_{\text{Bnd}}$, where $\mathcal{A}_{\text{DL}}, \mathcal{A}_{\text{Bnd}}$ have running time roughly that of $\mathcal{A}$, such that*

$$\begin{aligned} \text{AdvPBlind}_{\mathcal{A}}^{\text{BS}}(\lambda) &\leq 2kQ \cdot \left(\text{AdvWI}_{\mathcal{A}_{\text{WI}}}^{\text{NIPS}, \tilde{R}_{\text{Ped}}}(Q, \lambda) + \frac{1 + Q}{p} + \right. \\ &\quad \left. \sqrt{Q(\text{AdvDL}_{\mathcal{A}_{\text{DL}}}^{\text{GrGen}}(\lambda) + \text{AdvBnd}_{\mathcal{A}_{\text{Bnd}}}^{\text{Com}}(\lambda))} \right). \end{aligned}$$

A formal proof of Theorem 3 is given in Section 4.3.

4 Formal Security Proofs for BS

We present formal security proofs for our scheme in Section 3.

4.1 Correctness of BS

We give a formal proof of Theorem 1.

Proof. By definition, we know that NIPS_{Ped} may fail to output a proof π_{Ped} that successfully verifies with probability γ_{err} . In the following we will show, that, if NIPS_{Ped} does not fail to do so, BS always generates a valid signature.

By construction of our scheme BS, a message-signature pair (m, σ) with $\sigma = (S_1, S_2, \pi, (d_1, r_1), (d_2, r_2))$ and $\pi = (A_0, A_1, c, c_0, z_0, z_1)$ successfully verifies for common message τ and commitment parameters par_{Com} iff the following conditions hold:

1. $c = H_{\Sigma}(\mathbb{x}_0^C, \mathbf{A}_{\S,0}, \mathbb{x}_1, A_1, m, C_S, C_{A_0}, \text{par}_{\text{Com}})$, where $\mathbf{A}_{\S,0} = \mathbf{A}_0 - (d_2G, 0, 0)^\top$, $\mathbb{x}_0^C = (G, H_V(\tau), X, \mathbf{S}_{\S})$, $X = \overline{m}U + H$, $\mathbf{S}_{\S} = \mathbf{S} + (d_1G, 0, 0)^\top$, $\mathbf{S} = (S_1, S_2, U)$, $C_S = \text{Com}(d_1, r_1)$ and $C_A = \text{Com}(d_2, r_2)$
2. $\text{Verify}_0(\mathbb{x}_0, \mathbf{A}_0, c_0, \mathbf{z}_0) = 1$ with $\mathbb{x}_0 = (G, H_V(\tau), X, \mathbf{S})$ and $\text{Verify}_1(\mathbb{x}_1, A_1, c_1, z_1) = 1$ with $\mathbb{x}_1 = (G, W)$ and $W = H_{\text{DLog}}(\tau)$, i.e. it holds $\mathbf{A}_0 = \phi_0(X, (\mathbf{z}_0)) - c_0\mathbf{S}$ and $A_1 = \phi_1(z_0) - c_1W$.

It is clear that the first condition trivially holds by construction of the user algorithm $\text{User}(\text{vk}, m, \tau)$. In the following, we will show that the second condition is satisfied as well. We will proceed by setting the randomization parameters $s', c'_0, c'_1, \mathbf{z}'_0, z'_1, d'_1, r'_1, d'_2, r'_2$ to zero and by first showing that this non-randomized version of BS is correct. Then, we will gradually allow the randomization parameters to also have non-zero values and show that correctness still holds after each step. We recall that, by correctness of Σ_0 and Σ_1 , $\mathbf{A}_0^* = \phi_0(X_C, (\mathbf{z}_0^*)) - c_0^*\mathbf{T}^*$ and $A_1^* = \phi_1(z_0^*) - c_1^*W$.

Step 0 (Correctness of unblinded protocol). In this first step, we have $s' = c'_0 = c'_1 = z'_{0,1} = z'_{0,2} = z'_1 = d'_1 = r'_1 = d'_2 = r'_2 = 0$. Thus,

$$\begin{aligned} \mathbf{T}_{\S}^* &= \mathbf{T}^* = \phi_0(X_C, (s^*, u)) = \begin{pmatrix} uV + s^*X_C \\ s^*G \\ uG \end{pmatrix} = \begin{pmatrix} uV + s^*X \\ s^*G \\ uG \end{pmatrix} + \begin{pmatrix} s^*tG \\ 0 \\ 0 \end{pmatrix} \\ &= \phi_0(X, (s^*, u)) + (s^*tG, 0, 0)^\top, \end{aligned}$$

$$\mathbf{S} = \mathbf{S}^* = \mathbf{T}_{\S}^* - (t \cdot T_{\S,2}^*, 0, 0)^\top = \phi_0(X, (s^*, u)),$$

$$\begin{aligned} \mathbf{A}_0^* &= \phi_0(X_C, \mathbf{z}_0^*) - c_0^*\mathbf{T}^* = \begin{pmatrix} uV + z_{0,1}^*X_C \\ z_{0,1}^*G \\ z_{0,2}^*G \end{pmatrix} - c_0^* \begin{pmatrix} uV + s^*X_C \\ s^*G \\ uG \end{pmatrix} \\ &= \begin{pmatrix} uV + z_{0,1}^*X \\ z_{0,1}^*G \\ z_{0,2}^*G \end{pmatrix} - c_0^* \begin{pmatrix} uV + s^*X \\ s^*G \\ uG \end{pmatrix} + \begin{pmatrix} z_{0,1}^*tG - c_0^*s^*tG \\ 0 \\ 0 \end{pmatrix} \end{aligned}$$

and

$$\mathbf{A}_0 = \mathbf{A}_{\S,0} = \mathbf{A}_0^* - (t \cdot A_{0,2}^*, 0, 0)^\top = \phi_0(X, \mathbf{z}_0^*) - c_0^*\mathbf{S} = \phi_0(X, \mathbf{z}_0) - c_0\mathbf{S}.$$

As $A_1 = A_1^*$, $c_1 = c_1^*$ and $z_1 = z_1^*$, $A_1 = \phi_1(z_1) - c_1W$ trivially holds by correctness of Σ_1 .

Step 1 (Randomizing s'). In this step, we have $s' \in \mathbb{Z}_p$ and $c'_0 = c'_1 = z'_{0,1} = z'_{0,2} = z'_1 = d'_1 = r'_1 = d'_2 = r'_2 = 0$. We denote $\widehat{\mathbf{S}} := \mathbf{T}_{\S}^* - (t \cdot T_{\S,2}^*, 0, 0)^\top$ as well as $\widehat{\mathbf{z}}_0 := \mathbf{z}_0^*$ and note that $\widehat{\mathbf{S}}$ and $\widehat{\mathbf{z}}_0$ are equal to $\mathbf{S}$ and $\mathbf{z}_0$ from the previous step. We also note that c_0 and $\mathbf{A}_0$ are the same in both steps. Thus, in this step, we can use the result from the previous one in the form of $\mathbf{A}_0 = \phi_0(X, \widehat{\mathbf{z}}_0) - c_0\widehat{\mathbf{S}}$. Thus, we have,

$$\mathbf{S} = \mathbf{S}_{\S} = \mathbf{T}_{\S}^* - (t \cdot T_{\S,2}^*, 0, 0)^\top + \phi_0(X, (s', 0)) = \widehat{\mathbf{S}} + \phi_0(X, (s', 0))$$

and

$$\begin{aligned} \mathbf{A}_0 &= \phi_0(X, \widehat{\mathbf{z}}_0) - c_0\widehat{\mathbf{S}} \\ &= \phi_0(X, \widehat{\mathbf{z}}_0) - c_0\widehat{\mathbf{S}} - c_0 \cdot \phi_0(X, (s', 0)) + c_0 \cdot \phi_0(X, (s', 0)) \\ &= \phi_0(X, (\widehat{\mathbf{z}}_0 + c_0(s', 0))) - c_0\mathbf{S} \\ &= \phi_0(X, \mathbf{z}_0) - c_0\mathbf{S}, \end{aligned}$$

where we used linearity of ϕ_0 as well as the fact that

$$\mathbf{z}_0 = \mathbf{z}_0^* + c_0(s', 0) = \widehat{\mathbf{z}}_0 + c_0(s', 0).$$

As A_1 , c_1 and z_1 stay the same as in the last step, $A_1 = \phi_1(z_1) - c_1 W$ trivially also holds in this step.

Step 2 (Randomizing c). In this step, we have $s', c'_0, c'_1 \in \mathbb{Z}_p$ and $z'_{0,1} = z'_{0,2} = z'_1 = d_1^* = r_1^* = d'_1 = r'_1 = d'_2 = r'_2 = 0$. We denote $\widehat{\mathbf{A}}_0 := \mathbf{A}_0^* - (t \cdot A_2^*, 0, 0)^\top$, $\widehat{A}_1 := A_1^*$, $\widehat{c}_0 := c_0^*$ as well as $\widehat{c}_1 := c_1^*$ and note that $\widehat{\mathbf{A}}_0$, $\widehat{A}_1$, $\widehat{c}_0$ and $\widehat{c}_1$ are equal to $\mathbf{A}_0$, A_1 , c_0 and c_1 from the previous step. We also note that $\mathbf{z}_0$, z_1 , $\mathbf{S}$, $\mathbf{S}_\S$ and W are the same in both steps. Thus, in this step, we can use the result from the previous one in the form of $\widehat{\mathbf{A}}_0 = \phi_0(X, \mathbf{z}_0) - \widehat{c}_0 \mathbf{S}$ and $\widehat{A}_1 = \phi_0(X, z_1) - \widehat{c}_1 W$. Thus, we have,

$$\mathbf{A}_0 = \mathbf{A}_{\S,0} = \widehat{\mathbf{A}}_0 - c'_0 \mathbf{S}_\S = \phi_0(X, \mathbf{z}_0) - \widehat{c}_0 \mathbf{S}_\S - c'_0 \mathbf{S}_\S = \phi_0(X, \mathbf{z}_0) - (\widehat{c}_0 + c'_0) \mathbf{S}_\S = \phi_0(X, \mathbf{z}_0) - c_0 \mathbf{S},$$

where we used that $c_0 = c_0^* + c'_0 = \widehat{c}_0 + c'_0$ and $\mathbf{S}_\S = \mathbf{S}$.

The argument for $A_1 = \phi_1(z_1) - c_1 W$ is completely analogous.

Step 3 (Randomizing $\mathbf{A}_0, \mathbf{A}_1$). In this step, we have $s', c'_0, c'_1, z'_{0,1}, z'_{0,2}, z'_1 \in \mathbb{Z}_p$ and $d_1^* = r_1^* = d'_1 = r'_1 = d'_2 = r'_2 = 0$. We denote $\widehat{\mathbf{A}}_0 := \mathbf{A}_0^* - (t \cdot A_2^*, 0, 0)^\top - c'_0 \mathbf{S}_\S$, $\widehat{A}_1 := A_1^* - c'_1 W$, $\widehat{\mathbf{z}}_0 := \mathbf{z}_0^* + c_0^* \cdot (s', 0)$ as well as $\widehat{z}_1 := z_1^*$ and note that $\widehat{\mathbf{A}}_0$, $\widehat{A}_1$, $\widehat{\mathbf{z}}_0$ and $\widehat{z}_1$ are equal to $\mathbf{A}_0$, A_1 , $\mathbf{z}_0$ and z_1 from the previous step. We also note that c_0 , c_1 , $\mathbf{S}$, $\mathbf{S}_\S$ and W are the same in both steps. Thus, in this step, we can use the result from the previous one in the form of $\widehat{\mathbf{A}}_0 = \phi_0(X, \widehat{\mathbf{z}}_0) - c_0 \mathbf{S}$ and $\widehat{A}_1 = \phi_0(X, \widehat{z}_1) - c_1 W$. Thus, we have,

$$\begin{aligned} \mathbf{A}_0 = \mathbf{A}_{\S,0} &= \widehat{\mathbf{A}}_0 + \phi_0(X, \mathbf{z}'_0) = \phi_0(X, \widehat{\mathbf{z}}_0) - c_0 \mathbf{S}_\S + \phi_0(X, \mathbf{z}'_0) \\ &= \phi_0(X, (\widehat{\mathbf{z}}_0 + \mathbf{z}'_0)) - c_0 \mathbf{S}_\S = \phi_0(X, \mathbf{z}_0) - c_0 \mathbf{S}, \end{aligned}$$

where we used that $\mathbf{z}_0 = \mathbf{z}_0^* + \mathbf{z}'_0 + c_0^* \cdot (s', 0) = \widehat{\mathbf{z}}_0 + \mathbf{z}'_0$ and $\mathbf{S}_\S = \mathbf{S}$.

The argument for $A_1 = \phi_1(z_1) - c_1 W$ is completely analogous.

Step 4 (Randomizing $\mathbf{T}^*$). In this step, we have $s', c'_0, c'_1, z'_{0,1}, z'_{0,2}, z_1, d_1^*, r_1^*, d'_1, r'_1, d'_2, r'_2 \in \mathbb{Z}_p$. We denote $\widehat{\mathbf{S}}_\S := \mathbf{T}^* - (t \cdot T_2^*, 0, 0)^\top + \phi_0(X, (s', 0))$ and $\widehat{\mathbf{A}}_{\S,0} := \mathbf{A}_0^* - (t \cdot A_{0,2}^*, 0, 0)^\top + \phi_0(X, \mathbf{z}'_0) - c'_0 \widehat{\mathbf{S}}_\S$ and note that $\widehat{\mathbf{S}}_\S$ and $\widehat{\mathbf{A}}_{\S,0}$ are equal to $\mathbf{S}$ and to $\mathbf{A}_0$ from the previous step. We also note that c_0 , $\mathbf{z}_0$, $\mathbf{A}_0^*$ and $\mathbf{T}^*$ are the same in both steps. Thus, in this step, we can use the result from the previous one in the form of $\widehat{\mathbf{A}}_0 = \phi_0(X, \mathbf{z}_0) - c_0 \widehat{\mathbf{S}}_\S$. Thus, we have,

$$\begin{aligned} \mathbf{S}_\S &= \mathbf{T}_\S^* - (t \cdot T_{\S,2}^*, 0, 0)^\top + \phi_0(X, (s', 0)) + (d'_1 G, 0, 0)^\top \\ &= \mathbf{T}^* + (d'_1 G, 0, 0)^\top - (t \cdot T_2^*, 0, 0)^\top + \phi_0(X, (s', 0)) + (d'_1 G, 0, 0)^\top \\ &= \widehat{\mathbf{S}}_\S + ((d_1^* + d'_1)G, 0, 0)^\top, \end{aligned}$$

$$\begin{aligned} \mathbf{S} &= \mathbf{S}_\S - (d_1, 0, 0)^\top \\ &= \widehat{\mathbf{S}}_\S + ((d_1^* + d'_1)G, 0, 0)^\top - ((d_1^* + d'_1)G, 0, 0)^\top \\ &= \widehat{\mathbf{S}}_\S, \end{aligned}$$

$$\begin{aligned} \mathbf{A}_{\S,0} &= \mathbf{A}_0^* - (t \cdot A_{0,2}^*, 0, 0)^\top + \phi_0(X, \mathbf{z}'_0) - c'_0 \mathbf{S}_\S - (d'_2 G, 0, 0)^\top \\ &= \mathbf{A}_0^* - (t \cdot A_{0,2}^*, 0, 0)^\top + \phi_0(X, \mathbf{z}'_0) - c'_0 (\widehat{\mathbf{S}}_\S + ((d_1^* + d'_1)G, 0, 0)^\top) - (d'_2 G, 0, 0)^\top \\ &= \widehat{\mathbf{A}}_0 - ((c'_0(d_1^* + d'_1) + d'_2)G, 0, 0)^\top, \end{aligned}$$

$$\begin{aligned} \mathbf{A}_0 &= \mathbf{A}_{\S,0} + (d_2, 0, 0)^\top \\ &= \widehat{\mathbf{A}}_{\S,0} - ((c'_0(d_1^* + d'_1) + d'_2)G, 0, 0)^\top + ((c'_0(d_1^* + d'_1) + d'_2)G, 0, 0)^\top \\ &= \widehat{\mathbf{A}}_{\S,0}, \end{aligned}$$

where we used that $d_1 = d_1^* + d_1'$ and $d_2 = c_0' \cdot d_1 + d_2'$.

Putting these results together, we get $\mathbf{A}_0 = \widehat{\mathbf{A}}_0 = \phi_0(X, \mathbf{z}_0) - c_0 \widehat{\mathbf{S}}_s = \phi_0(X, \mathbf{z}_0) - c_0 \mathbf{S}$.

As the values A_1, c_1 and z_1 do not change in this step, $A_1 = \phi_1(z_1) - c_1 W$ trivially holds by the result of the previous step.

This concludes the proof.

4.2 One-more Unforgeability of BS

We give a formal proof of Theorem 2. We will make use of a lemma from [29]. Intuitively, the lemma says that, if it is hard to solve the CDH problem, it is also hard for any adversary that does not know the secret key u (or the discrete logarithm of V) to compute a BB signature (S_1^*, S_2^*) for a fresh hashed message $\bar{m}^* \in \mathbb{Z}_p$ of its choice, even given access to a signing oracle.

Lemma 3 (Unforgeability of BB). *For any algorithm $\mathcal{A}$, let $\epsilon_{\mathcal{A}}^{\text{BB}}$ be the probability that the following game outputs 1:*

1. Run $(\bar{m}^*, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$.
2. Sample $u \xleftarrow{\$} \mathbb{Z}_p$ and set $U := uG$.
3. Sample $(H, V) \xleftarrow{\$} \mathbb{G}$ and set $X_{\bar{m}^*} := \bar{m}^*U + H$.
4. Run $(S_1^*, S_2^*) \leftarrow \mathcal{A}^{\mathcal{O}}(G, U, H, V, \text{st}_{\mathcal{A}})$, where $\mathcal{O}$ is given as:
 - $\mathcal{O}(\bar{m})$: Output $\perp$ if $\bar{m} = \bar{m}^*$. Otherwise, sample $s \xleftarrow{\$} \mathbb{Z}_p$, set $X_{\bar{m}} = \bar{m} \cdot U + H$, and compute $\mathbf{S} := \phi_0(X_{\bar{m}}, (s, u))$. Then return (S_1, S_2) .
5. Set $\mathbf{x}_0^* := (G, V, X_{\bar{m}^*}, S_1^*, S_2^*, U)$ and output 1 if and only if $\mathbf{x}_0^* \in \mathcal{L}_{\text{bb}}$.

Then, for any PPT algorithm $\mathcal{A}$, there exists some PPT algorithm $\mathcal{B}$ with running time similar to $\mathcal{A}$ such that

$$\epsilon_{\mathcal{A}}^{\text{BB}} \leq \text{AdvCDH}_{\mathcal{B}}^{\text{GrGen}}(\lambda).$$

We will hereafter refer to the game presented in Lemma 3 as Game BB. Roughly, our proof strategy will be to start with the standard one-more unforgeability experiment and to gradually, via multiple game hops, transform it into a game that can be simulated as a subroutine by an adversary playing Game BB. With certain modifications, our proof follows the overall structure of the OMUF proof for the scheme presented in [29]. The reduction on the hardness of computing the DLog of W made in Game 11.3, is inspired by a similar reduction made in [13].

Proof. In the following, $\mathcal{A}$ will denote a PPT adversary against the one-more unforgeability property of BS. The number of signing queries made by $\mathcal{A}$ is given by Q_S . We will write $Q_{\Sigma}, Q_M, Q_{\text{DLog}}, Q_V$ and Q_{Ped} to refer to the number of oracle queries to $H_{\Sigma}, H_M, H_{\text{DLog}}, H_V$ and H_{Ped} , respectively. Hereby, $Q_{\Sigma}, Q_M, Q_{\text{DLog}}$ and Q_{Ped} also count the queries that are caused by the game. The extractor of NIPS_{Ped} is referred to as Ext_{Ped} . Lastly, the probability that Game i will return 1 is given by ϵ_i .

Game 0 (Honest). We start with the standard experiment for one-more unforgeability for scheme BS. During setup, Game 0 sets up the random oracles $H_V, H_{\Sigma}, H_M, H_{\text{DLog}}$ and H_{Ped} as well as oracle Next (with the help of which $\mathcal{A}$ can start a new signing session) and signing oracles $\mathcal{O}_{\text{Signer}_1}$ and $\mathcal{O}_{\text{Signer}_2}$. Then, it samples $u \xleftarrow{\$} \mathbb{Z}_p$, $H \xleftarrow{\$} \mathbb{G}$ and sets $U := uG$, $\text{vk} := (G, U, H)$ as well as $\text{sk} := u$. It also generates $\text{par}_{\text{Com}} \leftarrow \text{Com.Setup}(1^\lambda)$. Then, $\mathcal{A}$ is invoked on input vk . $\mathcal{A}$ also gets query access to the random oracles, to oracle Next and to both signing oracles. In the end, $\mathcal{A}$ will output a common message τ^* and k forged message-signature pairs (m_j^*, σ_j^*) with $j \in [k]$. $\mathcal{A}$ wins and the game outputs 1 if and only if all messages are pairwise distinct, i.e., iff $m_i \neq m_j$ for $i, j \in [k]$ and $i \neq j$, iff $\mathcal{A}$ made at most $k - 1$ queries to $\mathcal{O}_{\text{Signer}_2}$ with common message τ^* and iff all message-signature pairs $(m_j^*, \sigma_j^*)_{j \in [k]}$ successfully verify. A detailed description of oracles $H_V, H_{\Sigma}, H_M, H_{\text{DLog}}, H_{\text{Ped}}, \text{Next}, \mathcal{O}_{\text{Signer}_1}$ and $\mathcal{O}_{\text{Signer}_2}$ is given in figure Fig. 5. By definition, it holds that

$$\text{AdvOMUF}_{\mathcal{A}}^{\text{BS}}(\lambda) = \epsilon_0.$$

Game 1 (Abort if H_M collision). In order to obtain Game 1, we add a new abort condition to Game 0: Now, the game additionally aborts if any hash collisions $H_M(m_i) = H_M(m_j)$ for $m_i \neq m_j$ arise.

Let us denote the event where there occurs a hash collision for H_M (and Game 1 aborts due to the new abort condition) as E_1 . It holds that $\epsilon_0 = \Pr[\mathcal{A} \text{ wins Game 0} \wedge \neg E_1] + \Pr[\mathcal{A} \text{ wins Game 0} \wedge E_1]$.

By construction, we have $\Pr[\mathcal{A} \text{ wins Game 0} \wedge \neg E_1] = \varepsilon_1$, while $\Pr[\mathcal{A} \text{ wins Game 0} \wedge E_1] \leq \Pr[E_1]$ can be upper bounded via the birthday bound. Therefore, we have

$$|\varepsilon_0 - \varepsilon_1| \leq \frac{Q_M^2}{p}.$$

Game 2 (Sample $v \xleftarrow{\$} \mathbb{Z}_p$ instead of $V \xleftarrow{\$} \mathbb{G}$ in H_V). In Game 2, oracle H_V is modified: instead of sampling $V \xleftarrow{\$} \mathbb{G}$ and setting $\mathcal{Q}_{H_V}[\tau] \leftarrow V$, Game 2 now samples $v \xleftarrow{\$} \mathbb{Z}_p$, sets $T_V[\tau] \leftarrow v$ and $\mathcal{Q}_{H_V}[\tau] \leftarrow vG$ and returns value $\mathcal{Q}_{H_V}[\tau]$.

As the distribution of the return value V of H_V stays the same, the view of $\mathcal{A}$ does not change either. Thus, we have

$$\varepsilon_2 = \varepsilon_1.$$

From now on, for every τ , the game knows the $DLog$ of V which is stored in $T_V[\tau]$.

Game 3 (Compute T^* using v instead of $sk = u$). Game 3 computes the signature T^* without using $sk = u$. More specifically, it sets $T^* = (vU + s^*X_C, s^*G, U)$. As in the previous games, it holds: $T^* = (vU + s^*X_C, s^*G, U) = (uvG + s^*X_C, s^*G, uG) = (uV + s^*X_C, s^*G, uG) = \phi_0(X_C, (s^*, u))$. This dual structure of the signature T^* allows computing valid BB pre-signatures T^* as long as either $sk = u$ or v such that $V = vG$ is known.

As only the way T^* is computed changes, but the value T^* itself stays the same, the view of $\mathcal{A}$ does not change either. Thus, it holds that

$$\varepsilon_3 = \varepsilon_2.$$

Game 4 (Sample $w \xleftarrow{\$} \mathbb{Z}_p$ instead of $W \xleftarrow{\$} \mathbb{G}$). Whenever H_{DLog} is called on an input τ that had not previously served as input to H_{DLog} -queries, Game 4 no longer samples $W \xleftarrow{\$} \mathbb{G}$, but samples $w \xleftarrow{\$} \mathbb{Z}_p$ and sets $W = wG$ instead. Thus, for all common messages τ , the game now knows the witness $w_1 = w$ for $\mathfrak{x}_1 = (G, W)$.

As the distribution of W does not change, the view of $\mathcal{A}$ stays the same as in Game 3. Thus, we have

$$\varepsilon_4 = \varepsilon_3.$$

Game 5 (Use $DLog$ witness for Σ_1). Game 5 no longer uses the simulator Sim_1 to generate the transcript for Σ_1 , but uses its knowledge of $w_1 = w$ to compute the transcript honestly instead. In more detail, this means that Game 5 still samples $c_1^* \xleftarrow{\$} \mathbb{Z}_p$ in $\mathcal{O}_{Signer_1}$, but now runs $(A_1^*, st_1) \leftarrow Init_1(\mathfrak{x}_1, w_1)$ instead of $(A_1^*, z_1^*) \leftarrow Sim_1(\mathfrak{x}_1, c_1^*)$. In $\mathcal{O}_{Signer_2}$, it now computes $z_1^* \leftarrow Resp_1(st_1, c_1^*)$.

Note that c_1^* still gets sampled in the same way as in Game 4. Also, by perfect HVZK of Σ_1 , the distribution of the transcript (A_1^*, c_1^*, z_1^*) does not change in comparison to the previous game. Thus the view of $\mathcal{A}$ stays the same and we have

$$\varepsilon_5 = \varepsilon_4.$$

Game 6 (Simulate Σ_0). Game 6 no longer computes the transcript for Σ_0 honestly, but uses the simulator Sim_0 to generate it instead. More precisely, in $\mathcal{O}_{Signer_1}$, Game 6 samples $c_0^* \xleftarrow{\$} \mathbb{Z}_p$ instead of $c_1^* \xleftarrow{\$} \mathbb{Z}_p$ and runs $(A_0^*, z_0^*) \leftarrow Sim_0(\mathfrak{x}_0^C, c_0^*)$ instead of computing $(A_0^*, st_0) \leftarrow Init_0(\mathfrak{x}_0^t, w_0)$. In calls to $\mathcal{O}_{Signer_2}$, it now computes $c_1^* := c^* - c_0^*$ instead of $c_0^* := c^* - c_1^*$ and gives out the value z_0^* generated in $\mathcal{O}_{Signer_1}$ instead of running $z_0^* \leftarrow Resp_0(st_0, c_0^*)$.

We observe that, by construction, the challenges (c_0^*, c_1^*) are still distributed in the same way as in the previous games. Also, by perfect HVZK the distribution of the transcript (A_0^*, c_0^*, z_0^*) does not change either. Thus the view of $\mathcal{A}$ stays the same and we have

$$\varepsilon_6 = \varepsilon_5.$$

Note that the game now no longer uses the witness $w_0 = (s^*, u)$ for statement $\mathfrak{x}_0^t$. Instead, the game computes the signature T^* through knowledge of v and simulates the transcript for Σ_0 through the simulator Sim_0 .

Game 7 (Extract $(\bar{m}, t)$ from C). In Game 7, a change is made to $\mathcal{O}_{\text{Signer}_1}$: After the initial check that π_{Ped} successfully verifies (i.e., that $\text{NIPS}_{\text{Ped}}.\text{Ver}^{\text{H}_{\text{Ped}}}(\mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}}) = 1$), $\mathcal{O}_{\text{Signer}_1}$ now uses the extractor Ext_{Ped} and the queries made by $\mathcal{A}$ to H_{Ped} (which are stored in $\mathcal{Q}_{\text{H}_{\text{Ped}}}$) to compute a witness $\mathbb{w}_{\text{Ped}}$ for $\mathbb{X}_{\text{Ped}}$. More precisely, $\mathcal{O}_{\text{Signer}_1}$ runs $\mathbb{w}_{\text{Ped}} \leftarrow \text{Ext}_{\text{Ped}}(\mathcal{Q}_{\text{H}_{\text{Ped}}}, \mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}})$, parses $(\bar{m}, t) := \mathbb{w}_{\text{Ped}}$ and aborts if parsing $\mathbb{w}_{\text{Ped}}$ fails or if $C \neq \bar{m}U + tG$. Apart from that, $\mathcal{O}_{\text{Signer}_1}$ remains unchanged.

Let us denote the event where Game 7 aborts due to the new abort condition as E_7 . Analogously to the hop between Games 0 and 1, we have $\varepsilon_6 = \Pr[\mathcal{A} \text{ wins Game 6} \wedge \neg \text{E}_7] + \Pr[\mathcal{A} \text{ wins Game 6} \wedge \text{E}_7]$ and $\Pr[\mathcal{A} \text{ wins Game 6} \wedge \neg \text{E}_7] = \varepsilon_7$ by construction. E_7 happens if extraction of a witness $\mathbb{w}_{\text{Ped}}$ fails or if $\mathbb{w}_{\text{Ped}}$ is not a valid opening for C . The latter case might arise since NIPS_{Ped} is only relaxed knowledge sound and Ext_{Ped} might compute $\mathbb{w}_{\text{Ped}} = u$ such that $uG = U$ instead of a $\mathbb{w}_{\text{Ped}}$ such that $(\mathbb{X}_{\text{Ped}}, \mathbb{w}_{\text{Ped}}) \in \text{R}_{\text{Ped}}$. Let us denote the event where E_7 occurs and witness extraction fails as E_7^{W} and the event where E_7 occurs and witness extraction does not fail as $\text{E}_7^{\neg \text{W}}$. Thus, $\Pr[\text{E}_7] = \Pr[\text{E}_7^{\text{W}}] + \Pr[\text{E}_7^{\neg \text{W}}]$.

We can bound $\Pr[\text{E}_7^{\text{W}}]$ through a straightforward reduction $\mathcal{B}_1$ on the knowledge soundness of NIPS_{Ped} . $\Pr[\text{E}_7^{\neg \text{W}}]$ can be bounded by the probability of a reduction $\mathcal{B}_2$ succeeding to compute the discrete logarithm of U . In more detail, $\mathcal{B}_2$ works as follows: $\mathcal{B}_2$ plays the DLog game and receives a challenge tuple (G, U, L) from its challenger. It performs the setup of Game 6 except that it sets $U = L$ instead of sampling $u \xleftarrow{\$} \mathbb{Z}_p$ and setting $U = uG$. $\mathcal{A}$ receives input $\text{vk} := (G, U, H)$ as well as query access to the random oracles, to oracle Next and to the signing oracles as usual. If $\text{E}_7^{\neg \text{W}}$ occurs while simulating Game 7 to $\mathcal{A}$, $\mathcal{B}_2$ outputs $\mathbb{w}_{\text{Ped}} = u$. Clearly, $\mathcal{B}_2$ wins whenever $\text{E}_7^{\neg \text{W}}$ happens by sending the extracted witness $\mathbb{w}_{\text{Ped}} = u$ to its challenger. Also, the view of $\mathcal{A}$ is the same as in Game 6, as we may assume that the DLog challenge L received by the DLog game follows a uniformly random distribution. Thus, $U = L$ has the same distribution as in Game 6.

Putting everything together, we get:

$$|\varepsilon_7 - \varepsilon_6| \leq \text{AdvKS}_{\mathcal{B}_1}^{\text{NIPS}_{\text{Ped}}, \tilde{\text{R}}_{\text{Ped}}}(\lambda) + \text{AdvDL}_{\mathcal{B}_2}^{\text{GrGen}}(\lambda).$$

Game 8 (Guess τ^*). Game 8 guesses the index of the first query to H_{DLog} that contained the input τ^* . To do so, Game 8 samples a guess $q_{\tau^*} \xleftarrow{\$} [Q_{\text{DLog}}]$ in the beginning and checks whether $\tau^* = \tau_{q_{\tau^*}}$ once it has received the forgeries from $\mathcal{A}$. If the guess was wrong, it aborts.

We note that the queries to H_{DLog} made by the game are included in Q_{DLog} and that the game queries $\text{H}_{\text{DLog}}(\tau^*)$ during verification. Thus, we know that at least one query $\text{H}_{\text{DLog}}(\tau^*)$ must have been made.

We denote the event where Game 8 aborts due to the new abort condition as E_8 . It holds that $\Pr[\text{E}_8] = 1 - \frac{1}{Q_{\text{DLog}}}$. We have $\varepsilon_8 = \Pr[\mathcal{A} \text{ wins Game 7}] \cdot \Pr[\neg \text{E}_8]$ by construction. Therefore, we have

$$\varepsilon_7 \leq Q_{\text{DLog}} \cdot \varepsilon_8.$$

From now on, we can assume that the game knows τ^* . Since $\mathcal{O}_{\text{Signer}_1}$ queries H_{DLog} whenever it is called, the game learns the common message τ^* that $\mathcal{A}$ will use for the forgeries at the latest when $\mathcal{O}_{\text{Signer}_1}$ is called with τ^* for the first time.

Game 9 (Guess unsigned $\bar{m}^*$ in forgery). Game 9 additionally guesses the index q_{m^*} of the first query to H_M for which the following two conditions are satisfied:

1. The input $m_{q_{m^*}}$ of the q_{m^*} -th query to H_M is included in $\mathcal{A}$'s forgeries.
2. If $\text{change}\bar{m}^* = \text{H}_M(m_{q_{m^*}})$ gets extracted from the commitment C in a signing session with common message τ^* , the session does *not* get completed.

Similar to Game 8, Game 9 samples a guess $q_{m^*} \xleftarrow{\$} [Q_M]$ in the beginning and checks whether the above conditions hold for guess q_{m^*} once it has received the forgeries from $\mathcal{A}$. If the guess was wrong, it aborts.

If $\mathcal{A}$ wins Game 9, there must indeed be a message $m_{q_{m^*}} := m_j^*$ with $j \in [k]$ and $q_{m^*} \in [Q_M]$ such that conditions 1 and 2 hold: As $\mathcal{A}$ wins, it must have completed at most $k - 1$ signing sessions for common message τ^* and must have returned k message-signature pairs $(m_j^*, \sigma_j^*)_{j \in [k]}$ with common message τ^* where all k messages m_j^* are pairwise distinct. Since Game 1 excludes collisions for H_M , all k hashed messages $\bar{m}_j^* = \text{H}_M(m_j^*)$ must be pairwise distinct as well. This means, at most $k - 1$ of the hashed messages $\bar{m}_j^*$ can have been extracted in completed sessions. Thus, there is at least one message m_j^* that is part of $\mathcal{A}$'s forgeries for which $\bar{m}_j^* = \text{H}_M(m_j^*)$ did not get extracted in a later completed signing session.

We denote the event where Game 9 aborts due to the new abort condition as E_9 . Analogously to the previous game hop, it holds that $\Pr[E_9] = 1 - \frac{1}{Q_M}$. We have $\varepsilon_9 = \Pr[\mathcal{A} \text{ wins Game 8}] \cdot \Pr[\neg E_9]$ by construction. Therefore, we have

$$\varepsilon_8 \leq Q_M \cdot \varepsilon_9.$$

Subsequently, we use the notation $\bar{m}^* := H_M(m_{q_{m^*}})$. The game samples $\bar{m}^* \xleftarrow{\$} \mathbb{Z}_p$ at the start of the game and sets $\bar{m}^* := H_M(m_{q_{m^*}})$ during the q_{m^*} -th query to random oracle H_M . We may therefore assume, that the game knows $\bar{m}^*$ from the beginning if $\mathcal{A}$ succeeds.

As mentioned before, prospectively, we want to turn the honest OMUF Game into a game $\mathcal{G}$ such that, we can run any adversary $\mathcal{A}$ against $\mathcal{G}$ as a subroutine when we take the role of the adversary in Game BB. We observe, that, if we played Game BB and simulated the current game to an adversary $\mathcal{A}$ on the current game, for $\tau \neq \tau^*$, we could successfully compute $\mathbf{T}^*$ via knowledge of v without knowing $\text{sk} = u$. Also, we could successfully generate both transcripts for the OR-proof by simulating the transcript for Σ_0 (without using $\text{sk} = u$) and computing the transcript of Σ_1 honestly through knowledge of w . Note that for $\tau = \tau^*$ however, when paying Game BB, we will have to set $\mathcal{Q}_{H_V}[\tau^*] = V$ where V is received from Game BB and where we do not know v such that $V = vG$. The computation of the transcripts for Σ_0 and Σ_1 would not be affected by this, as by Game 6, we neither need v nor $\text{sk} = u$ for it. However, by Game 3, the computation of $\mathbf{T}^*$ does depend on the knowledge of v . For $m \neq \bar{m}^*$, this is not an issue, as Game BB provides an oracle giving out valid signatures $\mathbf{T}$ for any $m \neq \bar{m}^*$. For, $\bar{m} = \bar{m}^*$ however, we will not be able to query the oracle. Therefore, we need to find a way to simulate $\mathbf{T}^*$ in that case. This is done in Game 10.

Game 10 (Send random $\mathbf{T}_\* if $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$ was extracted from C by the game).

For the case where $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$ was extracted from C by the game, Game 10 does the following changes to $\mathcal{O}_{\text{Signer}_1}$: It samples $(d_1^*, r_1^*) \xleftarrow{\$} \mathbb{Z}_p^2$ and sets $C_S^* := \text{Com}(d_1^*, r_1^*)$ as before, but now samples $(T_{\$,1}^*, T_{\$,2}^*) \xleftarrow{\$} \mathbb{G}^2$, sets $\mathbf{T}_\$^* = (T_{\$,1}^*, T_{\$,2}^*, U)$ and sets $\mathbf{T}^* := \mathbf{T}_\$^* - (d_1^*G, 0, 0)^T$. Otherwise, $\mathcal{O}_{\text{Signer}_1}$ proceeds exactly as before. (Note that, although value $\mathbf{T}^*$ is not given out to the user, it is still necessary to compute it, as $\mathbf{T}^*$ is part of $\mathbf{x}_0^t$, which Sim_0 takes as input.)

We recall that, due to the constraints introduced in Game 9, for $\tau = \tau^*$, when $\bar{m} = \bar{m}^*$ was extracted from C by the game, the signing session does not get completed. This implies that for $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$, $\mathcal{O}_{\text{Signer}_2}$ does not get called and thus values $\mathbf{z}_0^*, z_1^*, c_0^*, d_1^*$ and r_1^* are never revealed to $\mathcal{A}$. In turn, regarding the probability distributions of the values sent to $\mathcal{A}$ by $\mathcal{O}_{\text{Signer}_1}$ in Game 9 in the case $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$, this means: By construction, r_1^* is uniformly random and only contained in C_S^* . Thus, C_S^* , which is never opened, is uniformly random as well and thereby independent from all other values output by $\mathcal{O}_{\text{Signer}_1}$. As d_1^* and r_1^* never get sent to $\mathcal{A}$, $\mathcal{A}$ cannot compute $\mathbf{T}^*$. Thus, $\mathbf{T}^*$ remains hidden from $\mathcal{A}$ and can be considered as a value only known to the game. As c_0^* never gets sent to $\mathcal{A}$ and as c_1^* never gets computed, c_0^* can also be considered as a uniformly random value only known to the game. Therefore, $\mathbf{A}_0^* \leftarrow \text{Sim}_0(\mathbf{x}_0^t, c_0^*)$ is uniformly random as well: For generating $\mathbf{A}_0^*$, $\text{Sim}_0(\mathbf{x}_0^t, c_0^*)$ samples two fresh values a_1, a_2 uniformly at random and thus, the distribution of $\Pr[(A_{0,1}^*, A_{0,2}^*, A_{0,3}^*)]$ with probability taken over a_1, a_2 and c_0 is uniformly random and independent of all other values output to $\mathcal{A}$. Finally, for $(T_{\$,1}^*, T_{\$,2}^*) = (T_1^*, T_2^*) + (d_1^*G, 0) = (uV + s^*X_C + d_1^*G, s^*G)$, the distribution $\Pr[(T_{\$,1}^*, T_{\$,2}^*)]$, where the probability is taken over s^* and d_1^* , is uniformly random as well.

Thus, the view of $\mathcal{A}$ stays the same as in the previous game and we have

$$\varepsilon_{10} = \varepsilon_9.$$

Finally, we will make sure that the tuple (S_1^*, S_2^*) contained in the forgery (m^*, σ^*) with $H_M(m^*) = \bar{m}^*$ and $\sigma_j^* = (S_1^*, S_2^*, \pi, (d_1, r_1), (d_2, r_2))$ is indeed accepted as a valid solution by Game BB. To do so, we need to make sure that $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \in \mathcal{L}_{\text{bb}}$. We will upper bound the probability that this does not hold by making a reduction on the binding property of the commitment scheme and another reduction on the hardness of computing the DLog of $W = H_{\text{DLog}}(\tau^*)$. We note that Game 10 needs to know the DLog of $W = H_{\text{DLog}}(\tau^*)$ for computing the transcript (A_1, c_1, z_1) . In order to establish our reduction, we therefore need to add two intermediate games that switch back to computing the transcript $(\mathbf{A}_0, c_0, \mathbf{z}_0)$ honestly via w_0 and simulating transcript (A_1, c_1, z_1) for $\tau = \tau^*$. We will undo these changes in two other intermediate games after the reduction.

Game 11.1 (Compute Σ_0 honestly for $\tau = \tau^*$). For $\tau = \tau^*$, Game 11.1 no longer uses the simulator Sim_0 to generate the transcript for Σ_0 , but uses its knowledge of $w_0 = (s^*, \text{sk})$ to compute

the transcript honestly instead. In more detail, this means that for $\tau = \tau^*$ Game 11.1 still samples $c_0^* \xleftarrow{\$} \mathbb{Z}_p$ in $\mathcal{O}_{\text{Signer}_1}$, but now runs $(\mathbf{A}_0^*, \text{st}_0) \leftarrow \text{Init}_0(\mathbb{x}_0, \mathbb{w}_0)$ instead of $(\mathbf{A}_0^*, \mathbf{z}_0^*) \leftarrow \text{Sim}_0(\mathbb{x}_0, c_0^*)$. In $\mathcal{O}_{\text{Signer}_2}$, it now computes $\mathbf{z}_0^* \leftarrow \text{Resp}_0(\text{st}_0, c_0^*)$.

Note that c_0^* still gets sampled in the same way as in Game 10. Also, by perfect HVZK of Σ_0 , the distribution of the transcript $(\mathbf{A}_0^*, c_0^*, \mathbf{z}_0^*)$ does not change in comparison to the previous game. Thus the view of $\mathcal{A}$ stays the same and we have

$$\varepsilon_{11.1} = \varepsilon_{10}.$$

Game 11.2 (Simulate Σ_1 for $\tau = \tau^*$). For $\tau = \tau^*$, Game 11.2 no longer computes the transcript for Σ_1 honestly, but uses simulator Sim_1 to generate it instead. More precisely, for $\tau = \tau^*$ Game 11.2 samples $c_1^* \xleftarrow{\$} \mathbb{Z}_p$ instead of $c_0^* \xleftarrow{\$} \mathbb{Z}_p$ and runs $(\mathbf{A}_1^*, z_1^*) \leftarrow \text{Sim}_1(\mathbb{x}_1, c_1^*)$ instead of computing $(\mathbf{A}_1^*, \text{st}_1) \leftarrow \text{Init}_1(\mathbb{x}_1, \mathbb{w}_1)$. In calls to $\mathcal{O}_{\text{Signer}_2}$ for $\tau = \tau^*$ it now computes $c_0^* := c^* - c_1^*$ instead of $c_1^* := c^* - c_0^*$ and gives out the value $\mathbf{z}_1^*$ generated in $\mathcal{O}_{\text{Signer}_1}$ instead of running $z_1^* \leftarrow \text{Resp}_1(\text{st}_1, c_1^*)$.

We observe that, by construction, the challenges (c_0^*, c_1^*) are still distributed in the same way as in the previous games. Also, by perfect HVZK the distribution of the transcript $(\mathbf{A}_1^*, c_1^*, z_1^*)$ does not change either. Thus the view of $\mathcal{A}$ stays the same and we have

$$\varepsilon_{11.2} = \varepsilon_{11.1}.$$

Game 11.3 (Abort if forgeries not in $\mathcal{L}_{\text{bb}}$). We obtain Game 11.3 by inserting an additional abort condition into Game 11.2. The game now also aborts if one of $\mathcal{A}$'s forgeries is not part of language $\mathcal{L}_{\text{bb}}$, i.e., if there is at least one forged signature $\sigma_j^* = (S_1^*, S_2^*, \pi, (d_1, r_1), (d_2, r_2))$ such that for message m_j^* , $X_j^* = \overline{m}_j^* U + H$ and $\overline{m}_j^* := \text{H}_M(m_j^*)$ it holds that $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \notin \mathcal{L}_{\text{bb}}$.

In more detail, during setup, the game samples $h \xleftarrow{\$} \mathbb{Z}_p$ and sets $H = hG$. Otherwise, the setup proceeds as in Game 11.2 and the oracles are simulated in the same way as in Game 11.2. Once the game receives the k forgeries from $\mathcal{A}$, it checks whether for all $j \in [k]$, the following condition is true:

$$S_{1,j}^* = uV + (\overline{m}_j^* \cdot u)S_{2,j}^* + hS_{2,j}^*. \quad (7)$$

Note that, by construction, the game knows both h and u and thus the condition can be evaluated efficiently. No further changes are made to the game.

Checking the inserted abort condition for all σ_j^* with $j \in [k]$ is indeed equivalent to checking whether it holds for all σ_j^* and their corresponding $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U)$ tuples with $j \in [k]$ that $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \in \mathcal{L}_{\text{bb}}$. This can be seen from the following equivalencies, where $U = uG$:

$$\begin{aligned} \text{Eq. (7)} &\iff S_{1,j}^* = u \cdot V + (\overline{m}_j^* \cdot u)S_{2,j}^* + hS_{2,j}^* \\ &\iff S_{1,j}^* = u \cdot V + (\overline{m}_j^* \cdot u \cdot s_{j,2}^*)G + (h \cdot s_{j,2}^*)G \wedge S_{2,j}^* = s_{j,2}^*G \\ &\iff S_{1,j}^* = u \cdot V + s_{j,2}^*(\overline{m}_j^*U + H) \wedge S_{2,j}^* = s_{j,2}^*G \\ &\iff S_{1,j}^* = u \cdot V + s_{j,2}^* \cdot X_j^* \wedge S_{2,j}^* = s_{j,2}^*G \\ &\iff (G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \in \mathcal{L}_{\text{bb}} \end{aligned}$$

We now look at the advantage $\mathcal{A}$ has in Game 11.3. First, we note that if the new abort condition does not hold, $\mathcal{A}$'s advantage is the same as in Game 11.2 since the distribution of H stays the same and thus the view of the adversary does not change.

Let us denote the event where Game 11.3 aborts but Game 11.2 does not as E . E happens exactly when $\mathcal{A}$ outputs k message-signature pairs $(m_j^*, \sigma_j^*)_{j \in [k]}$ such that for all $(j_1, j_2) \in [k]$ and $j_1 \neq j_2$, $m_{j_1}^* \neq m_{j_2}^*$, when for all $j \in [k]$, σ_j^* successfully verifies and when there exists at least one $j \in [k]$ such that $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \notin \mathcal{L}_{\text{bb}}$. So, $\Pr[\mathbf{G}_{11.3}^A] \geq \Pr[\mathbf{G}_{11.2}^A] - \Pr[\text{E}]$

Denote by E_j the event where the new abort condition is triggered only for message-signature pair (m_j^*, σ_j^*) . Thus, we have $\Pr[\text{E}] = \bigcup_{j=1}^k \Pr[\text{E}_j]$.

In the following, we will upper bound $\Pr[\text{E}_j]$. To do so, we first construct a wrapper $\mathcal{A}_j$ over $\mathcal{A}$. $\mathcal{A}_j$ takes as input the public parameters $\text{par} = ((\mathbb{G}, p, G), \text{par}_{\text{Com}})$ for $(\mathbb{G}, p, G) \leftarrow \text{GrGen}(1^\lambda)$ and $\text{par}_{\text{Com}} \leftarrow \text{Com.Setup}(1^\lambda)$, a value $h \in \mathbb{Z}_p$, an output tape $(c_1, \dots, c_{Q_{\text{H}_\Sigma}})$ and a random tape ρ . $\mathcal{A}_j$ performs the following steps:

- Read $(\mathbf{sk} \in \mathbb{Z}_p, (s_i^*, d_{1,i}^*, r_{1,i}^*, c_{0/1,i}^*, a_{1,i}, a_{2,i})_{i \in [Q_S]}, (h_i)_{i \in [Q_{H_M}]}, (w_i)_{i \in [Q_{H_{DLog}}]}, (p_i)_{i \in [Q_{H_{Ped}}]}, (v_i)_{i \in [Q_{H_V}]}, \rho')$ from the random tape ρ .
- Set $\mathbf{vk} = (G, U, h \cdot G)$.
- Run $(m_j^*, \sigma_j^*)_{j \in [k]} \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Signer}_1}, \mathcal{O}_{\text{Signer}_2}, H_M, H_\Sigma, H_{DLog}, H_V, H_{Ped}, \text{Next}}(\mathbf{vk}; \rho')$ with explicit randomness ρ' . Hereby, $\mathcal{A}$'s queries to the oracles get answered in the following way:
 - Answers to queries to $\mathcal{O}_{\text{Signer}_1}$ and $\mathcal{O}_{\text{Signer}_2}$ with session ID $i \in [Q_S]$ are computed as in Game 11.2 using $\mathbf{sk}$ and random values $s_i^*, d_{1,i}^*, r_{1,i}^*, c_{0/1,i}^*, a_{1,i}$ and $a_{2,i}$. (Here, $a_{1,i}$ and $a_{2,i}$ refer to the random values sampled during the call $(\mathbf{A}_0^*, \mathbf{st}_0) \leftarrow \text{Sim}_0(\mathbb{x}_0, c_0^*)$.)
 - The answer to the i -th new query to $H_M/H_{DLog}/H_{Ped}/H_V$ is computed using random value $h_i/w_i/p_i/v_i$, where $i \in Q_{H_M}/i \in Q_{H_{DLog}}/i \in Q_{H_{Ped}}/i \in Q_{H_V}$ respectively.
 - The answer to the i -th new query to H_Σ is value c_i from the output tape, where $i \in Q_{H_\Sigma}$.
 - The i -th query to Next is answered as in Game 11.2.
- Return $(\perp, \perp)$ if Event $\mathbf{E}_j$ does not occur.
- Return $(I, (m_j^*, \sigma_j^*))$ where I denotes the index of the query to H_Σ that belongs to (m_j^*, σ_j^*) . This means that, for $\sigma_j^* = (S_1^*, S_2^*, \pi^*, (d_1^*, r_1^*), (d_2^*, r_2^*))$, $\pi^* = (\mathbf{A}_0^*, A_1^*, c^*, c_0^*, \mathbf{z}_0^*, z_1^*)$, $\mathbf{A}_{\$,0}^* = \mathbf{A}_0^* - (d_2^*G, 0, 0)^\top$, $\mathbb{x}_0^{C^*} = (G, V, X^*, \mathbf{S}_\$^*)$, $\mathbf{S}_\$^* = \mathbf{S}^* + (d_1^*G, 0, 0)^\top$ and $X^* = \overline{m}^*U + H$, I is the index of the query $H_\Sigma(\mathbb{x}_0^{C^*}, \mathbf{A}_{\$,0}^*, \mathbb{x}_1, A_1, m, C_S, C_{A_0}, \text{par}_{\text{Com}})$ where $\mathbb{x}_0^{C^*} = \mathbb{x}_0^{C^*}$, $\mathbf{A}_{\$,0}^* = \mathbf{A}_{\$,0}^*$, $\mathbb{x}_1 = (G, W)$, $A_1 = A_1^*$, $m = m^*$, $C_S = C_S^*$ and $C_{A_0} = C_{A_0}^*$. As $H_\Sigma(\mathbb{x}_0^{C^*}, \mathbf{A}_{\$,0}^*, \mathbb{x}_1, A_1^*, m^*, C_S^*, C_{A_0}^*, \text{par}_{\text{Com}})$ is called when verifying (m_j^*, σ_j^*) and as we assume that all queries to random oracles made during verification of $\mathcal{A}$'s forgeries had previously also been made by $\mathcal{A}$, such an I must exist.

We note that $\mathcal{A}_j$ and $\mathcal{A}$ roughly have the same runtime.

We now construct an adversary $\mathcal{B}$ running $\mathcal{A}_j$ as a subroutine that either breaks the binding property of the commitment scheme or successfully computes the DLog of W .

$\mathcal{B}$ receives the public commitment parameters par_{Com} for the commitment scheme from the game challenging the binding property of the commitment scheme as well as a DLog challenge tuple (G, G, L) from the DLog game. $\mathcal{B}$ samples $h \xleftarrow{\$} \mathbb{Z}_p$, $c_1, \dots, c_{H_\Sigma} \xleftarrow{\$} \mathbb{Z}_p$ as well as the random tape ρ . The random tape ρ gets sampled uniformly at random except for the slot containing the answer to query $H_{DLog}(\tau^*)$ to which $\mathcal{B}$ writes the DLog challenge L . Note that due to the abort condition introduced in Game 8, we may assume that $\mathcal{B}$ knows the index of the query $H_{DLog}(\tau^*)$ and thus also the correct slot on the tape. $\mathcal{B}$ now runs $(I, (m, \sigma)) \leftarrow \mathcal{A}_j((G, p, G), h, (c_1, \dots, c_{H_\Sigma}), \rho)$. Since we may assume that the challenge L received from the DLog game follows a uniformly random distribution and since we may also assume that the public commitment parameters par_{Com} received from the binding property game follow the same distribution as the parameters generated in the normal setup of our scheme, the view of $\mathcal{A}_j$ is the same as when $\mathcal{B}$ samples ρ completely at random and generates $\text{par}_{\text{Com}} \leftarrow \text{Com.Setup}(1^\lambda)$ itself. If $I = \perp$, $\mathcal{B}$ aborts. Otherwise, it uses the forking strategy by Bellare and Neven by sampling $c'_I, \dots, c'_{H_\Sigma} \xleftarrow{\$} \mathbb{Z}_p$ and running $(I', (m', \sigma')) \leftarrow \mathcal{A}_j((G, p, G), h, (c_1, \dots, c_{I-1}, c'_I, \dots, c'_{H_\Sigma}), \rho)$. Because of the forking lemma we have that with non-negligible probability $I = I' \neq \perp$ and $c_I \neq c'_I$.

In this case, $\mathcal{B}$ obtains two related transcripts for (m, σ) and (m', σ') , where $m = m'$ and $c = c_I \neq c' = c'_I$. Since, when rewinding, all inputs given to the adversary until the answer to the hash query $H_\Sigma(\mathbb{x}_0^C, \mathbf{A}_{\$,0}, \mathbb{x}_1, A_1, m, C_S, C_A, \text{par}_{\text{Com}})$ stay the same, we have $\mathbb{x}_0^C = (G, V, X, \mathbf{S}_\$) = (G', V', X', \mathbf{S}'_\$) = \mathbb{x}_0^{C'}$, $\mathbf{A}_{\$,0} = \mathbf{A}'_{\$,0}$, $\mathbb{x}_1 = \mathbb{x}'_1$, $A_1 = A'_1$, $C_S = C'_S$, $C_A = C'_A$ and $\text{par}_{\text{Com}} = \text{par}_{\text{Com}}'$.

Now, if $d_1 \neq d'_1$, $\mathcal{B}$ has obtained two different valid openings for C_S and thus wins the binding property game by submitting (d_1, r_1) and (d'_1, r'_1) to the challenger. Analogously, if $d_2 \neq d'_2$, $\mathcal{B}$ wins the binding property game by sending (d_2, r_2) and (d'_2, r'_2) to the challenger.

Next, we will consider the case where $d_1 = d'_1$ and $d_2 = d'_2$. In this case, we have $\mathbf{A}_0 = \mathbf{A}_{\$,0} + (d_2G, 0, 0)^\top = \mathbf{A}'_{\$,0} + (d'_2G, 0, 0)^\top = \mathbf{A}_0'$. Also, $\mathbf{S} = \mathbf{S}_\$ - (d_1G, 0, 0)^\top = \mathbf{S}'_\$ - (d'_1G, 0, 0)^\top = \mathbf{S}'$.

Now, either it holds that $c_0 = c'_0$ or $c_0 \neq c'_0$.

case $\mathbf{c}_0 \neq \mathbf{c}'_0$. In this case, we have two related accepting transcripts for m_j^* with $\mathbf{A}_0 = \mathbf{A}'_0$, but $c_0 \neq c'_0$. This means, that by special soundness of Σ_0 we can compute a witness (u, s) proving that $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \in \mathcal{L}_{\text{bb}}$. This is a direct contradiction to our assumption that $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \notin \mathcal{L}_{\text{bb}}$. We may therefore rule out that $c_0 \neq c'_0$.

case $\mathbf{c}_0 = \mathbf{c}'_0$. As $c = c_0 + c_1 \neq c'_0 + c'_1 = c'$ but $c_0 = c'_0$, we must have $c_1 \neq c'_1$.

Again, since (m_k, σ_k) and (m'_k, σ'_k) both successfully verified, we know that both the Σ_1 and the Σ'_1 transcripts successfully verified. Thus, we have $A_1 = z_1G - c_1W$ and $A'_1 = z'_1G - c'_1W$ where $A_1 = A'_1$. Thus,

$$\begin{aligned}
z_1 G - c_1 W &= z'_1 G - c'_1 W \\
z_1 G - z'_1 G &= c_1 W - c'_1 W \\
(z_1 - z'_1) G &= (c_1 - c'_1) w G \\
\frac{z_1 - z'_1}{c_1 - c'_1} G &= w G \\
w &= \frac{z_1 - z'_1}{c_1 - c'_1},
\end{aligned}$$

which is the discrete logarithm of $W = L$. Thus, by the Forking Lemma (cf. Lemma 1), we have $\Pr[E_k] \leq \sqrt{Q_{H_\Sigma}(\text{AdvDL}_B^{\text{G}}(\lambda) + \text{AdvBnd}_B^{\text{Com}}(\lambda)) + \frac{Q_{H_\Sigma}}{p}}$. Recall that k denotes the number of forgeries. Putting everything together, we get

$$|\varepsilon_{11.3} - \varepsilon_{11.2}| \leq k \cdot \left(\sqrt{Q_{H_\Sigma}(\text{AdvDL}_B^{\text{GrGen}}(\lambda) + \text{AdvBnd}_B^{\text{Com}}(\lambda)) + \frac{Q_{H_\Sigma}}{p}} \right).$$

Game 11.4 and Game 11.5. Games 11.4 and 11.5 undo the changes made in Games 11.2 and 11.1 respectively and again do the same computations in the case $\tau = \tau^*$ that they also do in the case $\tau \neq \tau^*$. However, the abort condition introduced in Game 11.3 is kept. By the same argument used in the description of Games 5 and 6, we have that

$$\varepsilon_{11.3} = \varepsilon_{11.4} = \varepsilon_{11.5}.$$

Game 11.5 is given in figure Fig. 6. We will now proceed with constructing an adversary $\mathcal{A}_{\text{BB}}$ against Game BB that runs adversary $\mathcal{A}$ against Game 11.5 as a subroutine.

Reduction to CDH. In the beginning of Game BB, $\mathcal{A}_{\text{BB}}$ samples $\bar{m}^* \xleftarrow{\$} \mathbb{Z}_p$, sends $\bar{m}^*$ to the BB challenger, receives parameter tuple (G, U, V, H) back from the challenger and gets query access to signing oracle $\mathcal{O}$. $\mathcal{A}_{\text{BB}}$ then performs the setup of Game 11.5. Here, it sets $H_V(\tau^*) = V$ and $\text{vk} := (G, U, H)$. Apart from this, the setup proceeds exactly as in Game 11.5. $\mathcal{A}_{\text{BB}}$ then runs $\mathcal{A}$ with input vk . Oracle queries from $\mathcal{A}$ are answered by $\mathcal{A}_{\text{BB}}$ in the following way:

- Queries to $H_M, H_{\text{DLog}}, H_\Sigma, H_{\text{Ped}}, H_V, \text{Next}$ and $\mathcal{O}_{\text{Signer}_2}$ get answered in exactly the same way as in Game 11.5. We recall that $\mathcal{O}_{\text{Signer}_2}$ aborts if it is queried for $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$ where $\bar{m}^*$ had previously been extracted by the game. We also recall that H_M outputs $\bar{m}^*$ on the q_{m^*} -th query.
- Queries to $\mathcal{O}_{\text{Signer}_1}$ with $\tau \neq \tau^*$ or $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$ also get answered exactly as in Game 11.5.
- Upon queries to $\mathcal{O}_{\text{Signer}_1}$ with $\tau = \tau^*$ and $\bar{m} \neq \bar{m}^*$, just like Game 11.5, $\mathcal{A}_{\text{BB}}$ first performs the checks that NIPSPed successfully verifies for x_{Ped} and π_{Ped} and that the witness w_{Ped} extracted by ExtPed can successfully be parsed as $w_{\text{Ped}} = (\bar{m}, t)$ such that $C = \bar{m}U + tG$. Then, $\mathcal{A}_{\text{BB}}$ queries $(S_1, S_2) \leftarrow \mathcal{O}(\bar{m})$, sets $\mathbf{T}^* = (S_1 + tS_2, S_2, U)$ and from there on proceeds exactly like Game 11.5.

Upon receiving the k forgeries (m_j^*, σ_j^*) with common message τ^* from $\mathcal{A}$, $\mathcal{A}_{\text{BB}}$ verifies, whether for any of the messages m_j^* it holds that $H_M(m_j^*) = \bar{m}^*$. If it does find such a message, it parses $(S_1^*, S_2^*, \pi^*) \leftarrow \sigma$ and returns (S_1^*, S_2^*) to the BB challenger.

We first note that the running times of $\mathcal{A}$ and $\mathcal{A}_{\text{BB}}$ are roughly the same. Also, the distribution of vk stays the same as in Game 11.5. All oracle queries are answered in the exact same way as in Game 11.5 except for calls to $\mathcal{O}_{\text{Signer}_1}$ with $\tau = \tau^*$ and $\bar{m} \neq \bar{m}^*$. In the latter case, upon calling $\mathcal{O}(\bar{m})$, $\mathcal{A}_{\text{BB}}$ receives a tuple (S_1, S_2) where $S_1 = uV + sX_{\bar{m}}$, $S_2 = sG$, $s \xleftarrow{\$} \mathbb{Z}_p$ and $U = uG$. For the value $\mathbf{T}^*$ computed by $\mathcal{A}_{\text{BB}}$, it holds that:

$$\begin{aligned}
T_1^* &= S_1 + tS_2 = uV + sX_{\bar{m}} + (t \cdot s)G \\
&= uV + s(\bar{m}U + H + tG) = uV + s(X_C + H).
\end{aligned}$$

Thus, the distribution of the values $\mathbf{T}^*$ computed by $\mathcal{A}_{\text{BB}}$ is exactly the same as the distribution of the values $\mathbf{T}^*$ computed by the challenger in Game 11.5. Note that by construction $\mathcal{A}_{\text{BB}}$ does not know the discrete logarithm of H and therefore cannot check the abort condition introduced in Game 11.3. This, however, does not impact the view of $\mathcal{A}$, as in Games 11.3 to 11.5 the said abort condition

is only checked after the interaction with the adversary has finished. Therefore, the view of $\mathcal{A}$ is the same in the simulation of Game 11.5 by $\mathcal{A}_{\text{BB}}$ as in the original Game 11.5.

We know that, with probability at least $\varepsilon_{11.5}$, $\mathcal{A}$ outputs a message $H_M(m_j^*) = \bar{m}^*$ for which $(G, V, X_{\bar{m}^*}, S_1^*, S_2^*, U) \in \mathcal{L}_{\text{bb}}$ with $X_{\bar{m}^*} := \bar{m}^*U + H$. Thus, with probability at least $\varepsilon_{11.5}$, $\mathcal{A}_{\text{BB}}$ wins Game BB. We have

$$\varepsilon_{11.5} \leq \varepsilon_{\mathcal{A}_{\text{BB}}}^{BB} \leq \text{AdvCDH}_{\mathcal{A}_{\text{CDH}}}^{\text{GrGen}}(\lambda).$$

By putting all equalities and inequalities obtained through the game hops together we obtain Theorem 2.

4.3 Blindness of BS

We give a formal proof of Theorem 3.

Proof. In order to keep our proof as simple as possible, we will construct an upper bound for the advantage $\text{AdvPBlind}_{\mathcal{A}}^{\text{BS-1}}$ of any adversary $\mathcal{A}$ that only starts one single signing session. By a standard hybrid argument we can then directly derive a general upper bound for the advantage of any adversary starting an unlimited number of signing sessions.

We compute the upper bound of $\text{AdvPBlind}_{\mathcal{A}}^{\text{BS-1}}$ through a sequence of games. We start with the standard blindness game as depicted in Fig. 1b and gradually transform it into a game that computes the signatures $\sigma_0 = (S_{1,1}, S_{2,1}, \pi_1, (d_{1,1}, r_{1,1}), (d_{2,1}, r_{2,1}))$ and $\sigma_1 = (S_{1,2}, S_{2,2}, \pi_2, (d_{1,2}, r_{1,2}), (d_{2,2}, r_{2,2}))$ only after the interaction with the signer has ended and that generates all outputs sent to the signer during the protocol run independently from all inputs received by the signer.

Note that, by construction of the blindness experiment, the signing oracles $\mathcal{O}^{\text{User}_0}$, $\mathcal{O}^{\text{User}_1}$ and $\mathcal{O}^{\text{User}_2}$ call their respective user-side signing algorithms USign_0 , USign_1 and USign_2 twice: $\mathcal{O}^{\text{User}_0}$ calls USign_0 once with the message m_b as input and once with the message m_{1-b} as input. Analogously, $\mathcal{O}^{\text{User}_1}$ and $\mathcal{O}^{\text{User}_2}$ call USign_1 and USign_2 , respectively, once on the inputs $(\text{st}_{0,j-1}^U, \text{pm}_{0,j}^S)$ and once on the inputs $(\text{st}_{1,j-1}^U, \text{pm}_{1,j}^S)$, where $j = 1$ and $j = 2$, respectively. In order to differentiate between the inputs, outputs and intermediate values of both calls to the user-side signing algorithms, we use the following notation: We assign one extra index $i \in \{0, 1\}$ to each value, where $i = 0$ for each value involved in the first call to the respective user-side signing algorithm and $i = 1$ for the second one. When it is clear from the context or irrelevant to which call the respective values belong, we omit the extra index.

Game 0 (Honest). In the honest blindness game, the game first invokes $\text{par} \leftarrow \text{Setup}(1^\lambda)$, samples bit $b \xleftarrow{\$} \{0, 1\}$ and sets up the signing oracles $\mathcal{O}^{\text{User}_0}$, $\mathcal{O}^{\text{User}_1}$ and $\mathcal{O}^{\text{User}_2}$. The game then calls $b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{User}_0}, \mathcal{O}^{\text{User}_1}, \mathcal{O}^{\text{User}_2}}()$. If $b' = b$, the game outputs 1, otherwise it outputs 0. By definition, we have

$$\varepsilon_0 := \text{AdvPBlind}_{\mathcal{A}}(\lambda).$$

Game 1 (Abort if H_Σ was queried before $\mathcal{A}$ received (σ_0, σ_1)). Game 1 adds an additional abort condition to the game: Each time $\mathcal{O}^{\text{User}_1}$ calls $c := H_\Sigma(\mathbb{x}_0^C, \mathbf{A}_0^C, \mathbb{x}_1, A_1, m, C_S, C_{A_0}, \text{par}_{\text{Com}})$, the game aborts if H_Σ has been queried on this exact input before by either $\mathcal{A}$ or by the game itself. Let us denote the event where the abort condition holds (and the game aborts) as E_1 . By construction, we have $\varepsilon_0 = \Pr[\mathcal{A} \text{ wins Game 0} \wedge E_1] + \Pr[\mathcal{A} \text{ wins Game 0} \wedge \neg E_1]$ as well as $\varepsilon_1 = \Pr[\mathcal{A} \text{ wins Game 1} \wedge \neg E_1]$. As value A_1 is computed by masking input A_1^* with $\phi_1(z')$, where $z' \xleftarrow{\$} \mathbb{Z}_p$, A_1 is uniformly random and therefore $\Pr[E_1] = \frac{2Q_{H_\Sigma}}{p}$. Thus it holds

$$|\varepsilon_0 - \varepsilon_1| \leq \frac{2Q_{H_\Sigma}}{p}.$$

Game 2 (Guess index of query $H_{\text{DLog}}(\tau)$). Game 2 guesses the index of the first query to H_{DLog} that had τ as input. To do so, Game 2 samples a guess $q_{\tau^*} \xleftarrow{\$} [Q_{\text{DLog}}]$ in its setup phase and checks whether $\tau = \tau_{q_{\tau^*}}$ before oracle $\mathcal{O}^{\text{User}_2}$ outputs the signatures σ_0 and σ_1 . If the guess was wrong, it aborts.

We recall that, Q_{DLog} denotes the total number of queries made to random oracle H_{DLog} during the game and that it counts both the queries made by $\mathcal{A}$ and by the game itself. Furthermore, we note that the oracle $\mathcal{O}^{\text{User}_1}$ needs to include $\mathbb{x}_1 = (G, W)$, where $W = H_{\text{DLog}}(\tau)$, in its queries to H_Σ . Thus,

Game 0 (One-more Unforgeability) <pre> 1 : $\forall H \in \{H_{\text{DLog}}, H_V, H_\Sigma, H_M, H_{\text{Ped}}\}, Q_H[\cdot] := \perp$ 2 : $SID := \emptyset, \text{queried}[\cdot] := 0$ 3 : $\text{common}[\cdot] := \perp, \text{state}[\cdot] := \perp, \text{round}[\cdot] := \perp$ 4 : $u \xleftarrow{\\$} \mathbb{Z}_p, H \xleftarrow{\\$} \mathbb{G}, U := uG$ 5 : $\text{vk} := (G, U, H), \text{sk} := u$ 6 : $\text{oracles} := (\mathcal{O}_{\text{Signer}_1}, \mathcal{O}_{\text{Signer}_2}, H_{\text{DLog}}, H_V, H_\Sigma, H_M, H_{\text{Ped}})$ 7 : $(\tau^*, (m_j^*, \sigma_j^*)_{j \in [k]}) \leftarrow \mathcal{A}^{\text{oracles}}(\text{vk})$ 8 : abort if $\text{queried}[\tau^*] \geq k$ 9 : abort if $\exists j \in [k], \text{Verify}(\text{vk}, m_j^*, \tau^*, \sigma_j^*) = 0$ 10 : abort if $\exists (i, j) \in [k]^2, i \neq j, m_i^* = m_j^*$ 11 : return 1 </pre> $H_{\text{DLog}}(\tau)$ <pre> 1 : if $Q_{H_{\text{DLog}}}[\tau] = \perp$ then 2 : $W \xleftarrow{\\$} \mathbb{G}, Q_{H_{\text{DLog}}}[\tau] := W$ 3 : return $Q_{H_{\text{DLog}}}[\tau]$ </pre> $H_V(\tau)$ <pre> 1 : if $Q_{H_V}[\tau] = \perp$ then 2 : $V \xleftarrow{\\$} \mathbb{G}, Q_{H_V}[\tau] := V$ 3 : return $Q_{H_V}[\tau]$ </pre> $H_\Sigma(x)$ <pre> 1 : if $Q_{H_\Sigma}[x] = \perp$ then 2 : $c \xleftarrow{\\$} \mathbb{Z}_p, Q_{H_\Sigma}[x] := c$ 3 : return $Q_{H_\Sigma}[x]$ </pre> $H_M(m)$ <pre> 1 : if $Q_{H_M}[m] = \perp$ then 2 : $\bar{m} \xleftarrow{\\$} \mathbb{Z}_p, Q_{H_M}[m] := \bar{m}$ 3 : return $Q_{H_M}[m]$ </pre> $H_{\text{Ped}}(x)$ <pre> 1 : if $Q_{H_{\text{Ped}}}[m] = \perp$ then 2 : $y \xleftarrow{\\$} \mathcal{Y}_{\text{Ped}}, Q_{H_{\text{Ped}}}[x] := y$ 3 : return $Q_{H_{\text{Ped}}}[x]$ </pre>	Next(sid, τ) <pre> 1 : if $\text{sid} \in SID$ then return $\perp$ 2 : $SID := SID \cup \{\text{sid}\}$ 3 : $\text{common}[\text{sid}] := \tau, \text{round}[\text{sid}] := 0$ 4 : return 1 </pre> $\mathcal{O}_{\text{Signer}_1}(\text{sid}, C, \pi_{\text{Ped}})$ <pre> 1 : req $\text{round}[\text{sid}] = 0$ 2 : $\tau := \text{common}[\text{sid}]$ 3 : $\mathbb{X}_{\text{Ped}} := (C, U, G)$ 4 : req $\text{NIPS}_{\text{Ped}}.\text{Ver}^{H_{\text{Ped}}}(\mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}}) = 1$ 5 : $s^* \xleftarrow{\\$} \mathbb{Z}_p; \quad \mathbb{w}_0 := (s^*, \text{sk})$ 6 : $d_1^*, r_1^* \xleftarrow{\\$} \mathbb{Z}_p$ 7 : $X_C := C + H$ 8 : $W := H_{\text{DLog}}(\tau)$ 9 : $\mathbf{T}^* := \phi_0(X_C, \mathbb{w}_0); \quad \mathbf{T}_\\$^* := \mathbf{T}^* + (d_1^*G, 0, 0)^T$ 10 : $C_S^* := \text{Com}(d_1^*, r_1^*)$ 11 : $\mathbb{x}_0^t := (G, V, X_C, \mathbf{T}^*); \quad \mathbb{x}_1 := (G, W)$ 12 : $c_1^* \xleftarrow{\\$} \mathbb{Z}_p; \quad (A_1^*, z_1^*) \leftarrow \text{Sim}_1(\mathbb{x}_1, c_1)$ 13 : $(\mathbf{A}_0^*, \text{st}_0) \leftarrow \text{Init}_0(\mathbb{x}_0^t, \mathbb{w}_0)$ 14 : $\text{state}[\text{sid}] \leftarrow (z_1^*, c_1^*, d_1^*, r_1^*, \text{st}_0)$ 15 : $\text{round}[\text{sid}] \leftarrow 1$ 16 : return $(\mathbf{T}_\\$^*, \mathbf{A}_0^*, A_1^*, C_S^*)$ </pre> $\mathcal{O}_{\text{Signer}_2}(\text{sid}, c^*)$ <pre> 1 : req $\text{round}[\text{sid}] = 1$ 2 : req $c \in \mathbb{Z}_p$ 3 : $(z_1^*, c_1^*, d_1^*, r_1^*, \text{st}_0) := \text{state}[\text{sid}]$ 4 : $c_0^* := c^* - c_1^*$ 5 : $\mathbf{z}_0^* \leftarrow \text{Resp}_0(\text{st}_0, c_0^*)$ 6 : $\text{queried}[\tau] \leftarrow \text{queried}[\tau] + 1$ 7 : return $(\mathbf{z}_0^*, z_1^*, c_0^*, d_1^*, r_1^*)$ </pre>
--	---

Fig. 5: Game 0 of the OMUF proof for scheme BS. It is equivalent to the standard one-more unforgeability experiment.

Game 11.5 (One-more Unforgeability)	$\mathcal{O}_{\text{Signer}_1}(\text{sid}, C, \pi_{\text{Ped}})$
1 : $\forall H \in \{H_{\text{DLog}}, H_V, H_\Sigma, H_M, H_{\text{Ped}}\}, Q_H[\cdot] := \perp$ 2 : $STD := \emptyset, Q_T[\cdot] := 0, \text{queried}[\cdot] = 0$ 3 : $\text{common}[\cdot] := \perp, \text{state}[\cdot] = \perp, \text{round}[\cdot] = \perp$ 4 : $\text{ctr}_M := 0, q_{m^*} \xleftarrow{\$} [Q_M], \bar{m}^* \xleftarrow{\$} \mathbb{Z}_p$ // Game 9 5 : $\text{ctr}_{\text{DLog}} := 0, q_{\tau^*} \xleftarrow{\$} [Q_{\text{DLog}}], \tau_{q_{\tau^*}} := \perp$ // Game 8 6 : $h \xleftarrow{\$} \mathbb{Z}_p, H := hG$ // Game 11.3 7 : $u \xleftarrow{\$} \mathbb{Z}_p, U := uG$ 8 : $\text{vk} := (G, U, H), \text{sk} := u$ 9 : $\text{oracles} := (\mathcal{O}_{\text{Signer}_1}, \mathcal{O}_{\text{Signer}_2}, H_{\text{DLog}}, H_V, H_\Sigma, H_M, H_{\text{Ped}})$ 10 : $(\tau^*, (m_j^*, \sigma_j^*)_{j \in [k]}) \leftarrow \mathcal{A}^{\text{oracles}}(\text{vk})$ 11 : abort if $\text{queried}[\tau^*] \geq k$ 12 : abort if $\exists j \in [k], \text{Verify}(\text{vk}, m_j^*, \tau^*, \sigma_j^*) = 0$ 13 : abort if $\exists (i, j) \in [k]^2, i \neq j, m_i^* = m_j^*$ 14 : abort if $\forall j \in [k], H_M(m_j^*) \neq \bar{m}^*$ // Game 9 15 : abort if $\tau^* \neq \tau_{q_{\tau^*}}$ // Game 8 16 : parse $(S_{1,j}^*, S_{2,j}^*, \pi_j)_{j \in [k]} \leftarrow (\sigma_j^*)_{j \in [k]}$ // Game 11.3 abort if $\exists j \in [k], S_{1,j}^* \neq S_j, \pi_j \neq \pi_j^*$ // Game 11.3 17 : $S_j = uV + (H_M(m_j^*) \cdot u)S_{2,j}^* + hS_{2,j}^*$ 18 : return 1 $H_{\text{DLog}}(\tau)$ 1 : $\text{ctr}_{\text{DLog}} \leftarrow \text{ctr}_{\text{DLog}} + 1$ // Game 8 2 : if $\text{ctr}_{\text{DLog}} = q_{\tau^*}$ then $\tau_{q_{\tau^*}} := \tau$ // Game 8 3 : if $Q_{H_{\text{DLog}}}[\tau] = \perp$ then 4 : $w \leftarrow \mathbb{Z}_p, T_{\text{DLog}}[\tau] := w$ // Game 4 5 : $Q_{H_{\text{DLog}}}[\tau] := w \cdot G$ 6 : return $Q_{H_{\text{DLog}}}[\tau]$ $H_V(\tau)$ 1 : if $Q_{H_V}[\tau] = \perp$ then 2 : $v \leftarrow \mathbb{Z}_p, T_V[\tau] := v$ // Game 2 3 : $Q_{H_V}[\tau] := v \cdot G$ 4 : return $Q_{H_V}[\tau]$ $H_M(m)$ 1 : $\text{ctr}_M := \text{ctr}_M + 1$ // Game 9 2 : if $Q_{H_M}[m] = \perp$ then 3 : if $\text{ctr}_M = q_{m^*}$ then $\bar{m} := \bar{m}^*$ // Game 9 4 : else $\bar{m} \xleftarrow{\$} \mathbb{Z}_p$ 5 : abort if $\exists m, Q_{H_M}[m] = \bar{m}$ // Game 1 6 : $Q_{H_M}[m] := \bar{m}$ 7 : return $Q_{H_M}[m]$ $H_\Sigma(x)$ and $H_{\text{Ped}}(x)$ and $\text{Next}(\text{sid}, \tau)$ // Identical to H_Σ and H_{Ped} and Next in Game 0	1 : req $\text{round}[\text{sid}] = 0$ 2 : $\tau := \text{common}[\text{sid}]$ 3 : $\mathbb{X}_{\text{Ped}} := (C, U, G)$ 4 : req $\text{NIPS}_{\text{Ped}}.\text{Ver}^{H_{\text{Ped}}}(\mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}}) = 1$ 5 : $(\bar{m}, t) \leftarrow \text{Ext}_{\text{Ped}}(Q_{H_{\text{Ped}}}, \mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}})$ // Game 7 6 : abort if $C \neq \bar{m}U + tG$ // Game 7 7 : $d_1^*, r_1^* \xleftarrow{\$} \mathbb{Z}_p$ 8 : $X_C := C + H$ 9 : $W := H_{\text{DLog}}(\tau)$ 10 : if $\tau = \tau_{q_{\tau^*}} \wedge \bar{m}^* = \bar{m}$ then // Game 10 11 : $\mathbf{T}^* \xleftarrow{\$} \mathbb{G}^2 \times \{U\}$ // Game 10 12 : else // Game 10 13 : $s^* \xleftarrow{\$} \mathbb{Z}_p, w_0 := (s^*, \text{sk})$ 14 : $\mathbf{T}^* := (vU + s^*X_C, s^*G, U)^\top$ // Game 3 15 : $\mathbf{T}_s^* := \mathbf{T}^* + (d_1^*G, 0, 0)^\top$ 16 : $C_S^* := \text{Com}(d_1^*, r_1^*)$ 17 : $\mathbb{X}_0^t := (G, V, X_C, \mathbf{T}^*); \mathbb{X}_1 := (G, W)$ 18 : $c_0^* \xleftarrow{\$} \mathbb{Z}_p, (\mathbf{A}_0^*, \mathbf{z}_0^*) \leftarrow \text{Sim}_0(\mathbb{X}_0^t, c_0^*)$ // Game 6 19 : $w_1 := T_{\text{DLog}}[\tau]$ // Game 5 20 : $(\mathbf{A}_1^*, \text{st}_1) \leftarrow \text{Init}_1(\mathbb{X}_1, w_1)$ // Game 5 21 : $\text{state}[\text{sid}] := (\mathbf{z}_0^*, c_0^*, d_1^*, r_1^*, \text{st}_1, \bar{m})$ 22 : $\text{round}[\text{sid}] := 1$ 23 : return $(\mathbf{T}_s^*, \mathbf{A}_0^*, \mathbf{A}_1^*, C_S^*)$ $\mathcal{O}_{\text{Signer}_2}(\text{sid}, c^*)$ 1 : req $\text{round}[\text{sid}] = 1$ 2 : req $c \in \mathbb{Z}_p$ 3 : $(z_1^*, c_1^*, d_1^*, r_1^*, \text{st}_0, \bar{m}) := \text{state}[\text{sid}]$ 4 : abort if $\bar{m} = \bar{m}^*$ // Game 9 5 : $c_1^* := c^* - c_0^*$ // Game 6 6 : $z_1^* \leftarrow \text{Resp}_1(\text{st}_1, c_1^*)$ // Game 5 7 : $\text{queried}[\tau] \leftarrow \text{queried}[\tau] + 1$ 8 : return $(\mathbf{z}_0^*, z_1^*, c_0^*, d_1^*, r_1^*)$

Fig. 6: Overview of the games within the OMUF proof for scheme BS. Grey highlights mark the differences to Game 0.

we know that at least one query $H_{\text{DLog}}(\tau)$ must have been made by either $\mathcal{A}$ or the game itself to determine value W . Therefore, q_{τ^*} is well-defined. We also recall that by assumption $\mathcal{A}$ starts only one single signing session and thus only one value τ is used during the game and only one call to $\mathcal{O}^{\text{User}_1}$ is made.

We denote the event where Game 2 aborts due to the new abort condition as E_2 . It holds that $\Pr[E_2] = 1 - \frac{1}{Q_{\text{DLog}}}$. We have $\varepsilon_2 = \Pr[\mathcal{A} \text{ wins Game 1}] \cdot \Pr[\neg E_2]$ by construction. Therefore, we have

$$\varepsilon_1 \leq Q_{\text{DLog}} \cdot \varepsilon_2.$$

Game 3 (Abort if $(G, V, X_C, \mathbf{T}^*) \notin \mathcal{L}_{\text{bb}}$). Game 3 proceeds as Game 2 except that, before returning σ_0 and σ_1 , $\mathcal{O}^{\text{User}_2}$ additionally checks whether $(G, V, X_{C,0}, \mathbf{T}_0^*) \in \mathcal{L}_{\text{bb}}$ and $(G, V, X_{C,1}, \mathbf{T}_1^*) \in \mathcal{L}_{\text{bb}}$, where $V = H_V(\tau)$, $\mathbf{T}_0^* = \mathbf{T}_{\$,0}^* - (d_{1,0}^* G, 0, 0)^\top$ and $\mathbf{T}_1^* = \mathbf{T}_{\$,1}^* - (d_{1,1}^* G, 0, 0)^\top$. Recall that the commitment openings $d_{1,0}^*$ and $d_{1,1}^*$ are sent to $\mathcal{O}^{\text{User}_2}$ as part of the inputs $\text{pm}_{0,2}^S$ and $\text{pm}_{1,2}^S$ from the signer and that the values $\mathbf{T}_{\$,0}^*$ and $\mathbf{T}_{\$,1}^*$ are part of the user states $\text{st}_{0,1}^U$ and $\text{st}_{1,1}^U$, respectively. If at least one of the two checks fails, the game aborts.

To perform the checks, the oracle first inefficiently brute forces u such that $uG = T_{3,0}^*$ and s_0^*, s_1^* such that $s_0^* G = T_{2,0}^*$ and $s_1^* G = T_{2,1}^*$ and then checks whether $uV + s_0^* X_C = T_{1,0}^*$ and $uV + s_1^* X_C = T_{1,1}^*$. Clearly, this is a valid check for testing whether a tuple $(G, V, X_C, \mathbf{T}^*) \in \mathcal{L}_{\text{bb}}$, since iff $(G, V, X_C, \mathbf{T}^*) \in \mathcal{L}_{\text{bb}}$, then $\mathbf{T}^* = \phi_0(X_C, (s', u')) = (u'V + s'X_C, s'G, u'G)$ for some unique $u', s' \in \mathbb{Z}_p$. While the game becomes inefficient, the following steps are purely statistical. In this step, we show that if an abort occurs due to the introduced conditions, there exists an efficient adversary that either breaks the DLog assumption or the binding property of Com.

Let us denote the event where the new abort condition is true (and Game 3 aborts) as E . Let us additionally denote the event where the new abort condition is true for $(G, V, X_{C,0}, \mathbf{T}_0^*)$ as E_0 and the event where the new abort condition is true for $(G, V, X_{C,1}, \mathbf{T}_1^*)$ as E_1 . By construction, we have $\varepsilon_2 = \Pr[\mathcal{A} \text{ wins Game 2} \wedge \neg E] + \Pr[\mathcal{A} \text{ wins Game 2} \wedge E]$. Also by construction, we have $\Pr[\mathcal{A} \text{ wins Game 2} \wedge \neg E] = \varepsilon_3$, $\Pr[E] \leq \Pr[E_0] + \Pr[E_1]$ and $\Pr[E_0] = \Pr[E_1]$. In order to bound $\Pr[E_0]$, we use a similar argument as in Game 11.3 of the OMUF-proof: Again, we first define a wrapper $\mathcal{A}_0$ over $\mathcal{A}$. $\mathcal{A}_0$ takes as input public parameters par , an output tape $(c_1, \dots, c_{Q_{H_\Sigma}})$ as well as a random tape ρ and works as follows:

- Read $((t_i, s'_i, c'_{0,i}, c'_{1,i}, z'_{0,i}, z'_{1,i}, d'_{1,i}, r'_{1,i}, d'_{2,i}, r'_{2,i})_{i \in \{0,1\}}, (h_i)_{i \in [Q_{H_M}]}, (w_i)_{i \in [Q_{H_{\text{DLog}}]}}, (p_i)_{i \in [Q_{H_{\text{Ped}}}]}, (v_i)_{i \in [Q_{H_V}]}, \rho')$ from the random tape ρ .
- Run $\mathcal{A}$ until it has finished its interaction with $\mathcal{O}^{\text{User}_2}$. By then, the oracles have received the input m_b , the corresponding intermediate outputs $\mathbf{T}_\$, \mathbf{A}_0^*, \mathbf{A}_1^*, C_S^*$ as well as the corresponding final outputs $\mathbf{z}_0^*, z_1^*, c_0^*, d_1^*, r_1^*$ from $\mathcal{A}$. (The oracles have also received the same set of corresponding values for message m_{1-b} . However, as wrapper $\mathcal{A}_0$ focuses only on detecting event E_0 , they are of no importance for now). Referring to the set of values belonging to m_b write: $(\mathbf{T}_\$, \mathbf{A}_0^*, \mathbf{A}_1^*, C_S^*, \mathbf{z}_0^*, z_1^*, c_0^*, d_1^*, r_1^*) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{User}_0}, \mathcal{O}^{\text{User}_1}, \mathcal{O}^{\text{User}_2}, H_M, H_\Sigma, H_{\text{DLog}}, H_V, H_{\text{Ped}}}(\text{par})$. Hereby, $\mathcal{A}$'s queries to the oracles get answered in the following way:
 - Answers to queries to the signing oracles are computed as in Game 2 using random values $t_i, s'_i, c'_{0,i}, c'_{1,i}, z'_{0,i}, z'_{1,i}, d'_{1,i}, r'_{1,i}, d'_{2,i}$ and $r'_{2,i}$ where $i = 0$ and $i = 1$ respectively.
 - The answer to the i -th new query to $H_M/H_{\text{DLog}}/H_{\text{Ped}}/H_V$ is computed using random value $h_i/w_i/p_i/v_i$, where $i \in Q_{H_M}/i \in Q_{H_{\text{DLog}}}/i \in Q_{H_{\text{Ped}}}/i \in Q_{H_V}$ respectively.
 - The answer to the i -th new query to H_Σ is value c_i from the output tape, where $i \in Q_{H_\Sigma}$.
- Return $(\perp, \perp)$ if Event E_0 does not occur or if one of the following three checks on the values received by the oracles fails: $\mathbf{A}_0^* = \phi_0(X_C, \mathbf{z}_0^*) - c_0^* \mathbf{T}^*$; $\mathbf{A}_1^* = \phi_1(\mathbf{z}_1^*) - c_1^* W$; $C_S^* = \text{Com}(d_1^*, r_1^*)$.
- Return $(I, (\mathbf{T}_\$, \mathbf{A}_0^*, \mathbf{A}_1^*, C_S^*, \mathbf{z}_0^*, z_1^*, c_0^*, d_1^*, r_1^*))$ where I denotes the index of the first query to H_Σ that was made by $\mathcal{O}^{\text{User}_1}$. As $\mathcal{O}^{\text{User}_1}$ makes exactly two queries to H_Σ during its interaction with $\mathcal{A}$ (one for the set of values belonging to m_b and one for the set of values belonging to m_{1-b}), such an I must exist and must be unique. We also recall that due to the abort condition introduced in Game 1 no query to H_Σ had previously been made that had the same input as the query made by $\mathcal{O}^{\text{User}_1}$.

We note that $\mathcal{A}_0$ and $\mathcal{A}$ roughly have the same runtime.

Similar to Game 11.3 in the OMUF proof, we now construct an adversary $\mathcal{B}$ running $\mathcal{A}_0$ as a subroutine that either breaks the binding property of the commitment scheme or successfully computes the DLog of W .

$\mathcal{B}$ receives the setup parameters par_{Com} for the commitment scheme from the game challenging the binding property of the commitment scheme as well as a DLog challenge tuple (G, G, W) from the DLog game. It then sets $\text{par} \leftarrow ((G, p, G), \text{par}_{\text{Com}})$. Afterwards, it writes the DLog-challenge W on the slot of the random tape that contains the answer to the q_{τ^*} -th query to H_{Σ} and runs $(I, (\mathbf{T}_{\mathbb{S}}^*, \mathbf{A}_0^*, A_1^*, C_S^*, \mathbf{z}_0^*, z_1^*, c_0^*, d_1^*, r_1^*)) \leftarrow \mathcal{A}_0(\text{par}, (c_1, \dots, c_{Q_{\text{H}_{\Sigma}}}), \rho)$. Just like in Game 11.3 of the OMUF proof, we may assume the parameters received from the DLog game and the game challenging the binding property of the commitment scheme follow the same distribution as the parameters that are generated by Game 2. Thus, the view of $\mathcal{A}_0$ is the same as when $\mathcal{B}$ samples ρ completely at random and generates the public parameters itself. If $I = \perp$, $\mathcal{B}$ aborts. Otherwise, it uses the forking strategy by Bellare and Neven by sampling $\widehat{c}_I, \dots, \widehat{c}_{\text{H}_{\Sigma}} \xleftarrow{\$} \mathbb{Z}_p$ and running $(\widehat{I}, (\widehat{\mathbf{T}}_{\mathbb{S}}^*, \widehat{\mathbf{A}}_0^*, \widehat{A}_1^*, \widehat{C}_S^*, \widehat{\mathbf{z}}_0^*, \widehat{z}_1^*, \widehat{c}_0^*, \widehat{d}_1^*, \widehat{r}_1^*)) \leftarrow \mathcal{A}_j(\text{par}, (c_1, \dots, c_{I-1}, \widehat{c}_I, \dots, \widehat{c}_{\text{H}_{\Sigma}}), \rho)$. Because of the forking lemma it holds that with non-negligible probability $I = \widehat{I} \neq \perp$ and $c_I \neq \widehat{c}_I$.

In this case, $\mathcal{B}$ obtains two related transcripts $(m_b, \mathbf{T}_{\mathbb{S}}^*, \mathbf{A}_0^*, c^*, c_0^*, \mathbf{z}_0^*)$ and $(\widehat{m}_b, \widehat{\mathbf{T}}_{\mathbb{S}}^*, \widehat{\mathbf{A}}_0^*, \widehat{c}^*, \widehat{c}_0^*, \widehat{\mathbf{z}}_0^*)$, where $m_b = \widehat{m}_b$ and $c^* + c'_0 + c'_1 = c_I \neq \widehat{c}_I = \widehat{c}^* + \widehat{c}'_0 + \widehat{c}'_1$. Since, when rewinding, all inputs given to the adversary until the answer to the hash query $\text{H}_{\Sigma}(\mathbb{X}_0^C, \mathbf{A}_{\mathbb{S},0}, \mathbb{X}_1, A_1, m, C_S, C_A, \text{par}_{\text{Com}})$ as well as all random values used by both $\mathcal{A}$ and $\mathcal{A}_0$ stay the same, both $\mathcal{A}$ and $\mathcal{A}_0$ do the exact same computations until the said H_{Σ} -query in both runs and we have $\mathbf{T}_{\mathbb{S}}^* = \widehat{\mathbf{T}}_{\mathbb{S}}^*, \mathbf{A}_0^* = \widehat{\mathbf{A}}_0^*, A_1^* = \widehat{A}_1^*, C_S = \widehat{C}_S, C_A = \widehat{C}_A, c'_0 = \widehat{c}'_0$ and $c'_1 = \widehat{c}'_1$. Thus, $c^* \neq \widehat{c}^*$.

We recall that, as $\mathcal{A}_0$ does not output $(\perp, \perp)$, the following three equivalencies hold: $\mathbf{A}_0^* = \phi_0(X_C, \mathbf{z}_0^*) - c_0^* \mathbf{T}^*; A_1^* = \phi_1(z_1^*) - c_1^* W; C_S^* = \text{Com}(d_1^*, r_1^*)$.

Now, if $d_1^* \neq \widehat{d}_1^*$, $\mathcal{B}$ has two valid openings for C_S^* and wins the binding property game by submitting (d_1^*, r_1^*) and $(\widehat{d}_1^*, \widehat{r}_1^*)$ to the challenger.

If $d_1^* = \widehat{d}_1^*$, but $c_0^* \neq \widehat{c}_0^*$, $\mathcal{B}$ has two related accepting transcripts for m_b with $\mathbf{A}_0^* = \widehat{\mathbf{A}}_0^*$. This means, that by special soundness of Σ_0 one can compute a witness (u, s^*) proving that $(G, V, X_C, \mathbf{T}^*) \in \mathcal{L}_{\text{bb}}$. This is a direct contradiction to our assumption that $(G, V, X_C, \mathbf{T}^*) \notin \mathcal{L}_{\text{bb}}$. We may therefore rule out that $c_0^* \neq \widehat{c}_0^*$.

Finally, if $d_1^* = \widehat{d}_1^*$ and $c_0^* \neq \widehat{c}_0^*$, just like in Game 11.3 of the OMUF proof, $\mathcal{B}$ can compute the discrete logarithm of W as $w = \frac{z_1 - z_1'}{c_1 - c_1'}$ because of $A_1^* = z_1^* G - c_1^* W$ and $\widehat{A}_1^* = z_1^* G - \widehat{c}_1^* W$ where $A_1^* = \widehat{A}_1^*$.

By the bound for the probability of success given by the forking lemma, we thus have:

$$|\varepsilon_2 - \varepsilon_3| \leq 2 \cdot (\sqrt{Q_{\text{H}_{\Sigma}}} (\text{AdvDL}_{\mathcal{B}}^{\text{GrGen}}(\lambda) + \text{AdvBnd}_{\mathcal{B}}^{\text{Com}}(\lambda)) + \frac{Q_{\text{H}_{\Sigma}}}{p}).$$

Game 4 (Program H_{Σ}). Game 4 samples the values $(c_i^*, c_{0,i}, c_{1,i})_{i \in \{0,1\}} \xleftarrow{\$} \mathbb{Z}_p$ during setup and programs $\text{H}_{\Sigma}(\mathbb{X}_{0,0}^C, \mathbf{A}_{\mathbb{S},0,0}, \mathbb{X}_{1,0}, A_{1,0}, m_b, C_{S,0}, C_{A_{0,0}}, \text{par}_{\text{Com}}) := c_{0,0} + c_{1,0}$ as well as $\text{H}_{\Sigma}(\mathbb{X}_{0,1}^C, \mathbf{A}_{\mathbb{S},0,1}, \mathbb{X}_{1,1}, A_{1,1}, m_{1-b}, C_{S,1}, C_{A_{0,1}}, \text{par}_{\text{Com}}) := c_{0,1} + c_{1,1}$ in $\mathcal{O}^{\text{User}_2}$. Because of the abort condition introduced in Game 1 and because the distributions of the values $(c_i^*, c_{0,i}, c_{1,i})_{i \in \{0,1\}}$ stay the same as in the previous game, the view of $\mathcal{A}$ does not change either. Thus, we have:

$$\varepsilon_3 = \varepsilon_4.$$

Game 5 (Use HVZK simulation for $\mathbb{X}_1$). In Game 5, $\mathcal{O}^{\text{User}_2}$ computes the transcripts $(A_{1,i}, c_{1,i}, z_{1,i})_{i \in \{0,1\}}$ as a HVZK simulation for $\mathbb{X}_1 = (G, W)$ and $c_{1,0}$ and $c_{1,1}$, respectively.

As the game knows values $c_{1,0}$ and $c_{1,1}$ right from the beginning and programs H_{Σ} when $\mathcal{O}^{\text{User}_2}$ is called, it can defer all randomization computations until the call to $\mathcal{O}^{\text{User}_2}$. Also, by construction, the transcripts $(A_{1,i}, c_{1,i}, z_{1,i}) = (A_{1,i}^* + \phi(z_{1,i}') - c_{1,i}' W, c_{1,i}^* + c_{1,i}', z_{1,i}^* + z_{1,i}')_{i \in \{0,1\}}$ are randomizations of the transcripts $(A_{1,i}^*, c_{1,i}^*, z_{1,i}^*)$ for $i \in \{0,1\}$. As by Lemma 2 the transcripts of Σ_1 are randomizable, the randomized transcripts $(A_{1,i}, c_{1,i}, z_{1,i})$ from Game 4 follow the same distribution as the transcripts generated through HVZK simulation with $c_{1,0}, c_{1,1} \xleftarrow{\$} \mathbb{Z}_p$ in Game 5. Thus, the view of $\mathcal{A}$ does not change and we have:

$$\varepsilon_4 = \varepsilon_5.$$

Game 6 (Use HVZK simulation for $\mathbb{X}_0$). In Game 6, $\mathcal{O}^{\text{User}_2}$ additionally computes the transcripts $(\mathbf{A}_{0,i}, c_{0,i}, \mathbf{z}_{0,i})$ as HVZK simulations for $\mathbb{X}_0 = (G, V, X_i, \mathbf{S}_i)$ and $c_{0,i}$ for $i \in \{0,1\}$.

By construction, in the previous games, it holds for both $\mathbf{A}_{0,i}$:

$$\begin{aligned}
\mathbf{A}_{0,i} &= \mathbf{A}_{\$,0,i} + (d_{2,i}G, 0, 0)^\top \\
&= \mathbf{A}_{0,i}^* - (t_i \cdot A_{0,2,i}^*, 0, 0)^\top + \phi_0(X_i, \mathbf{z}'_{0,i}) - c'_{0,i}\mathbf{S}_{\$,i} - (d'_{2,i}G, 0, 0) + (d_{2,i}G, 0, 0)^\top \\
&= (r_{2,i}V + r_{1,i}X_{C,i}, r_{1,i}G, r_{2,i}G)^\top - (t_i r_{1,i}G, 0, 0)^\top + \phi_0(X_i, \mathbf{z}'_{0,i}) - c'_{0,i}\mathbf{S}_i - c'_{0,i}(d_{1,i}G, 0, 0)^\top \\
&\quad - (d'_{2,i}G, 0, 0)^\top + (d_{2,i}G, 0, 0)^\top \\
&= (r_{2,i}V + r_{1,i}X_i, r_{1,i}G, r_{2,i}G)^\top + \phi_0(X_i, \mathbf{z}'_{0,i}) - c'_{0,i}\mathbf{S}_i \\
&= \mathbf{A}_{0,i}^X + \phi_0(X_i, \mathbf{z}'_{0,i}) - c'_{0,i}\mathbf{S}_i
\end{aligned}$$

where $(\mathbf{A}_{0,i}^X, \mathbf{st}_i) \leftarrow \text{Init}_0(\mathbb{x}_{0,i}, \mathbb{w}_{0,i})$ with $\mathbb{x}_{0,i} = (G, V, X_i, \mathbf{S}_i)$. Thus, in the previous games, the transcripts $(\mathbf{A}_{0,i}, c_{0,i}, \mathbf{z}_{0,i})$ are by construction the randomizations of the transcripts $(\widehat{\mathbf{A}}_{0,i}, \widehat{c}_{0,i}, \widehat{\mathbf{z}}_{0,i})$ with $\widehat{\mathbf{A}}_{0,i} = \mathbf{A}_{0,i}^X$, $\widehat{\mathbf{z}}_{0,i} = \mathbf{z}_{0,i}^* + c_{0,i}^* \cdot (s'_i, 0)$, $\widehat{c}_{0,i} = c_{0,i}^*$ for statements $\widehat{\mathbb{x}}_{0,i} = (G, V, X_i, \mathbf{S}_i)$.

As by Lemma 2 the transcripts of Σ_0 are randomizable, the randomized transcripts $(\mathbf{A}_{0,i}, c_{0,i}, \mathbf{z}_{0,i})$ from the previous games follow the same distribution as the transcripts generated via HVZK simulations with $c_{0,i}, c_{1,i} \xleftarrow{\$} \mathbb{Z}_p$ in Game 6. Thus, again the view of $\mathcal{A}$ does not change and we have:

$$\varepsilon_5 = \varepsilon_6.$$

Game 7 (Self-sign $\overline{m}$). In Game 7, for $i \in \{0, 1\}$, $\mathcal{O}^{\text{User}_2}$ samples $s_i \xleftarrow{\$} \mathbb{Z}_p$ and computes $\mathbf{S}_i = \phi_0(X_i, (s_i, u))$ where $X_0 = \overline{m}_b U + H$ and $X_1 = \overline{m}_{1-b} U + H$. Note that the game can brute-force the discrete logarithm u of U to do so.

Indeed, this does not change the distribution of both values $\mathbf{S}_i$, as in Game 6 it holds:

$$\begin{aligned}
\mathbf{S}_i &= \mathbf{S}_{\$,i} - (d_{1,i}G, 0, 0)^\top \\
&= \mathbf{T}_{\$,i}^* - (t_i \cdot T_{\$,2,i}^*, 0, 0)^\top + \phi_0(X_i, (s'_i, 0)) + (d'_{1,i}G, 0, 0) - (d_{1,i}G, 0, 0)^\top = \\
&= (uV + s_i^* X_{C,i} + d_{1,i}^* G, s_i^* G_i, U)^\top - (t_i \cdot s_i^* G, 0, 0)^\top + \phi_0(X_i, (s'_i, 0)) + (d'_{1,i}G, 0, 0) - ((d_{1,i}^* + d'_{1,i})G, 0, 0)^\top \\
&= (uV + s_i^* X_i, s_i^* G, U)^\top + \phi_0(X_i, (s'_i, 0)) \\
&= \phi_0(X_i, (s_i^*, u)) + \phi_0(X_i, (s'_i, 0)) \\
&= \phi_0(X_i, ((s_i^* + s'_i), u))
\end{aligned}$$

The values s_i in Game 7 and $(s_i^* + s'_i)$ in Game 6 all follow a uniformly random distribution and thus both values $\mathbf{S}_i$ follow the same distribution in both games. Thus, it holds:

$$\varepsilon_6 = \varepsilon_7.$$

Game 8 (Simulate π_{Ped}). In Game 8, $\mathcal{O}^{\text{User}_0}$ simulates the proofs $\pi_{\text{Ped},0}$ and $\pi_{\text{Ped},1}$. To do so, for $i \in \{0, 1\}$, it brute-forces openings $\hat{t}_i$ for $\overline{m} = 0$ and computes proofs $\pi_{\text{Ped},i}$ for this. This is possible as for any Pedersen Commitment $C \in \mathbb{G}$ one can always compute a valid opening for any value v by brute-forcing t .

Clearly, any $\mathcal{A}$ that can distinguish between the proofs $\pi_{\text{Ped},i}$ from Game 7 and the proofs $\pi_{\text{Ped},i}$ as they are computed in Game 8, breaks witness indistinguishability of NIPS. Therefore, through a simple reduction, one can construct an adversary $\mathcal{A}_{\text{WI}}$ breaking witness indistinguishability of NIPS that runs $\mathcal{A}$ as a subroutine. By playing the witness indistinguishability game once for $\pi_{\text{Ped},0}$ and once for $\pi_{\text{Ped},1}$, we get:

$$\varepsilon_7 \leq \varepsilon_8 + 2 \cdot \text{Adv}_{\mathcal{A}_{\text{WI}}}^{\text{NIPS}}(Q_{\text{Ped}}, \lambda).$$

Game 9 ($C \xleftarrow{\$} \mathbb{G}$). In Game 9, $\mathcal{O}^{\text{User}_0}$ samples $C_0, C_1 \xleftarrow{\$} \mathbb{G}$ instead of computing $C_0 := \overline{m}_b U + t_0 G$ and $C_1 := \overline{m}_{1-b} U + t_1 G$. By Game 8, the values $t_0, t_1 \xleftarrow{\$} \mathbb{Z}_p$ are used only in the computation of the values C_0 and C_1 , respectively, and are not used anywhere else afterwards. Thus, also in Game 7, the distributions of C_0 and C_1 are uniformly random. Therefore, we have

$$\varepsilon_8 = \varepsilon_9.$$

Game 10 ($d_1, d_2, r_1, r_2 \xleftarrow{\$} \mathbb{Z}_p$). Finally, in Game 10, $\mathcal{O}^{\text{User}_2}$ samples $d_{1,i}, d_{2,i}, r_{1,i}, r_{2,i} \xleftarrow{\$} \mathbb{Z}_p$ instead of computing $d_{1,i} = d_{1,i}^* + d'_{1,i}$, $r_{1,i} = r_{1,i}^* + r'_{1,i}$, $d_{2,i} = c'_{0,i} \cdot d_{1,i} + d'_{2,i}$ and $r_{2,i} = c'_{0,i} \cdot r_{1,i} + r'_{2,i}$, where $d'_{1,i}, d'_{2,i}, r'_{1,i}, r'_{2,i} \xleftarrow{\$} \mathbb{Z}_p$. By Game 7, the values $d'_{1,i}, d'_{2,i}, r'_{1,i}$ and $r'_{2,i}$ are used only in the computation of the values $d_{1,i}, d_{2,i}, r_{1,i}$ and $r_{2,i}$. Thus, the distributions of $d_{1,i}, d_{2,i}, r_{1,i}$ and $r_{2,i}$ are uniformly random already in the previous games. Therefore, we have

$$\varepsilon_9 = \varepsilon_{10}.$$

Wrapping up. By Game 10, the signatures $\sigma_i = (S_{1,i}, S_{2,i}, \pi_i, (d_{1,i}, r_{1,i}), (d_{2,i}, r_{2,i}))$ are computed only after the interaction with the signer has finished and in a way that is completely independent from the values received by the signer during the interaction. Therefore, the advantage of the adversary in distinguishing between σ_1 and σ_2 is 0.

$$\varepsilon_{10} = 0.$$

By putting together all bounds obtained through the sequence of game hops and a standard hybrid argument, the statement in Theorem 3 follows.

5 TBS: Pairing-free Threshold Blind Signature from CDH

Here, we describe how to enhance our blind signature BS with a threshold signing procedure. We denote the resulting threshold blind signature by TBS. As we base our construction on the blind signature BS, we employ the same notation and building blocks. We refer to Section 3.1 for an overview.

More Preparations. Let us define the PPT algorithm `IssueShares`. On input of value $u \in \mathbb{Z}_p$, number of signers n and threshold $t + 1$, the algorithm $\{(i, u_i)\}_{i \in [n]} \leftarrow \text{IssueShares}(u, n, t + 1)$ computes n Shamir's shares $\{u_i\}_{i \in [n]}$ of value u , of which $t + 1$ will allow to reconstruct value u (cf. ??). The Lagrange coefficient belonging to the key share of signer k for signing set $\mathcal{S}$ is written as $\lambda_k^{\mathcal{S}}$. If the signing set $\mathcal{S}$ is clear from the context, we may omit it. Also, let $H_{\text{com}} : \{0, 1\}^* \rightarrow \mathbb{G}$ denote another hash function and let F_{msk} be a PRF that maps from $\{0, 1\}^*$ to $\mathbb{Z}_p^4$ with key space $\mathcal{K}_{\text{msk}}$.

5.1 Construction

We give our threshold blind signature scheme TBS in Figs. 7 and 8. On a high level, we let each signer generate a partial BB signature via its share u_i and employ a commit-and-open protocol to derive the challenge for the simulated Σ -protocol Σ_1 . Looking ahead, the latter enables us to switch simulation of Σ_1 and Σ_0 in the OMUF proof.

Overview. Let us give a brief description of our scheme TBS. `KeyGen` generates n shares $\{u_i\}_{i \in [n]}$ of secret key u , $t + 1$ of which are necessary to reconstruct u . Each signing party i receives one secret key share.

Signing Protocol. To initiate a signing session, the user again commits in C to $\bar{m} = H_M(m)$ as in our non-threshold blind signature BS. The user sends C and an opening proof π_{ped} to the signers.

To sign the commitment C , each signer k computes a partial BB signature $\mathbf{T}_k^* = \phi_0(X_C, (s_k^*, \lambda_k u_k))$ via its share u_k for $X_C = C + H$. As before, the partial signature $\mathbf{T}_k^*$ is masked via $\mathbf{T}_{\$,k}^* = \mathbf{T}_k^* + (d_{1,k}^*, 0, 0)^T$. Note that each signer chooses its own mask $d_{1,k}^*$ and commits to $d_{1,k}^*$ in $C_{S,k}^*$. Then, signer k initiates a Σ_0 proof via $\mathbf{A}_{0,k}^* \leftarrow \text{Init}_0$ to prove statement $\mathbf{x}_{0,k} = (G, V, X_C, \mathbf{T}_k^*)$ via witness $\mathbf{w}_{0,k} = (s_k^*, \lambda_k u_k)$ and computes a simulated Σ_1 transcript $(A_{1,k}^*, c_{1,k}^*, z_{1,k}^*)$ for the statement $\mathbf{x}_1 = (G, W)$ with challenge $c_{1,k}^* \in \mathbb{Z}_p$. The challenge $c_{1,k}^* = H_{\text{com}}(k, c_{1,k}^*)$ is committed to in cm_k^c via H_{com} .

As in BS, the masked (partial) signature $\mathbf{T}_{\$,k}^*$, the Σ -protocol commitments $\mathbf{A}_{0,k}^*$ and $A_{1,k}^*$ and the commitment $C_{S,k}^*$ are sent to the user. In addition, each signer k also sends the challenge commitment cm_k^c .

Upon receiving all signer messages, the user sums up the $t + 1$ randomized partial signatures $\mathbf{T}_{\$,i}^*$, the $t + 1$ commitments $C_{S,i}^*$, and the $t + 1$ Σ_0 and Σ_1 commitments $\mathbf{A}_{0,i}^*$ and $A_{1,i}^*$, respectively. Due to

linearity, these resulting values form a valid BS signer response which is then blinded by the user as in BS. Also, the user derives the blinded challenge c^* as in BS and forwards it to all signers.

In response, the signers open their challenge commitments $c_{1,k}^*$ and forward their opening to the user. The user sums up the challenges $c_{1,i}^*$ to derive the challenge c_1^* for Σ_1 and forwards the individual challenges to the signers. The signers then verify whether $c_{1,i}^*$ is indeed a correct opening for cm_i^c . If so, each signer derives the challenge c_1^* for Σ_1 (as above) and the challenge $c_0^* = c^* - c_1^*$ for Σ_0 . Finally, each signer sends their opening $(d_{1,k}^*, r_{1,k}^*)$ for $C_{S,k}^*$ and the honest and simulated Σ -protocol responses $\mathbf{z}_{0,k}^*$ and $z_{1,k}^*$ for Σ_0 and Σ_1 , respectively, to the user.

Again, by linearity, it suffices to sum up the openings and the Σ -protocol responses to obtain a valid BS signer response. Then, the user derives its signature as in BS.

Verification. As described above, signatures issued by TBS are of the same form as BS signatures and verification is identical.

Setup(1^λ)	KeyGen($n, t + 1$)
1 : $(\mathbb{G}, p, G) \leftarrow \text{GrGen}(1^\lambda)$	1 : $u \xleftarrow{\$} \mathbb{Z}_p; \quad U := uG$
2 : $\text{par}_{\text{Com}} \leftarrow \text{Com.Setup}(1^\lambda)$	2 : $\{(i, u_i)\}_{i \in [n]} \leftarrow \text{IssueShares}(u, n, t + 1)$
3 : $\text{par} := ((\mathbb{G}, p, G), \text{par}_{\text{Com}})$	3 : $(w_{i,j})_{(i,j) \in [n]^2, i < j} \xleftarrow{\$} \mathcal{K}_{\text{msk}}^{n(n-1)/2}$
4 : return par	4 : $\forall_{i,j \in [n], i > j} w_{i,j} := w_{j,i}$
Verify(vk, m, τ, σ)	5 : $H \xleftarrow{\$} \mathbb{G}$
// Verification as in BS (cf. Section 3)	6 : $\text{vk} := (G, U, H)$
1 : return BS.Verify(vk, m, τ, σ)	7 : for $i \in [n]$ do
	8 : $\text{sk}_i := u_i$
	9 : return (vk, $\{\text{sk}_i\}_{i \in [n]}$)

Fig. 7: The algorithms Setup, KeyGen and Verify belonging to scheme TBS. Differences to blind signature scheme BS due to translation into the threshold setting are highlighted in grey.

5.2 Security Analysis

We show correctness, one-more unforgeability and blindness. We refer to Section 6 for formal proofs.

Correctness. As alluded to above, the correctness of TBS follows from correctness of BS, and from the linearity of the commitment Com and the Σ -protocols.

Theorem 4 (Correctness). *TBS is correct with error γ_{err} , where γ_{err} is the correctness error of NIPS_{Ped} .*

A formal proof of Theorem 4 is given in Section 6.1.

One-more Unforgeability. First, we show that partial signing is compatible with the puncturing strategy, i.e., partial signatures $\mathbf{T}_k^*$ can be simulated for message $m \neq m^*$ while the verification key is punctured for m^* (cf. Lemma 4). Then, we carefully follow the proof strategy of BS (cf. Section 3.3) to argue one-more unforgeability, namely OMUF-3.

We note that the commit-and-open protocol to establish c_1^* is crucial to employ witness indistinguishability of the OR-proof. That is, the simulation can control the agreed upon challenge c_1^* after seeing the user's challenge c^* by programming the honest openings $c_{1,k}^*$ for cm_k^c . This allows to simulate either Σ_0 or Σ_1 at will, and to generate the other with the corresponding witness. Furthermore, we leverage the PRF-based masking to argue that unless some session is completed (i.e., all honest

$\text{ISign}_1(k, \text{sk}_k, \text{aux} := \perp, \text{ctx}, \tau, (C, \pi_{\text{Ped}}, \mathcal{S}))$ <pre> 1 : return $\perp$ if $k \notin \mathcal{S}$ 2 : $\mathbb{X}_{\text{Ped}} := (C, U, G)$ 3 : req $\text{NIPS}_{\text{Ped}}.\text{Ver}^{\text{HPed}}(\mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}}) = 1$ 4 : $s_k^* \xleftarrow{\\$} \mathbb{Z}_p$; $w_{0,k} := (s_k^*, \text{sk}_k = \lambda_k u_k)$ 5 : $d_{1,k}^*, r_{1,k}^* \xleftarrow{\\$} \mathbb{Z}_p$ 6 : $X_C := C + H$; $W := \text{H}_{\text{DLog}}(\tau)$ 7 : $\mathbf{T}_k^* := \phi_0(X_C, w_{0,k})$; $\mathbf{T}_{\\$,k}^* := \mathbf{T}_k^* + (d_k^* G, 0, 0)^\top$ 8 : $C_{S,k}^* := \text{Com.Commit}(d_{1,k}^*, r_{1,k}^*)$ 9 : $\mathbb{X}_{0,k}^t := (G, V, X_C, \mathbf{T}_k^*)$; $\mathbb{X}_1 := (G, W)$ 10 : $(\mathbf{A}_{0,k}^*, \text{st}_{0,k}) \leftarrow \text{Init}_0(\mathbb{X}_{0,k}^t, w_{0,k})$ 11 : $c_{1,k}^* \xleftarrow{\\$} \mathbb{Z}_p$; $\text{cm}_k^c := \text{H}_{\text{cm}}(k, c_{1,k}^*)$ 12 : $(A_{1,k}^*, z_{1,k}^*) \leftarrow \text{Sim}_1(\mathbb{X}_1, c_{1,k}^*)$ 13 : return $(\mathbf{T}_{\\$,k}^*, \mathbf{A}_{0,k}^*, A_{1,k}^*, C_{S,k}^*, \text{cm}_k^c)$ </pre> $\text{ISign}_2(k, (c^*, \{c_{1,i}^c\}_{i \in \mathcal{S}}))$ <pre> 1 : return $c_{1,k}^*$ </pre> $\text{ISign}_3(k, \{c_{1,i}^c\}_{i \in \mathcal{S}})$ <pre> 1 : $\text{ssid} := (\text{ctx}, \mathcal{S}, (c_{1,i}^c)_{i \in \mathcal{S}})$ 2 : for $i \in \mathcal{S}$ do 3 : return $\perp$ if $\text{cm}_i^c \neq \text{H}_{\text{cm}}(i, c_{1,i}^c)$ 4 : $c_1^* := \sum_{i \in \mathcal{S}} c_{1,i}^c$; $c_0^* := c^* - c_1^*$ 5 : $\Delta_k \leftarrow \sum_{j \in \mathcal{S}, j < k} \text{F}_{\text{msk}, w_{k,j}}(\text{ssid})$ 6 : $- \sum_{j \in \mathcal{S}, j > k} \text{F}_{\text{msk}, w_{k,j}}(\text{ssid})$ 7 : $\mathbf{z}_{0,k}^* \leftarrow \text{Resp}_0(\text{st}_{0,k}, c_0^*)$ 8 : $\tilde{\mathbf{z}}_{0,k}^* := \mathbf{z}_{0,k}^* + (\Delta_{k,1}, \Delta_{k,2})$ 9 : $\tilde{d}_{1,k}^* := d_{1,k}^* + \Delta_{k,3}$; $\tilde{r}_{1,k}^* := r_{1,k}^* + \Delta_{k,4}$ 10 : return $(\text{ssid}, \mathcal{S}, (\tilde{\mathbf{z}}_{0,k}^*, z_{1,k}^*, \tilde{d}_{1,k}^*, \tilde{r}_{1,k}^*))$ </pre>	$\text{USign}_0(\text{vk}, \text{aux} := \perp, m, \mathcal{S}, \tau)$ <pre> 1 : $(C, \pi_{\text{Ped}}) \leftarrow \text{BS.USign}_0(\text{vk}, m, \tau)$ 2 : return $(C, \pi_{\text{Ped}}, \mathcal{S})$ </pre> $\text{USign}_1(\{\mathbf{T}_{\$,i}^*, \mathbf{A}_{0,i}^*, A_{1,i}^*, C_{S,i}^*, \text{cm}_i^c\}_{i \in \mathcal{S}})$ <pre> 1 : $\mathbf{T}_{\\$}^* := \sum_{i \in \mathcal{S}} \mathbf{T}_{\\$,i}^*$ $C_S^* := \sum_{i \in \mathcal{S}} C_{S,i}^*$ 2 : $\mathbf{A}_0^* := \sum_{i \in \mathcal{S}} \mathbf{A}_{0,i}^*$; $A_1^* := \sum_{i \in \mathcal{S}} A_{1,i}^*$ 3 : $c^* \leftarrow \text{BS.USign}_1(\mathbf{T}_{\\$}^*, \mathbf{A}_0^*, A_1^*, C_S^*)$ 4 : return $(c^*, \{\text{cm}_i^c\}_{i \in \mathcal{S}})$ </pre> $\text{USign}_2(\{c_{1,i}^c\}_{i \in \mathcal{S}})$ <pre> 1 : $c_1^* := \sum_{i \in \mathcal{S}} c_{1,i}^c$ 2 : return $\{c_{1,i}^c\}_{i \in \mathcal{S}}$ </pre> $\text{USign}_3(\{c_{0,i}^*, \tilde{\mathbf{z}}_{0,i}^*, z_{1,i}^*, \tilde{d}_{1,i}^*, \tilde{r}_{1,i}^*\}_{i \in \mathcal{S}})$ <pre> 1 : $c_0^* := c^* - c_1^*$ 2 : $\mathbf{z}_0^* := \sum_{i \in \mathcal{S}} \tilde{\mathbf{z}}_{0,i}^*$; $z_1^* := \sum_{i \in \mathcal{S}} z_{1,i}^*$ 3 : $d_1^* := \sum_{i \in \mathcal{S}} \tilde{d}_{1,i}^*$; $r_1^* := \sum_{i \in \mathcal{S}} \tilde{r}_{1,i}^*$ 4 : $\sigma \leftarrow \text{BS.USign}_2(c_0^*, \mathbf{z}_0^*, z_1^*, d_1^*, r_1^*)$ 5 : return σ </pre>
---	--

Fig. 8: The interactive signing protocol executed by the signing parties and the user in scheme TBS. Algorithms ISign_1 , ISign_2 and ISign_3 are executed by the signers, algorithms USign_0 , USign_1 , USign_2 and USign_3 are executed by the user. We leave the user and signer states implicit for readability. Notable differences to blind signature scheme BS are highlighted in grey.

signers in $\mathcal{S}$ execute ISign_3 with the same ssid), the response in ISign_3 looks random in the view of the signer. This is important as partial signatures cannot be simulated for the message m^* for which vk is punctured.

Theorem 5 (OMUF-3). *Let p be the order of group $\mathbb{G}$. For any PPT adversary $\mathcal{A}$ making at most Q random oracle queries and finishing at most $k-1$ signing sessions, there exist reductions $\mathcal{A}_{\text{KS}}, \mathcal{A}_{\text{DL}}, \mathcal{A}_{\text{Bnd}}$ and $\mathcal{A}_{\text{CDH}}$ having a runtime similar to $\mathcal{A}$ such that*

$$\begin{aligned} \text{AdvTOMUF}_{\mathcal{A}}^{\text{TBS}}(\lambda) &\leq \frac{2Q^2}{p} + \text{AdvKS}_{\mathcal{A}_{\text{KS}}}^{\text{NIPSPed}, \tilde{\text{R}}_{\text{Ped}}}(\lambda) + \text{AdvDL}_{\mathcal{A}_{\text{DL}}}^{\text{GrGen}}(\lambda) + \\ &Q^2 \cdot \left(k \cdot \left(\sqrt{Q(\text{AdvDL}_{\mathcal{A}_{\text{DL}}}^{\text{GrGen}}(\lambda) + \text{AdvBnd}_{\mathcal{A}_{\text{Bnd}}}^{\text{Com}}(\lambda))} + \frac{Q}{p} \right) + \text{AdvCDH}_{\mathcal{A}_{\text{CDH}}}^{\text{GrGen}}(\lambda) \right). \end{aligned}$$

A formal proof of Theorem 5 is given in Section 6.2.

Blindness. As the user either forwards messages between signers or computes its output as in BS after aggregating the signers' messages, blindness follows immediately.

Theorem 6 (Blindness). *For any adversary $\mathcal{A}$ on blindness of scheme TBS there exists an adversary $\mathcal{A}_{\text{BS}}$ on blindness of scheme BS having runtime similar to $\mathcal{A}$, such that*

$$\text{AdvTPBlind}_{\mathcal{A}}^{\text{TBS}}(\lambda) \leq \text{AdvPBlind}_{\mathcal{A}_{\text{BS}}}^{\text{BS}}(\lambda).$$

A formal proof of Theorem 6 is given in Section 6.3.

6 Formal Security Proofs for TBS

We present formal security proofs for our scheme in Section 5.

6.1 Correctness of TBS

We give a formal proof of Theorem 4.

Proof. We know by correctness of the Σ_0 -protocol, that it holds that

$$\phi_0(X_C, \mathbf{z}_{0,k}^*) = \mathbf{A}_{0,k}^* + c_0^* \mathbf{T}_k^*,$$

where $\mathbf{T}_k^*$ is computed by the signing party k as $\mathbf{T}_k^* = \phi(X_C, (s_k^*, \lambda_k u_k))$, where $\mathbf{T}_k^* = \mathbf{T}_{\$,k}^* - (d_{1,k}^*, 0, 0)^\top$ and where $C_{S,k}^* = \text{Com.Commit}(d_{1,k}^*, r_{1,k}^*)$. We also know by correctness of the Σ_1 -protocol, that $\phi_1(z_{1,k}^*) = A_{1,i}^* + c_{1,k}^* W$.

To show correctness of scheme TBS, we need to show that also for the aggregated messages $\mathbf{A}_0^* = \sum_{k \in \mathcal{S}} \mathbf{A}_{0,k}^*$, $A_1^* = \sum_{k \in \mathcal{S}} A_{1,i}^*$, $z_1^* = \sum_{k \in \mathcal{S}} z_{1,k}^*$, $c_1^* = \sum_{k \in \mathcal{S}} c_{1,k}^*$, $s^* = \sum_{k \in \mathcal{S}} s_k^*$, $C_S^* = \sum_{k \in \mathcal{S}} C_{S,k}^*$ and $u = \sum_{k \in \mathcal{S}} \lambda_k u_k$ as well as for the aggregated blinded messages

$$\begin{aligned} \mathbf{z}_0^* &= \sum_{k \in \mathcal{S}} \tilde{\mathbf{z}}_{0,k}^* = \sum_{k \in \mathcal{S}} \mathbf{z}_{0,k}^* + (\Delta_{k,1}, \Delta_{k,2}) = \sum_{k \in \mathcal{S}} \mathbf{z}_{0,k}^*, \\ d_1^* &= \sum_{k \in \mathcal{S}} \tilde{d}_{1,k}^* = \sum_{k \in \mathcal{S}} d_{1,k}^* + \Delta_{k,3} = \sum_{k \in \mathcal{S}} d_{1,k}^* \end{aligned}$$

and

$$r_1^* = \sum_{k \in \mathcal{S}} \tilde{r}_{1,k}^* = \sum_{k \in \mathcal{S}} r_{1,k}^* + \Delta_{k,4} = \sum_{k \in \mathcal{S}} r_{1,k}^*,$$

where for $j \in [4]$

$$\sum_{k \in \mathcal{S}} \Delta_{k,j} = \sum_{k \in \mathcal{S}} \sum_{j \in \mathcal{S}, j < k} \text{F}_{\text{msk}, w_{k,j}}(\text{ssid}) - \sum_{j \in \mathcal{S}, j > k} \text{F}_{\text{msk}, w_{k,j}}(\text{ssid}) = 0$$

it holds that:

- $\phi_0(X_C, \mathbf{z}_0^*) = \mathbf{A}_0^* + c_0^* \mathbf{T}^*$, where $\mathbf{T}^* = \phi(X_C, (s^*, u))$, $\mathbf{T}_\S^* = \mathbf{T}^* + (d_1^*, 0, 0)^\top$ and $C_S^* = \text{Com.Commit}(d_1^*, r_1^*)$
- $\phi_1(z_1^*) = A_1^* + c_1^* W$.

If these equations do indeed hold, the aggregated values $\mathbf{A}_0^*, A_1^*, \mathbf{z}_0^*, z_1^*, c_1^*, s^*, C_S^*, d_1^*$ computed by the user in scheme TBS are identical to the values $\mathbf{A}_0^*, A_1^*, \mathbf{z}_0^*, z_1^*, c_1^*, s^*, C_S^*, d_1^*$ sent to the user in the non-threshold blind signature scheme BS. Since by construction both schemes perform the exact same operations on these values, correctness of scheme TBS then immediately follows from correctness of scheme BS.

For the combined first flow message $\mathbf{A}_0^*$ it holds that:

$$\begin{aligned}
\mathbf{A}_0^* &= \sum_{k \in \mathcal{S}} \mathbf{A}_{0,k}^* = \sum_{k \in \mathcal{S}} \phi_0(X_C, \mathbf{z}_{0,k}^*) - c_0^* \mathbf{T}_k^* \\
&= \phi_0(X_C, \sum_{k \in \mathcal{S}} \mathbf{z}_{0,k}^*) - c_0^* \sum_{k \in \mathcal{S}} \mathbf{T}_k^* \\
&= \phi_0(X_C, \sum_{k \in \mathcal{S}} \mathbf{z}_{0,k}^*) - c_0^* \sum_{k \in \mathcal{S}} \phi(X_C, (s_k^*, \lambda_k u_k)) \\
&= \phi_0(X_C, \sum_{k \in \mathcal{S}} \mathbf{z}_{0,k}^*) - c_0^* \phi(X_C, (\sum_{k \in \mathcal{S}} s_k^*, \sum_{k \in \mathcal{S}} \lambda_k u_k)) \\
&= \phi_0(X_C, \mathbf{z}_0^*) - c_0^* \phi(X_C, (s^*, u)) \\
&= \phi_0(X_C, \mathbf{z}_0^*) - c_0^* \mathbf{T}^*,
\end{aligned}$$

where $\mathbf{T}^* = \phi(X_C, (s^*, u))$. In the equation, we used linearity of ϕ_0 for fixed X_C to get the second and fourth lines and the fact that by the reconstruction mechanism of Shamir secret sharing $u = \sum_{k \in \mathcal{S}} \lambda_k u_k$ to get the fifth line.

Due to the commitment scheme Com being linearly homomorphic, we also have that

$$\begin{aligned}
C_S^* &= \sum_{k \in \mathcal{S}} C_{S,k}^* = \sum_{k \in \mathcal{S}} \text{Com.Commit}(d_{1,k}^*, r_{1,k}^*) \\
&= \text{Com.Commit}(\sum_{k \in \mathcal{S}} d_{1,k}^*, \sum_{k \in \mathcal{S}} r_{1,k}^*) = \text{Com.Commit}(d_1^*, r_1^*).
\end{aligned}$$

If the user wants to reconstruct $\mathbf{T}^*$, he can compute:

$$\mathbf{T}^* = \sum_{k \in \mathcal{S}} \mathbf{T}_k^* = \sum_{k \in \mathcal{S}} \mathbf{T}_{\S,k}^* - (d_{1,k}^*, 0, 0)^\top = \sum_{k \in \mathcal{S}} \mathbf{T}_{\S,k}^* - (\sum_{k \in \mathcal{S}} d_{1,k}^*, 0, 0)^\top = \mathbf{T}_\S^* - (d_1^*, 0, 0)^\top,$$

where $C_S^* = \text{Com.Commit}(d_1^*, r_1^*)$.

Also, for combined value A_1^* it holds that:

$$\begin{aligned}
A_1^* &= \sum_{k \in \mathcal{S}} A_{1,i}^* = \sum_{k \in \mathcal{S}} \phi_1(z_{1,k}^*) - c_{1,k}^* W \\
&= \phi_1(\sum_{k \in \mathcal{S}} z_{1,k}^*) - \sum_{k \in \mathcal{S}} c_{1,k}^* W \\
&= \phi_1(z_1^*) - c_1^* W.
\end{aligned}$$

This concludes the proof.

6.2 One-more Unforgeability of TBS

We give a formal proof of Theorem 5. First, let us generalize Lemma 3 to the threshold setting.

Lemma 4 (Unforgeability of thresholdized BB). *For any algorithm $\mathcal{A}$, let $\epsilon_{\mathcal{A}}^{\text{TBB}}$ be the probability that the following game outputs 1:*

1. Run $(\overline{m}^*, n, t+1, \mathcal{S}, \mathcal{C}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$ and output 1 if $n < t+1$, $\mathcal{S} \not\subseteq [n]$, $|\mathcal{S}| < t+1$ or $|\mathcal{C}| > t$.
2. Sample $u \xleftarrow{\$} \mathbb{Z}_p$ and set $U := uG$. Compute n shares of u as $\{(i, u_i)\}_{i \in [n]} \xleftarrow{\$} \text{IssueShares}(u, n, t+1)$ and set $U_i = u_i G$.

3. Sample $(H, V) \xleftarrow{\$} \mathbb{G}$ and set $X_{\bar{m}^*} := \bar{m}^*U + H$.
4. Run $(S_1^*, S_2^*) \leftarrow \mathcal{A}^\mathcal{O}(G, U, H, V, \{U_i\}_{i \in \mathcal{S}}, \{u_i\}_{i \in \mathcal{C}}, \text{st}_\mathcal{A})$, where $\mathcal{O}$ is given as:
 - $\mathcal{O}(k, \bar{m})$: Output $\perp$ if $\bar{m} = \bar{m}^*$ or $k \notin \mathcal{S}$. Otherwise, sample $s_k \xleftarrow{\$} \mathbb{Z}_p$, set $X_{\bar{m}} = \bar{m} \cdot U + H$, and compute $\mathbf{S}_k := \phi_0(X_{\bar{m}}, (s_k, \lambda_k u_k))$. Then return $(S_{1,k}, S_{2,k})$.
5. Set $\mathbb{x}_0^* := (G, V, X_{\bar{m}^*}, S_1^*, S_2^*, U)$ and output 1 if and only if $\mathbb{x}_0^* \in \mathcal{L}_{\text{bb}}$.

Then, for any PPT algorithm $\mathcal{A}$, there exists some PPT algorithm $\mathcal{B}$ with running time similar to $\mathcal{A}$ such that

$$\epsilon_{\mathcal{A}}^{\text{TBB}} \leq \text{AdvCDH}_B^{\text{GrGen}}(\lambda).$$

We will hereafter refer to the game presented in Lemma 4 as Game T-BB.

Proof. The main intuition of the final reduction to CDH is that a successful forgery of a signature $(S_1, S_2, S_3) = (uV + s_2X, S_2, U)$ with public key (G, U, V, H) always implicitly contains the CDH tuple uV in component S_1 . If we set $H = -\bar{m}^*U + \delta G$ for some $\delta \in \mathbb{Z}_p$, we get $X_{\bar{m}^*} = \bar{m}^*U + H = \delta G$ and thus

$$S_1^* - \delta S_2^* = uV + s_2^* X_{\bar{m}^*} - \delta S_2^* = uV,$$

which is the isolated CDH tuple.

More formally, we construct a reduction $\mathcal{B}$ that runs an adversary $\mathcal{A}$ against Game T-BB as a subroutine and wins the CDH game, whenever $\mathcal{A}$ wins Game T-BB. $\mathcal{B}$ proceeds as follows: It receives challenge tuple (G, U, V) from the CDH game, runs $(\bar{m}^*, n, t+1, \mathcal{S}, \mathcal{C}, \text{st}_\mathcal{A}) \leftarrow \mathcal{A}(1^\lambda)$ and outputs $\perp$ if $n < t+1$, $\mathcal{S} \not\subseteq [n]$, $|\mathcal{S}| < t+1$ or $|\mathcal{C}| > t$. Afterwards, it simulates a Shamir secret sharing of public key U as follows⁷:

- If $|\mathcal{C}| = t$, $\mathcal{B}$ sets $\mathcal{S}^C = \mathcal{C}$. Otherwise, it samples $t - |\mathcal{C}|$ indices $i \xleftarrow{\$} \mathcal{S} \setminus \mathcal{C}$ and sets $\mathcal{S}^C = \mathcal{C} \cup \{i_j\}_{j \in [t-|\mathcal{C}|]}$.
- For $j \in \mathcal{S}^C$, $\mathcal{B}$ samples $x_j \xleftarrow{\$} \mathbb{Z}_p$. Denote by f the polynomial, where $f(j) = x_j$ for each $j \in \mathcal{S}^C$ and which has the (unknown) discrete logarithm of U as a constant term $f(0) = u$.
- $\mathcal{B}$ then generates the set $\{L_0(Z), \{L_j(Z)\}_{j \in \mathcal{S}^C}\}$ of Lagrange polynomials for set $\mathcal{S}^C$.
- For $i \in [t]$, $\mathcal{B}$ computes the values $Y_i = \ell_{0,i}U + \sum_{j \in \mathcal{S}^C} \ell_{j,i} \cdot x_j \cdot G$, where we write $\ell_{j,i}$ for the i -th coefficient of Lagrange Polynomial $L_j(Z) = \ell_{j,0} + \ell_{j,1}Z + \dots + \ell_{j,t}Z^t$.
- For $i \in [n]$, $\mathcal{B}$ computes the values $X_i = U + \sum_{j \in [t]} i^j Y_j$, which is equal to $f(i) \cdot G$ implicitly.

Thus, $\mathcal{B}$ has obtained a simulated public key share X_i for all $i \in \mathcal{S}$ and corresponding simulated secret key shares x_i for all $i \in \mathcal{S}^C$. Now, $\mathcal{B}$ samples $\delta \xleftarrow{\$} \mathbb{Z}_p$, sets $H = -\bar{m}^*U + \delta G$ and runs $(S_1^*, S_2^*) \leftarrow \mathcal{A}^\mathcal{O}(G, U, H, V, \{U_i\}_{i \in \mathcal{S}}, \{u_i\}_{i \in \mathcal{C}}, \text{st}_\mathcal{A})$. Queries to $\mathcal{O}(k, \bar{m})$ with $\bar{m} = \bar{m}^*$ are answered with $\perp$. Queries for $\bar{m} \neq \bar{m}^*$, are answered in such a way that the answer $S_{1,k}, S_{2,k}$ is a valid signature but can be computed without the knowledge of values u, v and uV :

- If $k \in \mathcal{S}^C$, $\mathcal{B}$ samples $s_{2,k} \xleftarrow{\$} \mathbb{Z}_p$ and computes

$$S_{1,k} = \lambda_k u_k V + s_{2,k}(\bar{m}U + H).$$

- If $k \notin \mathcal{S}^C$, $\mathcal{B}$ sets $S_2 = s_2 G = \frac{\lambda_k}{\lambda_k^c} (r - \frac{v}{\Delta m}) G = \frac{\lambda_k}{\lambda_k^c} (rG - \frac{V}{\Delta m})$, where $\Delta m = \bar{m} - \bar{m}^*$, λ_k denotes the Lagrange coefficient of signing party k for the signing set $\mathcal{S}$ and λ_k^c denotes k 's Lagrange coefficient for the set of parties $\mathcal{S}^C \cup \{k\}$. Indeed, this gives a valid signature and s_2 is set in such a way that the term $\lambda_k u_k V$ gets canceled out and all left-over terms do not contain both u and v :

$$\begin{aligned} S_{1,k} &= \lambda_k r \Delta m U_k + \frac{\lambda_k}{\lambda_k^c} r \delta G - \frac{\lambda_k}{\lambda_k^c \Delta m} \delta V + \Delta m \left(\sum_{i \in \mathcal{S}^C} \lambda_i^c u_i \right) S_{2,k} \\ &= \lambda_k u_k V + \lambda_k r \Delta m U_k + \frac{\lambda_k}{\lambda_k^c} r \delta G - \lambda_k V u_k - \frac{\lambda_k}{\lambda_k^c} \frac{v}{\Delta m} \delta G + \Delta m \left(\sum_{i \in \mathcal{S}^C} \lambda_i^c u_i \right) S_{2,k} \\ &= \lambda_k u_k V + \frac{\lambda_k}{\lambda_k^c} r \Delta m \lambda_k^c U_k + \frac{\lambda_k}{\lambda_k^c} r \delta G - \frac{\lambda_k}{\lambda_k^c} \frac{v}{\Delta m} \Delta m \lambda_k^c U_k - \frac{\lambda_k}{\lambda_k^c} \frac{v}{\Delta m} \delta G + \Delta m \left(\sum_{i \in \mathcal{S}^C} \lambda_i^c u_i \right) S_{2,k} \end{aligned}$$

⁷ The technique for simulatig a sharing of public key U is taken from [?]

$$\begin{aligned}
&= \lambda_k u_k V + \frac{\lambda_k}{\lambda_k^c} \left(r - \frac{v}{\Delta m}\right) (\Delta m \lambda_k^c U_k + \delta G) + \Delta m \left(\sum_{i \in \mathcal{S}^c} \lambda_i^c u_i\right) S_{2,k} \\
&= \lambda_k u_k V + s_{2,k} (\Delta m \lambda_k^c U_k + \delta G) + s_{2,k} \Delta m \left(\sum_{i \in \mathcal{S}^c} \lambda_i^c U_i\right) \\
&= \lambda_k u_k V + s_{2,k} \left(\Delta m \sum_{i \in \mathcal{S}^c \cup \{k\}} \lambda_i^c U_i + \delta G\right) \\
&= \lambda_k u_k V + s_{2,k} (\Delta m U + \delta G) \\
&= \lambda_k u_k V + s_{2,k} (\overline{m} U - \overline{m}^* U + \delta G) \\
&= \lambda_k u_k V + s_{2,k} (\overline{m} U + H).
\end{aligned}$$

Clearly, $\mathcal{B}$ wins the CDH game by returning $S_1^* - \delta S_2^* = uV$, whenever $\mathcal{A}$ wins Game T-BB. We also note that $\mathcal{B}$ has roughly the same runtime as $\mathcal{A}$ and that it perfectly simulates Game T-BB to $\mathcal{A}$, as the value H is distributed uniformly just as in the original game. From this, Lemma 4 immediately follows.

One-more Unforgeability. We are now ready to prove Theorem 5.

Proof. The proof of OMUF-3 of scheme TBS is very similar to the proof of OMUF of scheme BS. Again, we start with the standard OMUF-3 security game and adapt it in a sequence of game hops such that any adversary attacking the game of the last game hop might be run as a subroutine by an adversary attacking Game T-BB. We employ the same notation as in the OMUF proof of BS for the number of random oracle and signing queries. We use the notation $\mathcal{O}^{\text{Sign}_i}$ to refer to queries to signing oracle $\mathcal{O}^{\text{Signer}}$ for signing round i .

Game 0 (Honest). We start with the standard OMUF-3 experiment for threshold blind signatures. In the beginning, the game runs $\text{par} \leftarrow \text{TBS.Setup}(1^\lambda)$. Afterwards, it sets up the bookkeeping variables $\text{ES}_3, \text{El}_3 := \emptyset$ and runs $\mathcal{C} \leftarrow \mathcal{A}(\text{par}, n, t)$, where $\mathcal{C}$ is the set of corrupted signing parties. If $|\mathcal{C}| > t$, it aborts. Then, the game runs $(\text{vk}, \{\text{sk}_i\}_{i \in [n]}) \leftarrow \text{TBS.KeyGen}(n, t)$, where vk and sk_i are defined as in Fig. 7. Finally, it runs $(\tau^*, \ell, \{m_i^*, \sigma_i^*\}_{i \in [\ell]}) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Sign}}}(\text{vk}, \{\text{sk}_i\}_{i \in \mathcal{C}})$ and outputs 1 if $\text{allow}_{\text{SB}}[\tau^*] < \ell$, if all ℓ messages m_i^* are pairwise distinct and if all message-signature pairs successfully verify. Oracle queries $\mathcal{O}^{\text{Sign}}(j, \text{sid}, i, \tau^{\text{sid}}, \text{pm}_{j-1}^{\text{U, sid}})$, where j denotes the signing round and i the index of a signing party are answered as indicated in Fig. 2a: The game first performs the bookkeeping of the OMUF-3 game and then runs $\text{ISign}_1(i, \text{sk}_i, \text{aux}, \text{ctx}^{\text{sid}}, \tau^{\text{sid}}, \text{pm}_0^{\text{U, sid}})$, where $\text{aux} = \perp$ and $\text{pm}_0^{\text{U, sid}} = (\mathcal{S}, C, \pi_{\text{Ped}})$ for $j = 1$ and $\text{ISign}_j(i, \text{st}_{j-1}^{(\mathcal{S}, i, \text{sid})}, \text{pm}_{j-1}^{\text{U, sid}})$, where $\text{st}_{j-1}^{(\mathcal{S}, i, \text{sid})}$ denotes the state of signer i after the $j-1$ -th round and $\text{pm}_{j-1}^{\text{U, sid}}$ denotes the values sent to signing party i by the user in the j -th round for $j \neq 1$. By definition, it holds that

$$\text{AdvTOMUF}_{\mathcal{A}}^{\text{TBS}}(\lambda) = \varepsilon_0.$$

Game 1 (Abort if H_M collision). In order to obtain Game 1, we add a new abort condition to Game 0: Now, the game additionally aborts if any hash collisions $H_M(m_i) = H_M(m_j)$ for $m_i \neq m_j$ arise.

As we can upper bound the probability of such a hash collision occurring by a standard birthday bound, we have

$$|\varepsilon_0 - \varepsilon_1| \leq \frac{Q_M^2}{p}.$$

Game 2 (Sample $v \xleftarrow{\$} \mathbb{Z}_p$ instead of $V \xleftarrow{\$} \mathbb{G}$ in H_V). In Game 2, oracle H_V is modified: Instead of sampling $V \xleftarrow{\$} \mathbb{G}$ and setting $Q_{H_V}[\tau] \leftarrow V$, Game 2 now samples $v \xleftarrow{\$} \mathbb{Z}_p$, sets $T_V[\tau] \leftarrow v$ and $Q_{H_V}[\tau] \leftarrow vG$ and returns value $Q_{H_V}[\tau]$.

As the distribution of the return value V of H_V stays the same, the view of $\mathcal{A}$ does not change either. Thus, we have

$$\varepsilon_2 = \varepsilon_1.$$

From now on, for every τ , the game knows the DLog of V which is stored in $Q_{H_V}^{\text{DLog}}[\tau]$.

Game 3 (Compute $\mathbf{T}_k^*$ using v instead of u_k). Game 3 computes the signature $\mathbf{T}_k^*$ without using u_k . More specifically, it sets $\mathbf{T}_k^* = (\lambda_k v U_k + s_k^* X_C, s_k^* G, \lambda_k U_k)$. As in the previous games, it holds: $\mathbf{T}_k^* = (\lambda_k v U_k + s_k^* X_C, s_k^* G, \lambda_k U_k) = (\lambda_k u_k v G + s_k^* X_C, s_k^* G, \lambda_k u_k G) = (\lambda_k u_k V + s_k^* X_C, s_k^* G, \lambda_k u_k G) = \phi_0(X_C, (s_k^*, \lambda_k u_k))$. Again, just like in the OMUF proof of scheme BS, this is possible due to the dual structure of the signature $\mathbf{T}^*$, which allows computing valid signatures $\mathbf{T}^*$ as long as either u or v such that $V = vG$ is known.

As only the way $\mathbf{T}_k^*$ is computed changes, but the value $\mathbf{T}_k^*$ itself stays the same, the view of $\mathcal{A}$ does not change either. Thus, it holds that

$$\varepsilon_3 = \varepsilon_2.$$

Game 4 (Sample $w \xleftarrow{\$} \mathbb{Z}_p$ instead of $W \xleftarrow{\$} \mathbb{G}$). Whenever $\mathbf{H}_{\text{DLog}}$ is called on an input τ that had not previously served as input to $\mathbf{H}_{\text{DLog}}$ -queries, Game 4 no longer samples $W \xleftarrow{\$} \mathbb{G}$, but samples $w \xleftarrow{\$} \mathbb{Z}_p$ and sets $W = wG$ instead. Thus, for all common messages τ , the game now knows the witness $w_1 = w$ for $\mathbb{x}_1 = (G, W)$.

As the distribution of W does not change, the view of $\mathcal{A}$ stays the same as in Game 3. Thus, we have

$$\varepsilon_4 = \varepsilon_3.$$

Game 5 (Use DLog witness for Σ_1). Game 5 no longer uses the simulator Sim_1 to generate the transcripts for Σ_1 , but uses its knowledge of $w_1 = w$ to compute the transcripts honestly instead. In more detail, this means that, in calls to $\mathcal{O}^{\text{ISign}_1}$ for signer $k \in \mathcal{S}$, Game 5 still samples $c_{1,k}^* \xleftarrow{\$} \mathbb{Z}_p$, but now runs $(A_{1,k}^*, \text{st}_{1,k}) \leftarrow \text{Init}_1(\mathbb{x}_1, w_1)$ instead of $(A_{1,k}^*, z_1^*) \leftarrow \text{Sim}_1(\mathbb{x}_1, c_{1,k}^*)$. In $\mathcal{O}^{\text{ISign}_3}$, the game now additionally computes $z_{1,k}^* \leftarrow \text{Resp}_1(\text{st}_{1,k}, c_{1,k}^*)$.

Note that $c_{1,k}^*$ still gets sampled in the same way as in Game 4. Also, by perfect HVZK of Σ_1 , the distribution of the transcripts $(A_{1,k}^*, c_{1,k}^*, z_{1,k}^*)$ does not change in comparison to the previous game. Thus the view of $\mathcal{A}$ stays the same and we have

$$\varepsilon_5 = \varepsilon_4.$$

Game 6 (Simulate Σ_0). Game 6 no longer computes the transcripts for Σ_0 honestly, but uses simulator Sim_0 to generate it instead. More precisely, the following changes are made to the signing oracles: When $\mathcal{O}^{\text{ISign}_1}$ gets called for the first time in a session, the game computes $c_0^* \xleftarrow{\$} \mathbb{Z}_p$. For all following calls to $\mathcal{O}^{\text{ISign}_1}$ in the same session, the previously computed value c_0^* stays the same. Thus all honest signers use the same c_0^* throughout one session. Furthermore, in calls to $\mathcal{O}^{\text{ISign}_1}$ for signer $k \in \mathcal{S}$, the game sets commitment $\text{cm}_k^c \xleftarrow{\$} \mathbb{Z}_p$ and computes $\mathbf{A}_{0,k}^*$ by $(\mathbf{A}_{0,k}^*, \mathbf{z}_{0,k}^*) \leftarrow \text{Sim}_0(\mathbb{x}_{0,k}^t, c_0^*)$. When $\mathcal{O}^{\text{ISign}_2}$ gets called for the first time in a session, the game computes $c_1^* = c^* - c_0^*$, samples $c_{1,i}^* \xleftarrow{\$} \mathbb{Z}_p$ for each of the first $x - 1$ honest signers and sets $c_{1,x}^* = c_1^* - \sum_{i \in \mathcal{S} \setminus \{x\}} c_{1,i}^*$, where $x = |\mathcal{S}_{\text{hon}}|$ denotes the number of honest signers in the signing set. Note that the game knows all values $c_{1,i}^*$ for $i \in \mathcal{S} \setminus \{x\}$: For honest signers the game itself samples $c_{1,i}^*$ and for corrupted signers it knows $c_{1,i}^*$, as $\mathcal{A}$ needs to call $\mathbf{H}_{\text{cm}}(i, c_{1,i}^*)$ to obtain value cm_i^c . The game then programs $\mathbf{H}_{\text{cm}}(k, c_{1,k}^*) = \text{cm}_k^c$ for each $k \in \mathcal{S} \setminus \mathcal{C}$. When $\mathcal{O}^{\text{ISign}_2}$ gets queried again in the same session for a different signing party k , the oracle simply returns the previously computed value $c_{1,k}^*$. In calls to $\mathcal{O}^{\text{ISign}_3}$ for signer $k \in \mathcal{S}$, the game outputs the value $\mathbf{z}_{0,k}$ computed in $\mathcal{O}^{\text{ISign}_1}$.

We see that, just like in Game 5, c_0^* as well as all $c_{1,k}^*$ are distributed uniformly at random (c_0^* and $\{c_{1,k}^*\}_{k \in [x-1]}$ directly by construction and $c_{1,x}^*$ as it is the difference of two random values). Thus, by perfect HVZK of Σ_0 , all partial transcripts $\Sigma_{0,k}$ are distributed identically as in Game 5. Finally, observe that the adversary can only detect the programming of $\mathbf{H}_{\text{cm}}$ if it manages to query some partial challenge $c_{1,i}^*$ before it is revealed. As there are at most Q_S such commitments in total, a union bound over all hash queries $Q_{\mathbf{H}_{\text{cm}}}$ to $\mathbf{H}_{\text{cm}}$ yields

$$|\varepsilon_6 - \varepsilon_5| \leq \frac{Q_{\mathbf{H}_{\text{cm}}} Q_S}{p}.$$

Note that the game now no longer uses the witness $w_{0,k}$ for statement $\mathbb{x}_{0,k}^t$: It computes the signature $\mathbf{T}_k^*$ through knowledge of v and simulates the transcript for $\Sigma_{0,k}$ through the simulator Sim_0 .

Game 7 (Extract $(\bar{m}, t)$ from C). In Game 7, a change is made to $\mathcal{O}^{\text{ISign}_1}$: After the initial check that π_{Ped} successfully verifies (i.e., that $\text{NIPSPed.Ver}^{\text{HPed}}(\mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}}) = 1$), $\mathcal{O}^{\text{ISign}_1}$ now uses the extractor Ext_{Ped} and the queries made by $\mathcal{A}$ to H_{Ped} (which are stored in $\mathcal{Q}_{\text{HPed}}$) to compute a witness $\mathbb{w}_{\text{Ped}}$ for $\mathbb{X}_{\text{Ped}}$. More precisely, $\mathcal{O}^{\text{ISign}_1}$ runs $\mathbb{w}_{\text{Ped}} \leftarrow \text{Ext}_{\text{Ped}}(\mathcal{Q}_{\text{HPed}}, \mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}})$, parses $(\bar{m}, t) := \mathbb{w}_{\text{Ped}}$ and aborts if parsing $\mathbb{w}_{\text{Ped}}$ fails or $C \neq \bar{m}U + tG$. Apart from that, $\mathcal{O}^{\text{ISign}_1}$ remains unchanged.

Exactly like in Game 7 of the proof of one-more unforgeability of scheme BS, we can upper bound the probability that the game aborts due to the new abort condition by making the same two reductions on the knowledge soundness of NIPSPed and the hardness of computing the discrete logarithm of U , respectively. From the reductions, we obtain that

$$|\varepsilon_7 - \varepsilon_6| \leq \text{AdvKS}_{\mathcal{B}_1}^{\text{NIPSPed}, \bar{\text{R}}_{\text{Ped}}}(\lambda) + \text{AdvDL}_{\mathcal{B}_2}^{\text{GrGen}}(\lambda).$$

Game 8 (Guess τ^*). Game 8 guesses the index of the first query to H_{DLog} that contained τ^* . To do so, Game 8 samples a guess $q_{\tau^*} \xleftarrow{\$} [Q_{\text{DLog}}]$ in the beginning and checks whether $\tau^* = \tau_{q_{\tau^*}}$ once it has received the forgeries from $\mathcal{A}$. If the guess was wrong, it aborts.

Just like in the OMUF proof for scheme BS, we know that at least one query $\text{H}_{\text{DLog}}(\tau^*)$ must have been made, as the game queries $\text{H}_{\text{DLog}}(\tau^*)$ during verification. Since the probability that the game guesses q_{τ^*} right is $1/Q_{\text{DLog}}$, we get

$$\varepsilon_7 \leq Q_{\text{DLog}} \cdot \varepsilon_8.$$

From now on, we can assume that the game knows τ^* . Since $\mathcal{O}^{\text{ISign}_1}$ queries H_{DLog} whenever it is called, the game learns the common message τ^* that $\mathcal{A}$ will use for the forgeries at the latest when $\mathcal{O}^{\text{ISign}_1}$ is called with τ^* for the first time.

Game 9 (Guess unsigned $\bar{m}^*$ in forgery). Game 9 additionally guesses the index q_{m^*} of the first query to H_{M} for which the following two conditions are satisfied:

1. The input $m_{q_{m^*}}$ of the q_{m^*} -th query to H_{M} is included in $\mathcal{A}$'s forgeries.
2. If $\bar{m}^* = \text{H}_{\text{M}}(m_{q_{m^*}})$ gets extracted from the commitment C in a signing session with common message τ^* , the session does *not* get *completed for more than $|\mathcal{S}_{\text{hon}}| - 1$ signing parties*, where $\mathcal{S}_{\text{hon}}$ is the subset of signing set $\mathcal{S}$ that consists of all honest signing parties.

Similar to Game 8, Game 9 samples a guess $q_{m^*} \xleftarrow{\$} [Q_{\text{M}}]$ in the beginning and checks whether the above conditions hold for guess q_{m^*} once it has received the forgeries from $\mathcal{A}$. If the guess was wrong, it aborts. If $\mathcal{A}$ wins Game 9, there must indeed be a message $m_{q_{m^*}} := m_j^*$ with $j \in [\ell]$ and $q_{m^*} \in [Q_{\text{M}}]$ such that conditions 1 and 2 hold: As $\mathcal{A}$ wins, there must be at most $\ell - 1$ signing sessions with common message τ^* that got completed for all $|\mathcal{S}_{\text{hon}}|$ honest signers that are part of the signing set and $\mathcal{A}$ must have returned ℓ message-signature pairs $(m_j^*, \sigma_j^*)_{j \in [\ell]}$ with common message τ^* , where all messages m_j^* are pairwise distinct. Since Game 1 excludes collisions for H_{M} , all ℓ hashed messages $\bar{m}_j^* = \text{H}_{\text{M}}(m_j^*)$ must be pairwise distinct as well. This means, at most $\ell - 1$ of the hashed messages $\bar{m}_j^*$ can have been extracted in a signing session with common message τ^* that got completed for $|\mathcal{S}_{\text{hon}}|$ signing parties and there is at least one message m_j^* for which this is not the case.

As the probability that the game guesses q_{m^*} right is $1/Q_{\text{M}}$, we have

$$\varepsilon_8 \leq Q_{\text{M}} \cdot \varepsilon_9.$$

Subsequently, we use the notation $\bar{m}^* := \text{H}_{\text{M}}(m_{q_{m^*}})$. The game samples $\bar{m}^* \xleftarrow{\$} \mathbb{Z}_p$ at the start of the game and sets $\bar{m}^* := \text{H}_{\text{M}}(m_{q_{m^*}})$ during the q_{m^*} -th query to random oracle H_{M} . We may therefore assume that the game knows $\bar{m}^*$ from the beginning, if $\mathcal{A}$ succeeds.

Game 10 (Abort if $w_{i,j} = w_{k,l}$ for any $\{i, j\} \neq \{k, l\}$). Game 10 introduces an additional abort condition and now aborts if for any two signers i, j the shared key $w_{i,j}$ is equal to the shared key $w_{k,l}$ of any two other signers k, l with $\{i, j\} \neq \{k, l\}$. The probability of such a collision occurring can be upper bounded by a standard birthday bound. It thus holds that

$$|\varepsilon_9 - \varepsilon_{10}| \leq \frac{(n^2 - n)^2}{4 \cdot |\mathcal{K}_{\text{msk}}|}.$$

Game 11 (Replace F_{msk} with random functions for honest signer keys). Game 11 replaces each instance of $\text{F}_{\text{msk}, w_{i,j}}$ that uses a key $w_{i,j}$ shared between two honest users i and j with a truly random

function. This is possible as $\mathcal{A}$ does not know any of the keys that are shared between two honest users and all shared keys are distinct. Thus, the values computed through $F_{\text{msk}, w_{i,j}}$ look uniformly random to $\mathcal{A}$ assuming PRF security. More formally, for any such $F_{\text{msk}, w_{i,j}}$ that is replaced by a truly random function an adversary distinguishing between the two would break PRF security of F_{msk} . Thus, by applying a standard reduction on PRF security of F_{msk} for each of the instances of $F_{\text{msk}, w_{i,j}}$ that uses a key $w_{i,j}$ shared between two honest users and doing a union bound, we get the following inequality:

$$|\varepsilon_{10} - \varepsilon_{11}| \leq \frac{n^2 - n}{2} \cdot \text{AdvPRF}_{\mathcal{A}}^{F_{\text{msk}}}(1^\lambda).$$

Game 12 (Send random values for $\tilde{\mathbf{z}}_{0,k}^*, \tilde{d}_{1,k}^*$ and $\tilde{r}_{1,k}^*$ when $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$). For $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$, Game 12 makes the following changes to the way signing queries to $\mathcal{O}^{\text{Sign}_3}$ are answered: it sets $\mathbf{z}_{0,k}^* \xleftarrow{\$} \mathbb{Z}_p^2$ as well as $\tilde{d}_{1,k}^*, \tilde{r}_{1,k}^* \xleftarrow{\$} \mathbb{Z}_p$, while all other values get computed just as before.

By construction, $\tilde{\mathbf{z}}_{0,k}^* := \mathbf{z}_{0,k}^* + (\Delta_{k,1}, \Delta_{k,2})$, $\tilde{d}_{1,k}^* := d_{1,k}^* + \Delta_{k,3}$ and $\tilde{r}_{1,k}^* := r_{1,k}^* + \Delta_{k,4}$ with $\Delta_k \leftarrow \sum_{j \in \mathcal{S}, j < k} F_{\text{msk}, w_{k,j}}(\text{ssid}) - \sum_{j \in \mathcal{S}, j > k} F_{\text{msk}, w_{k,j}}(\text{ssid})$ are computed as sums of $|\mathcal{S}_{\text{hon}}| - 1$ random values, where $\mathcal{S}_{\text{hon}}$ is the subset of signing set $\mathcal{S}$ that consists of all honest signing parties. By definition of OMUF-3 and by the abort condition introduced in Game 9, we know that the adversary will make at most $|\mathcal{S}_{\text{hon}}| - 1$ third-round signing oracle queries for $\bar{m} = \bar{m}^*$ and $\tau = \tau^*$. Therefore, each of the values $\tilde{\mathbf{z}}_{0,k}^*, \tilde{d}_{1,k}^*$ and $\tilde{r}_{1,k}^*$ given out to the adversary is distributed randomly, unless $\mathcal{A}$ makes two queries for the same ssid along two different signing sessions. In this case, the masks Δ_k would be the same for both signing sessions and thus no longer random. However, as $\text{ssid} = (\text{ctx}, \mathcal{S}, (c_{1,i}^*)_{i \in \mathcal{S}})$ and as the values $c_{1,i}^*$ are sampled uniformly at random for $i \in \mathcal{S}_{\text{hon}}$ by the game in each new signing session, ssid has high min-entropy. That is, the probability of two values ssid being the same in two different signing sessions is bounded by Q_S^2/p . Thus, we have

$$|\varepsilon_{12} - \varepsilon_{11}| = \frac{Q_S^2}{p}.$$

Game 13 (Send random $\mathbf{T}_{\$,k}^*$ and $C_{S,k}^*$ if $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$ was extracted from C by the game). For the case where $\tau = \tau^*$ and $m = \bar{m}^*$ was extracted from C by the game, Game 13 does the following changes to $\mathcal{O}^{\text{Sign}_1}$ when called for signing party k : It samples $C_{S,k}^* \xleftarrow{\$} \mathbb{G}$ and $\mathbf{T}_k^*, \mathbf{T}_{\$,k}^* \xleftarrow{\$} \mathbb{G}^2$. Otherwise, $\mathcal{O}^{\text{Sign}_1}$ proceeds exactly as before. Note that the value $\mathbf{T}_k^*$ is never given out to $\mathcal{A}$, but used internally as part of statement $\mathbf{x}_{0,k}^t$ during the simulation of Σ_0 .

Indeed, this does not change the distribution of $\mathbf{T}_k^*, \mathbf{T}_{\$,k}^*$ and $C_{S,k}^*$, as in Game 12 the following holds for calls to $\mathcal{O}^{\text{Sign}_1}$ for signer k in the case that $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$: If within the same session, $\mathcal{O}^{\text{Sign}_3}$ is never called for signer k , then the blinded openings $(\tilde{d}_{1,k}^*, \tilde{r}_{1,k}^*)$ to $C_{S,k}^*$ as well as $\tilde{\mathbf{z}}_{0,k}^*$, the blinded response of Σ -protocol Σ_0 , never get generated and sent to the user at all. If on the other hand, $\mathcal{O}^{\text{Sign}_3}$ does get called for signer k within the same session, then due to the changes introduced in Game 12, the blinded opening $(\tilde{d}_{1,k}^*, \tilde{r}_{1,k}^*)$ and the blinded Σ_0 response $\tilde{\mathbf{z}}_{0,k}^*$ get sampled uniformly at random. This means that in both cases, $\mathcal{A}$ does not receive a valid opening to the commitment $C_{S,k}^*$, which is statistically hiding. Thus, $C_{S,k}^*$ is uniformly random and thereby independent from all other values output by the game. This means also $d_k^* \xleftarrow{\$} \mathbb{Z}_p$ is independent from $C_{S,k}^*$. As d_k^* is never revealed to $\mathcal{A}$, also $\mathbf{T}_{\$,k}^* := \mathbf{T}_k^* + d_k^*$ is uniformly random and independent from all values sent to $\mathcal{A}$. As $\mathcal{A}$ does not receive a valid response $\mathbf{z}_{0,k}^*$ for Σ_0 , no information on $\mathbf{T}_k^*$ which is part of $\mathbf{x}_{0,k}^t$ is leaked to $\mathcal{A}$. $\mathbf{T}_k^*$ is therefore independent from all values sent to $\mathcal{A}$ and may be chosen uniformly at random.

Thus, the view of $\mathcal{A}$ stays the same as in the previous game and we have

$$\varepsilon_{13} = \varepsilon_{12}.$$

Game 14.1 (Compute $\Sigma_{0,k}$ honestly for $\tau = \tau^*$). For $\tau = \tau^*$, Game 14.1 no longer uses the simulator Sim_0 to generate the transcripts for Σ_0 , but uses its knowledge of $\mathbf{w}_0 = (s^*, \text{sk})$ to compute the transcript honestly instead. In more detail, this means that, in the signing sessions with $\tau = \tau^*$, Game 14.1 still samples $c_0^* \xleftarrow{\$} \mathbb{Z}_p$ when $\mathcal{O}^{\text{Sign}_1}$ gets queried for the first time. But now, when $\mathcal{O}^{\text{Sign}_1}$ gets called for signer $k \in \mathcal{S}$, the game runs $(\mathbf{A}_{0,k}^*, \text{st}_{0,k}) \leftarrow \text{Init}_0(\mathbf{x}_{0,k}^t, \mathbf{w}_{0,k})$ instead of $(\mathbf{A}_{0,k}^*, \mathbf{z}_{0,k}^*) \leftarrow \text{Sim}_0(\mathbf{x}_{0,k}^t, c_0^*)$. When $\mathcal{O}^{\text{Sign}_3}$ gets called for signer $k \in \mathcal{S}$, it now computes $\mathbf{z}_{0,k}^* \leftarrow \text{Resp}_0(\text{st}_{0,k}, c_0^*)$.

Note that c_0^* still gets sampled in the same way as in Game 13. Also, by perfect HVZK of Σ_0 , the distribution of the transcripts $(\mathbf{A}_{0,k}^*, c_0^*, \mathbf{z}_{0,k}^*)$ does not change in comparison to the previous game. Thus the view of $\mathcal{A}$ stays the same and we have

$$\varepsilon_{14.1} = \varepsilon_{13}.$$

Game 14.2 (Simulate $\Sigma_{1,k}$ for $\tau = \tau^*$). For $\tau = \tau^*$, Game 14.2 no longer computes the transcripts for Σ_1 honestly, but uses simulator Sim_1 to generate it instead. More precisely, in the signing sessions with $\tau = \tau^*$, Game 14.2 no longer samples $c_0^* \xleftarrow{\$} \mathbb{Z}_p$, when $\mathcal{O}^{\text{Sign}_1}$ is queried for the first time. Instead, in each query to $\mathcal{O}^{\text{Sign}_1}$ for a signer $k \in \mathcal{S}$, it samples $c_{1,k}^* \xleftarrow{\$} \mathbb{Z}_p$, computes $\text{cm}_k^c = \text{H}_{\text{com}}(k, c_{1,k}^*)$ and runs $(A_{1,k}^*, z_{1,k}^*) \leftarrow \text{Sim}_1(\mathbb{x}_1, c_{1,k}^*)$ instead of computing $(A_{1,k}^*, \text{st}_{1,k}) \leftarrow \text{Init}_1(\mathbb{x}_1, w_1)$. In calls to $\mathcal{O}^{\text{Sign}_3}$ for $\tau = \tau^*$ and $k \in \mathcal{S}$, it now computes $c_1^* = \sum_{i \in \mathcal{S}} c_{1,i}^*$ as well as $c_0^* := c^* - c_1^*$ and gives out the value $z_{1,k}^*$ generated in $\mathcal{O}^{\text{Sign}_1}$ instead of running $z_{1,k}^* \leftarrow \text{Resp}_1(\text{st}_{1,k}, c_{1,k}^*)$.

We observe that, by construction, the challenges (c_0^*, c_1^*) as well as the partial challenges $c_{1,k}^*$ are still distributed in the same way as in the previous games. Also, by perfect HVZK the distribution of the transcript $(A_{1,k}^*, c_{1,k}^*, z_{1,k}^*)$ does not change either. Thus the view of $\mathcal{A}$ stays the same and we have

$$\varepsilon_{14.2} = \varepsilon_{14.1}.$$

Game 14.3 (Abort if forgeries not in $\mathcal{L}_{\text{bb}}$). We obtain Game 14.3 by inserting an additional abort condition into Game 14.2. The game now also aborts if one of $\mathcal{A}$'s forgeries is not part of language $\mathcal{L}_{\text{bb}}$, i.e., if there is at least one forged signature $\sigma_j^* = (S_1^*, S_2^*, \pi, (d_1, r_1), (d_2, r_2))$ such that for message m_j^* , $X_j^* = \overline{m}_j^* U + H$ and $\overline{m}_j^* := \text{H}_M(m_j^*)$ it holds that $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \notin \mathcal{L}_{\text{bb}}$.

In more detail, during setup, the game samples $h \xleftarrow{\$} \mathbb{Z}_p$ and sets $H = hG$. Otherwise, the setup proceeds as in Game 14.2 and the oracles are simulated in the same way as in Game 14.2. Once the game receives the k forgeries from $\mathcal{A}$, it checks whether for all $j \in [k]$, the following condition is true:

$$S_{1,j}^* = uV + (\overline{m}_j^* \cdot u)S_{2,j}^* + hS_{2,j}^*. \quad (8)$$

By construction, the game knows both h and u and thus the condition can be evaluated efficiently. From Game 14.3 of the OMUF proof for scheme BS we know that checking this condition is indeed equivalent to checking whether it holds for all σ_j^* and their corresponding $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U)$ tuples with $j \in [k]$ that $(G, V, X_j^*, S_{1,j}^*, S_{2,j}^*, U) \in \mathcal{L}_{\text{bb}}$.

By an analogous argument to the argument made in the OMUF proof of scheme BS, we get that

$$|\varepsilon_{14.3} - \varepsilon_{14.2}| \leq k \cdot \left(\sqrt{Q_{\text{H}_\Sigma}(\text{AdvDL}_B^{\text{GrGen}}(\lambda) + \text{AdvBnd}_B^{\text{Com}}(\lambda))} + \frac{Q_{\text{H}_\Sigma}}{p} \right).$$

Game 14.4 and Game 14.5. Game 14.4 reverts the changes made in Game 14.2 and again computes the values $c_0^*, c_1^*, c_{1,k}^*, \text{cm}_k^c, A_{1,k}^*$ and $z_{1,k}^*$ in signing sessions with $\tau^* = \tau$ in the same way they are computed in sessions with $\tau^* \neq \tau$. However, it keeps the additional abort condition introduced in Game 14.3. Analogously, Game 14.5 reverts the changes made in Game 14.1 and again computes the values $\mathbf{A}_{0,k}^*$ and $\mathbf{z}_{0,k}^*$ in sessions with $\tau = \tau^*$ in the same way they are computed also in the other sessions. Also Game 14.5 keeps the abort condition introduced in Game 14.3. By the same argument used in the description of Games 5 and 6, we have that

$$\varepsilon_{14.3} = \varepsilon_{14.4} = \varepsilon_{14.5}.$$

We will now proceed with constructing an adversary $\mathcal{A}_{\text{BB}}$ against Game T-BB (cf. Lemma 4) that runs adversary $\mathcal{A}$ against Game 14.5 as a subroutine.

Reduction to CDH. $\mathcal{A}_{\text{BB}}$ starts off by running $\text{par} \leftarrow \text{TBS.Setup}(1^\lambda)$, choosing some $n, t \leftarrow \mathbb{N}^+$ with $t < n$ and $\mathcal{S} \subseteq [n]$ with $|\mathcal{S}| \geq t + 1$. It then runs $\mathcal{C} \leftarrow \mathcal{A}(\text{par}, n, t + 1)$ and aborts if $|\mathcal{C}| > t$. Afterwards, $\mathcal{A}_{\text{BB}}$ samples $\overline{m}^* \xleftarrow{\$} \mathbb{Z}_p$, sends $(\overline{m}^*, n, t + 1, \mathcal{S}, \mathcal{C})$ to the challenger of Game T-BB, receives parameter tuple $(G, U, V, H, \{U_i\}_{i \in \mathcal{S}}, \{u_i\}_{i \in \mathcal{C}})$ back from the T-BB challenger and gets query access to signing oracle $\mathcal{O}$. $\mathcal{A}_{\text{BB}}$ then sets $\text{H}_V(\tau^*) = V$ as well as $\text{vk} := (G, U, H)$ and runs $\mathcal{A}^{\mathcal{O}^{\text{Sign}}}(\text{vk}, \{u_i\}_{i \in \mathcal{C}})$. Oracle queries from $\mathcal{A}$ are answered by $\mathcal{A}_{\text{BB}}$ in the following way:

- Queries to $\text{H}_M, \text{H}_{\text{DLog}}, \text{H}_\Sigma, \text{H}_{\text{Ped}}, \text{H}_V, \mathcal{O}^{\text{Sign}_2}$ and $\mathcal{O}^{\text{Sign}_3}$ get answered in exactly the same way as in Game 14.5. We recall that $\mathcal{O}^{\text{Sign}_3}$ sends out random values for $\tilde{d}_{1,k}^*, \tilde{r}_{1,k}^*$ and $\tilde{\mathbf{z}}_{0,k}^*$ if it is queried for $\tau = \tau^*$ and $\overline{m} = \overline{m}^*$ where $\overline{m}^*$ had previously been extracted by the game. We also recall that H_M outputs $\overline{m}^*$ on the q_{m^*} -th query.

- Queries to $\mathcal{O}^{\text{Sign}_1}$ with $\tau \neq \tau^*$ or $\tau = \tau^*$ and $\bar{m} = \bar{m}^*$ also get answered exactly as in Game 14.5.
- For queries to $\mathcal{O}^{\text{Sign}_1}$ for signer k with $\tau = \tau^*$ and $\bar{m} \neq \bar{m}^*$, just like Game 14.5, $\mathcal{A}_{\text{BB}}$ first performs the checks that NIPS_{Ped} successfully verifies for $\mathbf{x}_{\text{Ped}}$ and π_{Ped} and that the witness $\mathbf{w}_{\text{Ped}}$ extracted by Ext_{Ped} can successfully be parsed as $\mathbf{w}_{\text{Ped}} = (\bar{m}, t)$ such that $C = \bar{m}U + tG$. Then, $\mathcal{A}_{\text{BB}}$ queries $(S_{1,k}, S_{2,k}) \leftarrow \mathcal{O}(k, \bar{m})$, sets $\mathbf{T}_k^* = (S_{1,k} + tS_{2,k}, S_{2,k}, \lambda_k U_k)$ and from there on proceeds exactly like Game 14.5.

Upon receiving the k forgeries (m_j^*, σ_j^*) with common message τ^* from $\mathcal{A}$, $\mathcal{A}_{\text{BB}}$ verifies, whether for any of the messages m_j^* it holds that $\mathbf{H}_{\text{M}}(m_j^*) = \bar{m}^*$. If it does find such a message, it parses $(S_1^*, S_2^*, \pi^*) \leftarrow \sigma$ and returns (S_1^*, S_2^*) to the BB challenger.

We first note that the running times of $\mathcal{A}$ and $\mathcal{A}_{\text{BB}}$ are roughly the same. Also, the distribution of vk stays the same as in Game 14.5. All oracle queries are answered in the exact same way as in Game 14.5 except for calls to $\mathcal{O}^{\text{Sign}_1}$ with $\tau = \tau^*$ and $\bar{m} \neq \bar{m}^*$. In the latter case, upon calling $\mathcal{O}(k, \bar{m})$, $\mathcal{A}_{\text{BB}}$ receives a tuple $(S_{1,k}, S_{2,k})$ where $S_{1,k} = \lambda_k u_k V + sX_{\bar{m}}$, $S_{2,k} = s_k G$, $s_k \xleftarrow{\$} \mathbb{Z}_p$ and $U_k = u_k G$. For the value $\mathbf{T}_k^*$ computed by $\mathcal{A}_{\text{BB}}$, it thus holds that:

$$\begin{aligned} T_{1,k}^* &= S_{1,k} + tS_{2,k} = \lambda_k u_k V + s_k X_{\bar{m}} + (t \cdot s_k)G \\ &= \lambda_k u_k V + s_k(\bar{m}U + H + tG) = \lambda_k u_k V + s_k(X_C + H). \end{aligned}$$

Thus, the distribution of the values $\mathbf{T}_k^*$ computed by $\mathcal{A}_{\text{BB}}$ and by the challenger in Game 14.5 is exactly the same. Therefore, the view of $\mathcal{A}$ is the same in the simulation of Game 14.5 by $\mathcal{A}_{\text{BB}}$ as in the original Game 14.5.

We know that, with probability at least $\varepsilon_{14.5}$, $\mathcal{A}$ outputs a message m_j^* for which $(G, V, X_{\bar{m}^*}, S_1^*, S_2^*, U) \in \mathcal{L}_{\text{bb}}$ with $\mathbf{H}_{\text{M}}(m_j^*) = \bar{m}^*$ and $X_{\bar{m}^*} := \bar{m}^*U + H$. Thus, with probability at least $\varepsilon_{14.5}$, $\mathcal{A}_{\text{BB}}$ wins Game BB. We have

$$\varepsilon_{14.5} \leq \varepsilon_{\mathcal{A}_{\text{BB}}}^{BB} \leq \text{AdvCDH}_{\mathcal{A}_{\text{CDH}}}^{\text{GrGen}}(\lambda).$$

Putting all equalities and inequalities obtained through the game hops together, we obtain Theorem 5.

6.3 Blindness of TBS

We give a formal proof of Theorem 6.

Proof. Consider an adversary $\mathcal{A}_{\text{TBS}}$ playing the threshold blindness game for scheme TBS. For simplicity, we assume that $\mathcal{A}_{\text{TBS}}$ starts a single signing session. The argument is straightforward to generalize to a polynomial number of sessions. We construct an adversary $\mathcal{A}_{\text{BS}}$ attacking blindness of scheme BS that runs $\mathcal{A}_{\text{TBS}}$ as a subroutine and that wins the blindness game for scheme BS whenever $\mathcal{A}_{\text{TBS}}$ wins the threshold blindness game for scheme TBS. By a standard hybrid argument, Theorem 6 then directly follows.

$\mathcal{A}_{\text{BS}}$ has access to the oracles $\mathcal{O}^{\text{User}_0}, \mathcal{O}^{\text{User}_1}, \mathcal{O}^{\text{User}_2}$. It receives par from the challenger of the blindness experiment for scheme BS, selects an additional hash function $\mathbf{H}_{\text{com}} : \{0, 1\}^* \rightarrow \mathbb{G}$, invokes $b' \leftarrow \mathcal{A}_{\text{TBS}}^{\mathcal{O}^{\text{User}_0}, \mathcal{O}^{\text{User}_1}, \mathcal{O}^{\text{User}_2}, \mathcal{O}^{\text{Sign}_3}}()$ and returns b' . Queries to the signing oracles are answered in the following way:

- $\mathcal{O}^{\text{User}_0}(\text{sid}, \text{vk}_{\text{sid}}, \tau_{\text{sid}}, m_{0,\text{sid}}, m_{1,\text{sid}}, S_0, S_1)$: $\mathcal{A}_{\text{BS}}$ queries $(C_0, \pi_{\text{Ped},0}), (C_1, \pi_{\text{Ped},1}) \leftarrow \mathcal{O}^{\text{User}_0}(\text{sid}, \text{vk}_{\text{sid}}, \tau_{\text{sid}}, m_{0,\text{sid}}, m_{1,\text{sid}})$. It then returns $((C_0, \pi_{\text{Ped},0}, S_0), (C_1, \pi_{\text{Ped},1}, S_1))$.
- $\mathcal{O}^{\text{User}_1}(\text{sid}, (\{\mathbf{T}_{\$,i}^*, \mathbf{A}_{0,i}, A_{1,i}, C_{S,i}^*\}_{i \in S_0}), (\{\mathbf{T}_{\$,i}^*, \mathbf{A}_{0,i}, A_{1,i}, C_{S,i}^*\}_{i \in S_1}))$: $\mathcal{A}_{\text{BS}}$ queries $c_0^*, c_1^* \leftarrow \mathcal{O}^{\text{User}_1}(\text{sid}, (\sum_{i \in S_0} \mathbf{T}_{\$,i}^*, \sum_{i \in S_0} \mathbf{A}_{0,i}, \sum_{i \in S_0} A_{1,i}, \sum_{i \in S_0} C_{S,i}^*), (\sum_{i \in S_1} \mathbf{T}_{\$,i}^*, \sum_{i \in S_1} \mathbf{A}_{0,i}, \sum_{i \in S_1} A_{1,i}, \sum_{i \in S_1} C_{S,i}^*))$ and returns (c_0^*, c_1^*) .
- $\mathcal{O}^{\text{User}_2}(\text{sid}, (\{c_{1,i}^*\}_{i \in S_0}), (\{c_{1,i}^*\}_{i \in S_1}))$: $\mathcal{A}_{\text{BS}}$ returns $(\{c_{1,i}^*\}_{i \in S_0}), (\{c_{1,i}^*\}_{i \in S_1})$.
- $\mathcal{O}^{\text{Sign}_3}(\text{sid}, (c_0^*, \{\tilde{\mathbf{z}}_{0,i}^*, z_{1,i}^*, \tilde{d}_{1,i}^*, \tilde{r}_{1,i}^*\}_{i \in S_0}), (c_0^*, \{\tilde{\mathbf{z}}_{0,i}^*, z_{1,i}^*, \tilde{d}_{1,i}^*, \tilde{r}_{1,i}^*\}_{i \in S_1}))$: $\mathcal{A}_{\text{BS}}$ computes $\mathbf{z}_{0,S_0}^* = \sum_{i \in S_0} \tilde{\mathbf{z}}_{0,i}^*$, $z_{1,S_0}^* = \sum_{i \in S_0} z_{1,i}^*$, $\tilde{d}_{1,S_0}^* = \sum_{i \in S_0} \tilde{d}_{1,i}^*$ and $\tilde{r}_{1,S_0}^* = \sum_{i \in S_0} \tilde{r}_{1,i}^*$ as well as $\mathbf{z}_{0,S_1}^* = \sum_{i \in S_1} \tilde{\mathbf{z}}_{0,i}^*$, $z_{1,S_1}^* = \sum_{i \in S_1} z_{1,i}^*$, $\tilde{d}_{1,S_1}^* = \sum_{i \in S_1} \tilde{d}_{1,i}^*$ and $\tilde{r}_{1,S_1}^* = \sum_{i \in S_1} \tilde{r}_{1,i}^*$. It then queries $\sigma_0, \sigma_1 \leftarrow \mathcal{O}^{\text{User}_2}(\text{sid}, (c_0^*, \mathbf{z}_{0,S_0}^*, z_{1,S_0}^*, \tilde{d}_{1,S_0}^*, \tilde{r}_{1,S_0}^*), (c_0^*, \mathbf{z}_{0,S_1}^*, z_{1,S_1}^*, \tilde{d}_{1,S_1}^*, \tilde{r}_{1,S_1}^*))$. It returns (σ_0, σ_1) .

$\mathcal{A}_{\text{BS}}$ imitates the threshold blindness experiment perfectly, as in oracles $\mathcal{O}^{\text{USign}_0}$, $\mathcal{O}^{\text{USign}_1}$ and $\mathcal{O}^{\text{USign}_3}$ it performs the exact same computations before calling oracles $\mathcal{O}^{\text{User}_0}$, $\mathcal{O}^{\text{User}_1}$ and $\mathcal{O}^{\text{User}_2}$, that TBS.USign_0 , TBS.USign_1 and TBS.USign_3 do before calling BS.USign_0 , BS.USign_1 and BS.USign_3 . $\mathcal{O}^{\text{USign}_2}$ reflects the sets of messages $\{c_{1,i}^*\}_{i \in \mathcal{S}_0}$ and $\{c_{1,i}^*\}_{i \in \mathcal{S}_1}$ in exactly the same way that USign_2 reflects the set of messages $\{c_{1,i}^*\}_{i \in \mathcal{S}}$. Also, $\mathcal{A}_{\text{BS}}$ roughly has the same runtime as $\mathcal{A}_{\text{TBS}}$ and wins whenever $\mathcal{A}_{\text{TBS}}$ wins.

7 pTBS: Pairing-based Threshold Blind Signature from CDH

We now describe our pairing-based threshold blind signature pTBS. Our scheme pTBS is based on [27], which itself is based on BB, similar to BS and TBS. We do not consider partial blindness here, as compared to our previous schemes, it would require a structural change on BB. At the cost of an additional group element in the signature, we believe that achieving partial blindness is possible with the modifications from [27, Section 7.2].

Preparations. As in Sections 3 and 5, we employ a random oracle $H_M : \{0, 1\}^* \leftarrow \mathbb{Z}_p$ to preprocess the message. Also, we employ a proof system $\text{NIPS}_{\text{Ped}} = (\text{NIPS}_{\text{Ped}}.\text{Prove}^{\text{HPed}}, \text{NIPS}_{\text{Ped}}.\text{Ver}^{\text{HPed}})$ with random oracle H_{Ped} for relation R_{Ped} , as defined in Section 3. Analogously to Section 5, IssueShares is a PPT algorithm computing n Shamir's shares $\{u_i\}_{i \in [n]}$ of value u on input $(n, t + 1)$, of which $t + 1$ are necessary to reconstruct value u .

Note that F_{msk} now denotes a PRF with key space $\mathcal{K}_{\text{msk}}$ that maps from $\{0, 1\}^*$ to $\mathbb{G}_1$, again employed to mask the individual signer messages. In addition, we employ a PRF F_{rnd} with key space $\mathcal{K}_{\text{rnd}}$ that maps from $\{0, 1\}^*$ to $\mathbb{Z}_p$ to derandomize each signer.

7.1 Construction

Our pairing-based threshold blind signature scheme pTBS is given in Fig. 9. The issued signatures $\sigma = (\sigma_0, \sigma_1)$ in pTBS follow the structure

$$\sigma_0 = \text{bb-sk} + r \cdot (m \cdot U_1 + H_1), \quad \sigma_1 = rG_1,$$

where $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2) \leftarrow \text{PGen}(1^\lambda)$ are pairing-friendly groups, and $U_1 = \alpha G_1$ and $H_1 = \gamma G_1$ and $\text{bb-sk} = \alpha\beta G_1$. Notably, the structure of σ can be verified via the pairing e and additional elements $U_2 = \alpha G_2$, $H_2 = \gamma G_2$ and $V = \alpha\beta \cdot e(G_1, G_2)$ specified in the public key.

Overview. As in our scheme TBS, the signers receive linear shares bb-sk_i of the secret key bb-sk during key generation.

The signing protocol is initiated by the user sending a Pedersen commitment C to the message, along with a proof π_{Ped} of opening knowledge, to the signers. Each signer k then uses their share bb-sk_k of sk to compute a partial signature σ_k^* on C . As in TBS, signer k masks their partial signature σ_k^* with a zero share Δ_k . Importantly, the partial signature is derived deterministically via F_{rnd} .

Finally, the user obtains a valid signature to its message m by linearly combining all $t + 1$ partial signatures and removing the remaining randomness introduced by C from the resulting signature σ .

7.2 Security Analysis

We show correctness, one-more unforgeability and blindness. We refer to Section 8 for formal proofs.

Correctness. Again, correctness is immediate.

Theorem 7 (Correctness). *pTBS is correct with error γ_{err} , where γ_{err} is the correctness error of NIPS_{Ped} .*

A formal proof of Theorem 7 is given in Section 8.1.

One-more Unforgeability. The argument is based on puncturing BB signatures. Roughly, we let the simulation guess an H_M query such that its input m^* appears in the forgeries, yet is never extracted from π_{Ped} in *finished* sessions. Also, we show that partial BB signatures can be simulated even with a verification key vk punctured for m^* . Together, this breaks CDH.

An important but subtle detail of the proof is that as in TBS, partial signatures may leak information. In particular, we cannot sign for extracted m^* with punctured setup, but incomplete sessions

Setup (1^λ)	USign₀ (vk, aux := $\perp$, m, $\mathcal{S}$, τ := $\perp$)
1 : $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2) \leftarrow \text{PGen}(1^\lambda)$ 2 : par := $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2)$ 3 : return par	1 : $\overline{m} := H_M(m)$ 2 : $t \xleftarrow{\$} \mathbb{Z}_p, C := \overline{m}U_1 + tG_1$ 3 : $\mathbb{X}_{\text{Ped}} := (C, U_1, G_1), \mathbb{W}_{\text{Ped}} := (\overline{m}, t)$ 4 : $\pi_{\text{Ped}} \leftarrow \text{NIPS}_{\text{Ped}}.\text{Prove}^{\text{HPed}}(\mathbb{X}_{\text{Ped}}, \mathbb{W}_{\text{Ped}})$ 5 : return ($\mathcal{S}, C, \pi_{\text{Ped}}$)
KeyGen ($n, t + 1$)	ISign₁ ($k, \text{sk}_k, \text{aux} := \perp, \text{ctx}, \tau := \perp, (\mathcal{S}, C, \pi_{\text{Ped}})$)
1 : $\alpha, \beta, \gamma \xleftarrow{\$} \mathbb{Z}_p$ 2 : $U_1 := \alpha G_1, U_2 := \alpha G_2, H_1 := \gamma G_1$ 3 : $H_2 := \gamma G_2, V := \alpha\beta \cdot e(G_1, G_2)$ 4 : $\{\beta_i\}_{i \in [n]} \leftarrow \text{IssueShares}(\beta, n, t + 1)$ 5 : $\{w_{i,j}\}_{i,j \in [n], i < j} \xleftarrow{\$} \mathcal{K}_{\text{msk}}^{n(n-1)/2}$ 6 : $\forall i,j \in [n], i > j, w_{i,j} := w_{j,i}$ 7 : $\{w_i\}_{i \in [n]} \xleftarrow{\$} \mathcal{K}_{\text{rnd}}^n$ 8 : $\text{bb-sk}_i := \beta_i \alpha G_1$ 9 : $\text{sk}_i := (\text{bb-sk}_i, \{w_{i,j}\}_{j \in [n] \setminus \{i\}}, w_i)$ 10 : $\text{vk} := (U_1, U_2, H_1, H_2, V)$ 11 : return ($\{\text{sk}_i\}_{i \in [n]}, \text{vk}$)	1 : return $\perp$ if $k \notin \mathcal{S}$ 2 : req $\text{NIPS}_{\text{Ped}}.\text{Ver}^{\text{HPed}}(\mathbb{X}_{\text{Ped}}, \pi_{\text{Ped}}) = 1$ 3 : $\text{ssid} := (\text{ctx}, \mathcal{S}, C, \pi_{\text{Ped}})$ 4 : $r_k \xleftarrow{\$} F_{\text{rnd}, w_k}(\text{ssid}), \sigma_{k,1}^* := r_k G_1$ 5 : $\Delta_k \leftarrow \sum_{j \in \mathcal{S}, j < k} F_{\text{msk}, w_{k,j}}(\text{ssid})$ $\quad \quad \quad - \sum_{j \in \mathcal{S}, j > k} F_{\text{msk}, w_{k,j}}(\text{ssid})$ 6 : $\sigma_{k,0}^* := \lambda_k \cdot \text{bb-sk}_k + r_k(H_1 + C) + \Delta_k$ 7 : $\sigma_k^* := (\sigma_{k,0}^*, \sigma_{k,1}^*)$ 8 : return ($\text{ssid}, \mathcal{S}, \sigma_k^*$)
Verify (vk, m, σ)	USign₁ ($\{\sigma_i^*\}_{i \in \mathcal{S}}$)
1 : $\overline{m} := H_M(m)$ 2 : req $e(\sigma_0, G_2) = V + e(\sigma_1, \overline{m}U_2 + H_2)$ 3 : return 1	1 : $\sigma^* = (\sigma_0^*, \sigma_1^*) := \left(\sum_{i \in \mathcal{S}} \sigma_{i,0}^*, \sum_{i \in \mathcal{S}} \sigma_{i,1}^* \right)$ 2 : req $e(\sigma_0^*, G_2) = V + e(\sigma_1^*, \overline{m}U_2 + tG_2 + H_2)$ 3 : $r' \xleftarrow{\$} \mathbb{Z}_p$ 4 : $\sigma := (\sigma_0^* - t \cdot \sigma_1^* + r'(\overline{m}U_1 + H_1), \sigma_1^* + r'G_1)$ 5 : return σ

Fig. 9: Our pairing-based TBS scheme pTBS based on [27]. Algorithm **ISign₁** is executed by the signers, algorithms **USign₀** and **USign₁** are executed by the user. We leave the user and signer states implicit for readability. Notable differences to the blind signature from [27] due to translation into the threshold setting are highlighted in grey.

may leak information on such partial signatures σ_i^* on m^* . This is solved by masking partial signatures σ_i^* with zero shares Δ_i . It is important to note that Δ_i cannot be derived from high min-entropy input here, as signers must agree on the input without interaction. Therefore, the zero share Δ_i may appear in more than one session, in which case it does not mask the partial signatures as desired. This is where deterministic signing is important: if Δ_i appears more than once, the derived partial signature is identical, and therefore such sessions leak no additional information.

Theorem 8 (OMUF-3). *Let p be the order of group $\mathbb{Z}_p$. For any PPT adversary $\mathcal{A}$ making at most Q_M random oracle queries, there exist reductions $\mathcal{A}_{KS}$, $\mathcal{A}_{DL}$ and $\mathcal{A}_{CDH}$ having a runtime similar to $\mathcal{A}$ such that*

$$\text{AdvTOMUF}_{\mathcal{A}}^{\text{pTBS}}(\lambda) \leq \text{AdvKS}_{\mathcal{A}_{KS}}^{\text{NIPSPed}, \bar{R}_{\text{Ped}}}(\lambda) + \text{AdvDL}_{\mathcal{A}_{DL}}^{\text{PGen}}(\lambda) + \frac{Q_M^2}{p} + Q_M \cdot \text{AdvCDH}_{\mathcal{A}_{CDH}}^{\text{PGen}}(\lambda).$$

A formal proof of Theorem 8 is given in Section 8.2.

Blindness. Blindness follows as in [27].

Theorem 9 (Blindness). *For any PPT adversary $\mathcal{A}$ causing at most Q random oracle queries there exists a reduction $\mathcal{A}_{WI}$ such that*

$$\text{AdvTPBlind}_{\mathcal{A}}^{\text{pTBS}}(\lambda) \leq 2 \cdot \text{AdvWI}_{\mathcal{A}_{WI}}^{\text{NIPSPed}, \bar{R}_{\text{Ped}}}(Q, \lambda).$$

A formal proof of Theorem 9 is given in Section 8.3.

8 Formal Security Proofs for pTBS

We present formal security proofs for our scheme in ??.

8.1 Correctness of pTBS

We give a formal proof of Theorem 7.

Proof. By definition, we know that NIPSPed may fail to output a proof π_{Ped} that successfully verifies with probability γ_{err} . In the following we will show, that, if NIPSPed does not fail to do so, pTBS always generates a valid signature.

Let $(\{\text{sk}_i\}_{i \in [n]}, \text{vk}) \leftarrow \text{pTBS.KeyGen}(n, k)$. Then, $\text{USign}_0(\text{vk}, m, \mathcal{S})$ outputs $(\mathcal{S}, C, \pi_{\text{Ped}})$, where $C := \bar{m}U_1 + tG_1$ and $t \xleftarrow{\$} \mathbb{Z}_p$.

Assuming π_{Ped} successfully verifies, $\text{ISign}(i, \text{sk}_i, \text{ctx}, (\mathcal{S}, C, \pi_{\text{Ped}}))$ returns $\sigma_i^* = (\sigma_{i,0}^*, \sigma_{i,1}^*)$, where

$$\sigma_{i,0}^* = \lambda_i \text{bb-sk}_i + r_i(H_1 + C) + \sum_{j \in \mathcal{S}, j < i} F_{\text{msk}, w_{i,j}}(\text{ctx}, \mathcal{S}, C, \pi_{\text{Ped}}) - \sum_{j \in \mathcal{S}, j > i} F_{\text{msk}, w_{i,j}}(\text{ctx}, \mathcal{S}, C, \pi_{\text{Ped}})$$

and $\sigma_{i,1}^* = r_i G_1$.

$\text{USign}_1(\{\sigma_i^*\}_{i \in \mathcal{S}})$ then computes

$$\begin{aligned} \sigma^* &= (\sigma_0^*, \sigma_1^*) = \left(\sum_{i \in \mathcal{S}} \sigma_{i,0}^*, \sum_{i \in \mathcal{S}} \sigma_{i,1}^* \right) \\ &= \left(\sum_{i \in \mathcal{S}} (\lambda_i \text{bb-sk}_i + r_i(H_1 + C) + \sum_{j \in \mathcal{S}, j < i} F_{\text{msk}, w_{i,j}}(\text{ctx}, \mathcal{S}, C, \pi_{\text{Ped}}) - \sum_{j \in \mathcal{S}, j > i} F_{\text{msk}, w_{i,j}}(\text{ctx}, \mathcal{S}, C, \pi_{\text{Ped}})), \sum_{i \in \mathcal{S}} r_i G_1 \right) \\ &= \left(\text{bb-sk} + r^*(H_1 + C), r^* G_1 \right), \end{aligned}$$

where $r^* = \sum_{i \in \mathcal{S}} r_i$.

This σ^* will pass the well-formedness check as

$$\begin{aligned}
e(\sigma_0^*, G_2) &= e(\text{bb-sk} + r^*(H_1 + C), G_2) \\
&= e(\text{bb-sk}, G_2) + e(r^*(H_1 + C), G_2) \\
&= \alpha\beta \cdot e(G_1, G_2) + r^* \cdot e(H_1 + \overline{m}U_1 + tG_1, G_2) \\
&= V + r^* e(G_1, H_2 + \overline{m}U_2 + tG_2) \\
&= V + e(\sigma_1^*, H_2 + \overline{m}U_2 + tG_2)
\end{aligned}$$

The σ returned by USign_1 successfully verifies as for

$$\begin{aligned}
(\sigma_0, \sigma_1) &= (\sigma_0^* - t \cdot \sigma_1^* + r'(\overline{m}U_1 + H_1), \sigma_1^* + r'G_1) \\
&= (\text{bb-sk} + r^*(H_1 + C) - t \cdot \sigma_1^* + r'(\overline{m}U_1 + H_1), r^*G_1 + r'G_1) \\
&= (\text{bb-sk} + (r^* + r')(\overline{m}U_1 + H_1), (r^* + r')G_1) \\
&= (\text{bb-sk} + r(\overline{m}U_1 + H_1), rG_1)
\end{aligned}$$

it holds that

$$\begin{aligned}
e(\sigma_0, G_2) &= e(\text{bb-sk} + r(\overline{m}U_1 + H_1), G_2) \\
&= e(\text{bb-sk}, G_2) + e(r(\overline{m}U_1 + H_1), G_2) \\
&= \alpha\beta \cdot e(G_1, G_2) + r \cdot e(\overline{m}U_1 + H_1, G_2) \\
&= V + r \cdot e(G_1, \overline{m}U_2 + H_2) \\
&= V + e(\sigma_1^*, \overline{m}U_2 + H_2),
\end{aligned}$$

where $r = r^* + r'$.

8.2 OMUF-3 Security of pTBS

We give a formal proof of theorem 8.

Proof. Game 0 (Honest). We start with the honest OMUF-3 experiment for threshold blind signatures as described in section 2. In the beginning, the game sets up random oracle H_M , signing oracle $\mathcal{O}^{\text{ISign}}$ and runs $\text{par} \leftarrow \text{pTBS.Setup}(1^\lambda)$. Afterwards, it sets up the bookkeeping variables $\text{ES}_3, \text{El}_3, \mathcal{Q}_{H_{\text{Ped}}}$ and runs $\mathcal{C} \leftarrow \mathcal{A}(\text{par}, n, t)$, where $\mathcal{C}$ is the set of corrupted signing parties. If $|\mathcal{C}| > t$, it aborts. Then, the game runs $(\{\text{sk}_i\}_{i \in [n]}, \text{vk}) \leftarrow \text{pTBS.KeyGen}(n, t)$, where vk and sk_i are defined as in Fig. 9. Finally, it runs $(\ell, \{m_i^*, \sigma_i^*\}_{i \in [\ell]}) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{ISign}}}(\text{vk}, \{\text{sk}_i\}_{i \in \mathcal{C}})$ and outputs 1 if the winning threshold $\text{allow}_3 < \ell$, if all ℓ messages m_i^* are pairwise distinct and if all message-signature pairs successfully verify. Oracle queries $\mathcal{O}^{\text{ISign}}(1, \text{sid}, i, \text{pm}_0^{\text{U, sid}})$, where i denotes the index of a signing party are answered as indicated in Fig. 2a: The game first performs the bookkeeping of the OMUF-3 game and then runs $\text{Signer}(i, \text{sk}_i, \text{ctx}, \text{pm}_0^{\text{U, sid}})$, where $\text{pm}_0^{\text{U, sid}} = (\mathcal{S}, C, \pi_{\text{Ped}})$. By definition, we have

$$\text{AdvTOMUF}_{\mathcal{A}}^{\text{pTBS}}(\lambda) = \varepsilon_0.$$

Game 1 (Extract $C = (\overline{m}, t)$ from C). In Game 1, a change is made to the signing oracle $\mathcal{O}^{\text{ISign}}$: After the initial check that π_{Ped} successfully verifies (i.e., that $\text{NIPS}_{\text{Ped}}.\text{Ver}^{H_{\text{Ped}}}(\mathbb{x}_{\text{Ped}}, \pi_{\text{Ped}}) = 1$), $\mathcal{O}^{\text{ISign}}$ now uses the extractor Ext_{Ped} and the queries made by $\mathcal{A}$ to H_{Ped} (which are stored in $\mathcal{Q}_{H_{\text{Ped}}}$) to compute a witness w_{Ped} for $\mathbb{x}_{\text{Ped}}$. More precisely, $\mathcal{O}^{\text{ISign}}$ runs $\text{w}_{\text{Ped}} \leftarrow \text{Ext}_{\text{Ped}}(\mathcal{Q}_{H_{\text{Ped}}}, \mathbb{x}_{\text{Ped}}, \pi_{\text{Ped}})$, parses $(\overline{m}, t) := \text{w}_{\text{Ped}}$ and aborts if parsing w_{Ped} fails or $C \neq \overline{m}U + tG$. Apart from that, $\mathcal{O}^{\text{ISign}}$ remains unchanged.

Let us denote the event where Game 1 aborts due to the new abort condition as E_1 . E_1 happens if extraction of a witness w_{Ped} fails or if w_{Ped} is not a valid opening for C . The latter case might arise since NIPS_{Ped} is only relaxed knowledge sound and Ext_{Ped} might compute $\text{w}_{\text{Ped}} = u$ such that $uG = U$ instead of a w_{Ped} such that $(\mathbb{x}_{\text{Ped}}, \text{w}_{\text{Ped}}) \in \text{R}_{\text{Ped}}$. Let us denote the event where E_1 occurs and witness

extraction fails as E_1^W and the event where E_1 occurs and witness extraction does not fail as E_1^{-W} . Thus, $\Pr[E_1] = \Pr[E_1^W] + \Pr[E_1^{-W}]$.

We can bound $\Pr[E_1^W]$ through a straightforward reduction $\mathcal{B}_1$ on the knowledge soundness of NIPS_{Ped} . $\Pr[E_1^{-W}]$ can be bounded by the probability of a reduction $\mathcal{B}_2$ succeeding to compute the discrete logarithm of U in the pairing setting. In more detail, $\mathcal{B}_2$ works as follows: It first selects $n, t \in \mathbb{N}$ with $n \geq t$, runs $\text{par} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G_1, G_2) \leftarrow \text{pTBS.Setup}(1^\lambda)$ and calls $\mathcal{C} \leftarrow \mathcal{A}(\text{par}, n, t)$. From the co-DLog challenger, $\mathcal{B}_2$ then receives challenge tuple (L_1, L_2) , where $L_1 \in \mathbb{G}_1$ and $L_2 \in \mathbb{G}_2$. $\mathcal{B}_2$ now runs $(\{\text{sk}_i\}_{i \in [n]}, \text{vk}) \leftarrow \text{pTBS.KeyGen}(n, t+1)$ with a slight modification: Instead of sampling $\alpha \xleftarrow{\$} \mathbb{Z}_p$ and setting $U_1 = \alpha G_1$, $U_2 = \alpha G_2$, it sets $U_1 = L_1$ and $U_2 = L_2$. The secret keys bb-sk_i get computed as $\text{bb-sk}_i := \beta_i L_1$. Apart from that, pTBS.KeyGen is run as normal. Finally, $\mathcal{B}_2$ calls $\mathcal{A}^{\mathcal{O}^{\text{Signer}}(\cdot)}(\text{vk}, (\text{sk}_i)_{i \in \mathcal{C}})$. Queries to signing oracle $\mathcal{O}^{\text{Signer}}$ for signer k and user output pm_0^U are answered by performing the bookkeeping of the OMUF game and calling $\text{ISign}_1(k, \text{sk}_k, \text{ctx}, \text{pm}_0^U)$. Whenever event E_1^W happens while playing the OMUF game with $\mathcal{A}$, $\mathcal{B}_2$ wins the co-DLog game by sending $w = u$ to the co-DLog challenger. Clearly, the view of $\mathcal{A}$ is the same as in game 0, as we can assume that the challenge tuple (L_1, L_2) is uniformly random and as signing queries are answered exactly as in game 0.

From these reductions, we obtain that

$$|\varepsilon_1 - \varepsilon_0| \leq \text{AdvKS}_{\mathcal{B}_1}^{\text{NIPS}_{\text{Ped}}, \tilde{\text{R}}_{\text{Ped}}}(\lambda) + \text{AdvDL}_{\mathcal{B}_2}^{\text{PGen}}(\lambda).$$

Game 2 (Abort if H_M collision). In order to obtain Game 2, we add a new abort condition to the previous game: Now, our game additionally aborts if any hash collisions $H_M(m_i) = H_M(m_j)$ for $m_i \neq m_j$ arise.

We can upper bound the probability of such a hash collision by a standard birthday bound. Thus, we have

$$|\varepsilon_0 - \varepsilon_1| \leq \frac{Q_M^2}{p}.$$

Game 3 (Guess unsigned $\bar{m}^*$ in forgery). Game 3 guesses the index q_{m^*} of the first query to H_M for which the following two conditions are satisfied:

1. The input $m_{q_{m^*}}$ of the q_{m^*} -th query to H_M is included in $\mathcal{A}$'s forgeries.
2. $\bar{m}^* = H_M(m_{q_{m^*}})$ does not get extracted from the commitment C in signing queries for more than $|\mathcal{S}_{\text{hon}}| - 1$ honest signing parties within one signing session, where $\mathcal{S}_{\text{hon}}$ is the subset of signing set $\mathcal{S}$ that consists of all honest signing parties.

Game 3 samples a guess $q_{m^*} \xleftarrow{\$} [Q_M]$ in the beginning and checks whether the above conditions hold for guess q_{m^*} once it has received the forgeries from $\mathcal{A}$. If the guess was wrong, it aborts. If $\mathcal{A}$ wins Game 3, there must indeed be a message $m_{q_{m^*}} := m_j^*$ with $j \in [\ell]$ and $q_{m^*} \in [Q_M]$ such that conditions 1 and 2 hold: As $\mathcal{A}$ wins, at most $\ell - 1$ signing queries were made for $|\mathcal{S}_{\text{hon}}|$ honest signing parties and $\mathcal{A}$ must have returned ℓ pairwise distinct message-signature pairs $(m_j^*, \sigma_j^*)_{j \in [\ell]}$. Since Game 2 excludes collisions for H_M , all ℓ hashed messages $\bar{m}_j^* = H_M(m_j^*)$ must be pairwise distinct as well. This means, at most $\ell - 1$ of the hashed messages $\bar{m}_j^*$ can have been extracted in signing sessions in which the signing oracle was called for all $|\mathcal{S}_{\text{hon}}|$ honest signing parties and there is at least one message m_j^* for which this is not the case.

As the probability that the game guesses q_{m^*} right is $1/Q_M$, we have

$$\varepsilon_2 \leq Q_M \cdot \varepsilon_3.$$

Subsequently, we use the notation $\bar{m}^* := H_M(m_{q_{m^*}})$. The game samples $\bar{m}^* \xleftarrow{\$} \mathbb{Z}_p$ at the start of the game and sets $\bar{m}^* := H_M(m_{q_{m^*}})$ during the q_{m^*} -th query to random oracle H_M . We may therefore assume that the game knows $\bar{m}^*$ from the beginning, if $\mathcal{A}$ succeeds.

Game 4 (Abort if $w_{i,j} = w_{k,l}$ for any $\{i, j\} \neq \{k, l\}$). Game 4 introduces an additional abort condition and now aborts if for any two signers i, j the shared key $w_{i,j}$ is equal to the shared key $w_{k,l}$ of any two other signers k, l with $\{i, j\} \neq \{k, l\}$. The probability of such a collision occurring can be upper bounded by a standard birthday bound. It thus holds that

$$|\varepsilon_3 - \varepsilon_4| \leq \frac{(n^2 - n)^2}{4 \cdot |\mathcal{K}_{\text{msk}}|}.$$

Game 5 (Replace F_{msk} with random functions for honest signer keys). Game 5 replaces each instance of $F_{\text{msk}, w_{i,j}}$ that uses a key $w_{i,j}$ shared between two honest users i and j with a truly random function. This is possible as $\mathcal{A}$ does not know any of the keys that are shared between two honest users and all shared keys are distinct. Thus, the values computed through $F_{\text{msk}, w_{i,j}}$ look uniformly random to $\mathcal{A}$ assuming PRF security. More formally, for any such $F_{\text{msk}, w_{i,j}}$ that is replaced by a truly random function an adversary distinguishing between the two would break PRF security of F_{msk} . Thus, by applying a standard reduction on PRF security of F_{msk} for each of the instances of $F_{\text{msk}, w_{i,j}}$ that uses a key $w_{i,j}$ shared between two honest users, we get the following bound:

$$|\varepsilon_4 - \varepsilon_5| \leq \frac{n^2 - n}{2} \cdot \text{AdvPRF}_{\mathcal{A}}^{F_{\text{msk}}}(\lambda).$$

Game 6 (Send random σ_i shares for $\bar{m}^*$). In Game 6, queries asking for the signature share of signer k for hashed message $\bar{m}^*$ get answered in the following way. First, the game initializes a table $R[\cdot] = \perp$ which later stores the answers for each signing query. On each signing query, the game first checks in table R , if $\mathcal{O}^{\text{Sign}}$ already gave out a partial signature for signer k , context data ctx and user input $\text{pm}_0^U = (\mathcal{S}, C, \pi_{\text{Ped}})$, i.e., if $R[k, \text{ctx}, \text{pm}_0^U] \neq \perp$. If this was the case, it returns the same value $R[k, \text{ctx}, \text{pm}_0^U]$ as before. Otherwise, it returns random values $\sigma_k^* = (\sigma_{k,0}^*, \sigma_{k,1}^*) \xleftarrow{\$} \mathbb{G}_1^2$ and sets $R[k, \text{ctx}, \text{pm}_0^U] \leftarrow \sigma_k^*$. Apart from this change, Game 6 is equivalent to Game 5.

By Game 5, all values of F_{msk} computed with keys $w_{i,j}$ shared between two honest signers are replaced by values computed by truly random functions and thus are uniformly random. Therefore, by construction, each value $\sigma_{k,0}^* = \lambda_k \text{bb-sk}_k + r_k(H_1 + C) + \Delta_k$ with $\Delta_k \leftarrow \sum_{j \in \mathcal{S}, j < k} F_{\text{msk}, w_{k,j}}(\text{ssid}) - \sum_{j \in \mathcal{S}, j > k} F_{\text{msk}, w_{k,j}}(\text{ssid})$ is the sum of $|\mathcal{S}_{\text{hon}}| - 1$ random values, where $\mathcal{S}_{\text{hon}}$ is the subset of signing set $\mathcal{S}$ that consists of all honest signing parties. By definition of OMUF-3 and by the abort condition introduced in Game 3, we know that the adversary will make at most $|\mathcal{S}_{\text{hon}}| - 1$ signature share queries for $\bar{m}^*$, keeping each $\sigma_{k,0}^*$ given out to the adversary for $\bar{m}^*$ random. As the handed out $\sigma_{k,0}^*$ values are random and independent from the $\sigma_{k,1}^*$ values, each $\sigma_{k,1}^*$ handed out for $\bar{m}^*$ is random as well. By storing the returned values σ_k^* in table R , consistency is ensured across multiple calls with the same input. Thus, the distribution of the σ_k^* shares is the same in both Games 5 and 6.

$$\varepsilon_5 = \varepsilon_6.$$

Reduction to CDH. We now construct an adversary $\mathcal{B}$ to the CDH game running the adversary $\mathcal{A}$ of Game 4 as a subroutine. From the CDH challenger, $\mathcal{B}$ receives a challenge tuple $(G_1, G_2, A_1, A_2, B_1, B_2)$, while $\mathcal{A}$ sends us the corruption set $\mathcal{C}$. If $|\mathcal{C}| > t$, $\mathcal{B}$ aborts. Using the same technique also applied in the final reduction to CDH in the OMUF-proof of scheme TBS, $\mathcal{B}$ then simulates a secret sharing of B_1 and obtains t simulated values β_i as well as n simulated values B_i . From this, $\mathcal{B}$ computes t values $\text{bb-sk}_i = \beta_i A_1 = \beta_i \alpha G_1$. $\mathcal{B}$ then generates all shared keys $\{w_{i,j}\}_{i,j \in [n], i \neq j}$ and all personal keys $\{w_i\}_{i \in [n]}$ just as in the previous games and sets $\text{sk}_i = (\text{bb-sk}_i, \{w_{i,j}\}_{j \in [n]}, w_i)$ for all $i \in [\mathcal{C}]$. Finally, $\mathcal{B}$ samples $\delta \xleftarrow{\$} \mathbb{Z}_p$, sets $H_1 = \delta G_1 - \bar{m}^* A_1$, $H_2 = \delta G_2 - \bar{m}^* A_2$ (such that implicitly $\gamma = -\bar{m}^* \cdot \alpha + \delta$), $V = e(A_1, B_2) = \alpha \beta(G_1, G_2)$ as well as $\text{vk} = (A_1, A_2, H_1, H_2, V)$ and sends $\{\text{sk}_i\}_{i \in [\mathcal{C}]}$ as well as vk to $\mathcal{A}$.

Queries to the signing oracle for an honest signer i and messages m with $\bar{m} \neq \bar{m}^*$ are simulated by setting $\sigma_{i,1} = \tilde{r}_i G_1 - \frac{1}{\Delta \bar{m}} \lambda_i B_i$ and $\sigma_{i,0} = \Delta \bar{m} \tilde{r}_i A_1 + (t + \delta) \tilde{r}_i G_1 - \frac{t + \delta}{\Delta \bar{m}} \lambda_i B_i$, where $\tilde{r}_i \xleftarrow{\$} \mathbb{Z}_p$, $\Delta \bar{m} = \bar{m} - \bar{m}^*$, λ_k is the Lagrange coefficient of signer i and t is part of the extracted witness $w = (\bar{m}, t)$. This yields the correct distribution as

$$\sigma_{i,0} = \Delta \bar{m} \tilde{r}_i A_1 + (t + \delta) \tilde{r}_i G_1 - \frac{t + \delta}{\Delta \bar{m}} \lambda_i B_i \quad (9)$$

$$= \lambda_i \beta_i \alpha G_1 - \lambda_i \beta_i \frac{\Delta \bar{m}}{\Delta \bar{m}} A_1 + \Delta \bar{m} \tilde{r}_i A_1 + (t + \delta) \tilde{r}_i G_1 - \frac{t + \delta}{\Delta \bar{m}} \lambda_i \beta_i G_1 \quad (10)$$

$$= \lambda_i \beta_i \alpha G_1 + (\tilde{r}_i - \frac{\lambda_i \beta_i}{\Delta \bar{m}})(\Delta \bar{m} A_1 + (t + \delta) G_1) \quad (11)$$

$$= \lambda_i \beta_i \alpha G_1 + (\tilde{r}_i - \frac{\lambda_i \beta_i}{\Delta \bar{m}})(\bar{m} A_1 + t G_1 - \bar{m}^* A_1 + \delta G_1) \quad (12)$$

$$= \lambda_i \beta_i \alpha G_1 + r_i (C + H_1), \quad (13)$$

where $r_i = \tilde{r}_i - \frac{\lambda_i \beta_i}{\Delta \bar{m}}$. Queries to the signing oracle for an honest signer i and messages m with $\bar{m} = \bar{m}^*$ are answered with random values just as in Game 4.

Finally, when $\mathcal{A}$ returns its ℓ forgeries $\{m_j, \sigma_j\}_{j \in [\ell]}$, $\mathcal{B}$ computes the CDH solution through $\sigma_{j^*,0} - \delta \sigma_{j^*,1}$ (where j^* such that $H_M(m_{j^*}) = \bar{m}^*$) and sends this value to the CDH challenger.

$\mathcal{B}$ always wins the CDH game, whenever $\mathcal{A}$ wins Game 4, as

$$e(\sigma_{j^*,0}, G_2) = V \cdot +e(\sigma_{j^*,1}, \bar{m}^* U_2 + H_2) \quad (14)$$

$$\implies e(\sigma_{j^*,0}, G_2) = V \cdot +e(\sigma_{j^*,1}, \bar{m}^* A_2 + \delta G_2 - \bar{m}^* A_2) \quad (15)$$

$$\implies e(\sigma_{j^*,0}, G_2) = V \cdot +e(\sigma_{j^*,1}, \delta G_2) \quad (16)$$

$$\implies e(\sigma_{j^*,0} - \delta \sigma_{j^*,1}, G_2) = V. \quad (17)$$

Thus,

$$\varepsilon_4 \leq \text{AdvCDH}_{\mathcal{B}}^{\text{PGen}}(\lambda).$$

The lemma directly follows by putting together all equalities and inequalities obtained through the above game hops.

8.3 Blindness of pTBS

We give a formal proof of Theorem 9.

Proof. Game 0 (Honest). We start with the honest threshold blindness game: the game first invokes $\text{par} \leftarrow \text{pTBS.Setup}(1^\lambda)$, samples bit $b \xleftarrow{\$} \{0,1\}$ and sets up the signing oracles $\mathcal{O}^{\text{User}_0}$ and $\mathcal{O}^{\text{User}_1}$. The game then calls $b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{User}_0}, \mathcal{O}^{\text{User}_1}}()$. If $b' = b$, the game outputs 1, otherwise it outputs 0. By definition, we have

$$\varepsilon_0 := \text{AdvPBlind}_{\mathcal{A}}(\lambda).$$

Game 1 (Self-sign $\bar{m}$). Game 1 modifies $\mathcal{O}^{\text{User}_1}$: It still adds up all signature shares $\sigma^* = \sum_{i \in \mathcal{S}} (\sigma_{i,0}^*, \sigma_{i,1}^*)$ and performs the check of well-formedness on σ^* , but then brute forces bb-sk , samples $r \xleftarrow{\$} \mathbb{Z}_p$ and sets $\sigma = (\text{bb-sk} + r(\bar{m}U_1 + H_2), rG_1)$.

Indeed, this does not change the distribution of σ as for every σ^* that passes the well-formedness check in game 0, it holds that

$$\begin{aligned} e(\sigma_0^*, G_2) &= V + e(\sigma_1^*, \bar{m}U_2 + tG_2 + H_2) \\ &= \alpha\beta e(G_1, G_2) + (\bar{m}\alpha + t + \gamma) \cdot e(\sigma_1^*, G_2) \end{aligned}$$

If we rewrite σ_1^* as $\sigma_1^* = r^*G_1$, we obtain that

$$\begin{aligned} \sigma_0^* &= (\alpha\beta + r^*(\bar{m}\alpha + t + \gamma))G_1 \\ &= \text{bb-sk} + r^*(\bar{m}C + H_1) \end{aligned}$$

This means that the signature σ given out by game 0 can be rewritten as:

$$\begin{aligned} (\sigma_0, \sigma_1) &= (\sigma_0^* - t\sigma_1^* + r'(\bar{m}U_1 + H_1), \sigma_1^* + r'G_1) \\ &= (\text{bb-sk} + r^*(\bar{m}C + H_1) - t\sigma_1^* + r'(\bar{m}U_1 + H_1), r^*G_1 + r'G_1) \\ &= (\text{bb-sk} + (r^* + r')(\bar{m}C + H_1), (r^* + r')G_1) \end{aligned}$$

As value r' follows a uniformly random distribution and remains statistically hidden from $\mathcal{A}$, the signatures σ follow the same distribution in both games. Thus, it holds:

$$\varepsilon_0 = \varepsilon_1.$$

While the game becomes inefficient, this and the following steps are purely statistical.

Game 2 (Simulate π_{Ped}). In Game 2, $\mathcal{O}^{\text{User}_0}$ simulates the proofs $\pi_{\text{Ped},0}$ and $\pi_{\text{Ped},1}$. To do so, for $i \in \{0,1\}$, it brute-forces openings $\hat{t}_i$ for $\bar{m} = 0$ and computes proofs $\pi_{\text{Ped},i}$ for this. This is possible as for any Pedersen Commitment $C \in \mathbb{G}$ one can always compute a valid opening for any value v by brute-forcing t .

Clearly, any $\mathcal{A}$ that can distinguish between the proofs $\pi_{\text{Ped},i}$ from Game 1 and the proofs $\pi_{\text{Ped},i}$ as they are computed in Game 2, breaks witness indistinguishability of NIPS. Therefore, through a simple reduction, one can construct an adversary $\mathcal{A}_{\text{WI}}$ breaking witness indistinguishability of NIPS that runs $\mathcal{A}$ as a subroutine. By playing the witness indistinguishability game once for $\pi_{\text{Ped},0}$ and once for $\pi_{\text{Ped},1}$, we get:

$$|\varepsilon_1 - \varepsilon_2| \leq 2 \cdot \text{AdvWI}_{\mathcal{A}_{\text{WI}}}^{\text{NIPS}}(Q_{\text{Ped}}, \lambda).$$

Game 3 ($C \xleftarrow{\$} \mathbb{G}$). In Game 3, $\mathcal{O}^{\text{User}_0}$ samples $C_0, C_1 \xleftarrow{\$} \mathbb{G}$ instead of computing $C_0 := \bar{m}_b U_1 + t_0 G_1$ and $C_1 := \bar{m}_{1-b} U_1 + t_1 G_2$. By Game 2, the values $t_0, t_1 \xleftarrow{\$} \mathbb{Z}_p$ are used only in the computation of the values C_0 and C_1 , respectively, and are not used anywhere else afterwards. Thus, already in Game 2, the distributions of C_0 and C_1 are uniformly random. Therefore, we have

$$\varepsilon_8 = \varepsilon_9.$$

Wrapping up. By Game 3, the signatures σ_i are computed only after the interaction with the signer has finished and in a way that is completely independent from the values received by the signer during the interaction. Therefore, the advantage of the adversary in distinguishing between σ_0 and σ_1 is 0.

$$\varepsilon_3 = 0.$$

By putting together the bounds obtained through the above sequence of game hops, the statement in Theorem 9 directly follows.

References

1. Abe, M.: A secure three-move blind signature scheme for polynomially many signatures. In: Pfitzmann, B. (ed.) EUROCRYPT 2001. LNCS, vol. 2045, pp. 136–151. Springer, Berlin, Heidelberg (May 2001). https://doi.org/10.1007/3-540-44987-6_9
2. Abe, M., Fuchsbaauer, G., Groth, J., Haralambiev, K., Ohkubo, M.: Structure-preserving signatures and commitments to group elements. In: Rabin, T. (ed.) CRYPTO 2010. LNCS, vol. 6223, pp. 209–236. Springer, Berlin, Heidelberg (Aug 2010). https://doi.org/10.1007/978-3-642-14623-7_12
3. Abe, M., Jutla, C.S., Ohkubo, M., Roy, A.: Improved (almost) tightly-secure simulation-sound QA-NIZK with applications. In: Peyrin, T., Galbraith, S. (eds.) ASIACRYPT 2018, Part I. LNCS, vol. 11272, pp. 627–656. Springer, Cham (Dec 2018). https://doi.org/10.1007/978-3-030-03326-2_21
4. Abe, M., Okamoto, T.: Provably secure partially blind signatures. In: Bellare, M. (ed.) CRYPTO 2000. LNCS, vol. 1880, pp. 271–286. Springer, Berlin, Heidelberg (Aug 2000). https://doi.org/10.1007/3-540-44598-6_17
5. Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In: Juels, A., Wright, R.N., De Capitani di Vimercati, S. (eds.) ACM CCS 2006. pp. 390–399. ACM Press (Oct / Nov 2006). <https://doi.org/10.1145/1180405.1180453>
6. Benhamouda, F., Lepoint, T., Loss, J., Orrù, M., Raykova, M.: On the (in)security of ROS. In: Canteaut, A., Standaert, F.X. (eds.) EUROCRYPT 2021, Part I. LNCS, vol. 12696, pp. 33–53. Springer, Cham (Oct 2021). https://doi.org/10.1007/978-3-030-77870-5_2
7. Blazy, O., Fuchsbaauer, G., Pointcheval, D., Vergnaud, D.: Short blind signatures. *Journal of computer security* **21**(5), 627–661 (2013)
8. Boldyreva, A.: Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In: Desmedt, Y. (ed.) PKC 2003. LNCS, vol. 2567, pp. 31–46. Springer, Berlin, Heidelberg (Jan 2003). https://doi.org/10.1007/3-540-36288-6_3
9. Boneh, D., Boyen, X.: Efficient selective-ID secure identity based encryption without random oracles. In: Cachin, C., Camenisch, J. (eds.) EUROCRYPT 2004. LNCS, vol. 3027, pp. 223–238. Springer, Berlin, Heidelberg (May 2004). https://doi.org/10.1007/978-3-540-24676-3_14
10. Boneh, D., Boyen, X.: Short signatures without random oracles. In: Cachin, C., Camenisch, J. (eds.) EUROCRYPT 2004. LNCS, vol. 3027, pp. 56–73. Springer, Berlin, Heidelberg (May 2004). https://doi.org/10.1007/978-3-540-24676-3_4
11. Brandt, N., Hofheinz, D., Klooß, M., Reichle, M.: Tightly-secure blind signatures in pairing-free groups. *Cryptology ePrint Archive, Report 2024/2075* (2024), <https://eprint.iacr.org/2024/2075>
12. Chairattana-Apirom, R., Hanzlik, L., Loss, J., Lysyanskaya, A., Wagner, B.: PI-cut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part III. LNCS, vol. 13509, pp. 3–31. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15982-4_1
13. Chairattana-Apirom, R., Tessaro, S., Zhu, C.: Pairing-free blind signatures from CDH assumptions. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part I. LNCS, vol. 14920, pp. 174–209. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68376-3_6
14. Chaum, D.: Blind signatures for untraceable payments. In: Chaum, D., Rivest, R.L., Sherman, A.T. (eds.) CRYPTO’82. pp. 199–203. Plenum Press, New York, USA (1982). https://doi.org/10.1007/978-1-4757-0602-4_18
15. Cramer, R., Damgård, I., Schoenmakers, B.: Proofs of partial knowledge and simplified design of witness hiding protocols. In: Desmedt, Y. (ed.) CRYPTO’94. LNCS, vol. 839, pp. 174–187. Springer, Berlin, Heidelberg (Aug 1994). https://doi.org/10.1007/3-540-48658-5_19
16. Crites, E.C., Komlo, C., Maller, M., Tessaro, S., Zhu, C.: Snowblind: A threshold blind signature in pairing-free groups. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part I. LNCS, vol. 14081, pp. 710–742. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38557-5_23
17. Del Pino, R., Katsumata, S., Maller, M., Mouhartem, F., Prest, T., Saarinen, M.J.O.: Threshold racoon: Practical threshold signatures from standard lattice assumptions. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part II. LNCS, vol. 14652, pp. 219–248. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58723-8_8
18. Fuchsbaauer, G., Hanser, C., Kamath, C., Slamanig, D.: Practical round-optimal blind signatures in the standard model from weaker assumptions. In: Zikas, V., De Prisco, R. (eds.) SCN 16. LNCS, vol. 9841, pp. 391–408. Springer, Cham (Aug / Sep 2016). https://doi.org/10.1007/978-3-319-44618-9_21
19. Fuchsbaauer, G., Hanser, C., Slamanig, D.: Practical round-optimal blind signatures in the standard model. In: Gennaro, R., Robshaw, M.J.B. (eds.) CRYPTO 2015, Part II. LNCS, vol. 9216, pp. 233–253. Springer, Berlin, Heidelberg (Aug 2015). https://doi.org/10.1007/978-3-662-48000-7_12
20. Fuchsbaauer, G., Plouviez, A., Seurin, Y.: Blind Schnorr signatures and signed ElGamal encryption in the algebraic group model. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part II. LNCS, vol. 12106, pp. 63–95. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45724-2_3

21. Fuchsbauer, G., Wolf, M.: Concurrently secure blind Schnorr signatures. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part II. LNCS, vol. 14652, pp. 124–160. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58723-8_5
22. Ghadafi, E.: Efficient round-optimal blind signatures in the standard model. In: Kiayias, A. (ed.) FC 2017. LNCS, vol. 10322, pp. 455–473. Springer, Cham (Apr 2017). https://doi.org/10.1007/978-3-319-70972-7_26
23. Hanzlik, L., Loss, J., Wagner, B.: Rai-choo! Evolving blind signatures to the next level. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 753–783. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30589-4_26
24. Hauck, E., Kiltz, E., Loss, J.: A modular treatment of blind signatures from identification schemes. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part III. LNCS, vol. 11478, pp. 345–375. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17659-4_12
25. Kastner, J., Loss, J., Xu, J.: The Abe-Okamoto partially blind signature scheme revisited. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part IV. LNCS, vol. 13794, pp. 279–309. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22972-5_10
26. Kastner, J., Loss, J., Xu, J.: On pairing-free blind signature schemes in the algebraic group model. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022, Part II. LNCS, vol. 13178, pp. 468–497. Springer, Cham (Mar 2022). https://doi.org/10.1007/978-3-030-97131-1_16
27. Katsumata, S., Reichle, M., Sakai, Y.: Practical round-optimal blind signatures in the ROM from standard assumptions. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part II. LNCS, vol. 14439, pp. 383–417. Springer, Singapore (Dec 2023). https://doi.org/10.1007/978-981-99-8724-5_12
28. Klooß, M., Reichle, M.: Blind signatures from proofs of inequality. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part VI. LNCS, vol. 16005, pp. 157–189. Springer, Cham (Aug 2025). https://doi.org/10.1007/978-3-032-01887-8_6
29. Klooß, M., Reichle, M., Wagner, B.: Practical blind signatures in pairing-free groups. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part I. LNCS, vol. 15484, pp. 363–395. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0875-1_12
30. Klooß, M., Lai, R.W.F., Reichle, M.: Blind signatures from arguments of inequality. Cryptology ePrint Archive, Paper 2025/1886 (2025), <https://eprint.iacr.org/2025/1886>
31. Kuchta, V., Manulis, M.: Rerandomizable threshold blind signatures. In: Revised Selected Papers of the 6th International Conference on Trusted Systems - Volume 9473. p. 70–89. INTRUST 2014, Springer-Verlag, Berlin, Heidelberg (2014). https://doi.org/10.1007/978-3-319-27998-5_5, https://doi.org/10.1007/978-3-319-27998-5_5
32. Lehmann, A., Nazarian, P., Özbay, C.: Stronger security for threshold blind signatures. In: Fehr, S., Fouque, P.A. (eds.) EUROCRYPT 2025, Part II. LNCS, vol. 15602, pp. 335–364. Springer, Cham (May 2025). https://doi.org/10.1007/978-3-031-91124-8_12
33. Niot, G., Reichle, M., Takemure, K.: Adaptively-secure three-round threshold schnorr from DL. Cryptology ePrint Archive, Paper 2025/1941 (2025), <https://eprint.iacr.org/2025/1941>
34. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum, J. (ed.) CRYPTO'91. LNCS, vol. 576, pp. 129–140. Springer, Berlin, Heidelberg (Aug 1992). https://doi.org/10.1007/3-540-46766-1_9
35. Pointcheval, D., Stern, J.: Security arguments for digital signatures and blind signatures. Journal of Cryptology **13**(3), 361–396 (Jun 2000). <https://doi.org/10.1007/s001450010003>
36. Reichle, M., Reinke, Z.: Threshold blind signatures from CDH. Cryptology ePrint Archive, Paper 2025/1798 (2025), <https://eprint.iacr.org/2025/1798>
37. Shamir, A.: How to share a secret. Communications of the Association for Computing Machinery **22**(11), 612–613 (Nov 1979). <https://doi.org/10.1145/359168.359176>
38. Tessaro, S., Zhu, C.: Short pairing-free blind signatures with exponential security. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part II. LNCS, vol. 13276, pp. 782–811. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_27
39. Vo, D.L., Zhang, F., Kim, K.: A new threshold blind signature scheme from pairings. In: SCIS2003. pp. 233–238. SCIS (2003)
40. Wagner, D.: A generalized birthday problem. In: Yung, M. (ed.) CRYPTO 2002. LNCS, vol. 2442, pp. 288–303. Springer, Berlin, Heidelberg (Aug 2002). https://doi.org/10.1007/3-540-45708-9_19